

Appendix H: ResNet50V2 Tests 3_5

In [1]: `!nvidia-smi`

```
Fri Mar 14 21:02:07 2025
+-----+
+-----+
| NVIDIA-SMI 550.54.15                  Driver Version: 550.54.15          CUDA Version:
12.4    |
|-----+-----+-----+-----+
+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Unc
orr. ECC |
| Fan   Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Co
mpute M. |
|              |                    |                      |
MIG M. |
|=====+=====+=====+=====+
=====|
|    0   NVIDIA A100-SXM4-40GB                Off |  00000000:00:04.0 Off |
0|
| N/A    30C    P0              42W /  400W |      0MiB /  40960MiB |      0%
Default |
|              |                    |                      |
Disabled |
+-----+-----+-----+-----+
+-----+
+-----+
| Processes:
|
| GPU   GI    CI          PID    Type    Process name                        GP
U Memory |
|       ID    ID                                   |
age      |
|=====+=====+=====+=====+
=====|
| No running processes found
|
+-----+-----+-----+-----+
+-----+
```

In [2]: `#!/pip install tensorflow`
`import tensorflow as tf`
`print(tf.config.list_physical_devices('GPU'))`

[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

Import libraries

In [3]: `import tensorflow as tf # Access to models, datasets and training`
`from tensorflow.keras.datasets import cifar100 # Access to CIFAR-100`
`from tensorflow.keras.applications import ResNet50 # Access to pre-trained model`
`from tensorflow.keras import layers, models, optimizers # Access to building blo`

```

from tensorflow.keras.models import load_model # Ability to load best_saved mode
from tensorflow import keras # Access to stuff for model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
from sklearn.model_selection import train_test_split # To help split dataset up
from sklearn.metrics import confusion_matrix, classification_report # Analysis of model
import numpy as np # Manipulate data
import pandas as pd # Statistical analysis of data
import seaborn as sns
import matplotlib.pyplot as plt # Plot data

```

Function definitions

```

In [4]: def display_imgs(imgs, labels):
        plt.subplots(figsize=(10,10))
        for i in range(16):
            plt.subplot(4, 4, i+1)
            k = np.random.randint(0, imgs.shape[0])
            if i == 0:
                print(f"labels[{k}].shape: {labels[k].shape}")
                print(f"imgs[{k}].shape: {imgs[k].shape}")
            plt.imshow(imgs[k])
            #plt.title(labels[k])
            plt.axis('off')
        plt.tight_layout()
        plt.show()

def class_distrib(y, labels_names, dataset_name):
    counts = pd.DataFrame(data=y).value_counts().sort_index()
    #print(f"counts:\n{counts}")
    fig, ax = plt.subplots(figsize=(20,10))
    ax.bar(labels_names, counts)
    ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
    plt.title(f"Distribution of '{dataset_name}' Dataset", fontsize=25)
    plt.grid()
    plt.tight_layout()
    plt.show()

def augment_dataset(x, y):
    x = tf.image.resize(x, (224, 224)) # Resize images
    x = tf.image.random_flip_left_right(x) # Random horizontal flip
    x = tf.image.random_brightness(x, max_delta=0.2) # Adjust brightness
    x = tf.image.random_contrast(x, lower=0.8, upper=1.2) # Adjust contrast
    y = tf.squeeze(tf.one_hot(y, depth=100, dtype=tf.float32)) # One-hot encoding
    return x, y

def plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc):
    # Plot Training and Validation Loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_loss, label='Training Loss')
    plt.plot(epochs, val_loss, label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.title('Training and Validation Loss')
    plt.legend()
    plt.grid()

```

```

# Plot Training and Validation Accuracy
plt.subplot(1, 2, 2)
plt.plot(epochs, train_acc, label='Training Accuracy')
plt.plot(epochs, val_acc, label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy')
plt.legend()
plt.grid()

plt.tight_layout()
plt.show()

```

Dropout(0.6), weight_decay(3e-2)

```

In [5]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
        X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
        print(f"X_temp.shape: {X_temp.shape}\n")

        # Split temp data into equal validation (4000) and testing (4000) data
        X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
        print(f"X_temp_val.shape: {X_temp_val.shape}")
        print(f"y_temp_val.shape: {y_temp_val.shape}")
        print(f"X_temp_test.shape: {X_temp_test.shape}")
        print(f"y_temp_test.shape: {y_temp_test.shape}\n")

        # Split test data into validation (5000) and testing (5000)
        X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

        # Add temp_val to validation (9000) and temp_test to testing (9000) to get a
        X_val = np.concatenate((X_val, X_temp_val), axis=0)
        y_val = np.concatenate((y_val, y_temp_val), axis=0)
        X_test = np.concatenate((X_test, X_temp_test), axis=0)
        y_test = np.concatenate((y_test, y_temp_test), axis=0)

        print(f"X_train.shape: {X_train.shape}")
        print(f"y_train.shape: {y_train.shape}")
        print(f"X_val.shape: {X_val.shape}")
        print(f"y_val.shape: {y_val.shape}")
        print(f"X_test.shape: {X_test.shape}")
        print(f"y_test.shape: {y_test.shape}\n")

        display_imgs(X_train, y_train)

        # Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
        X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
        display_imgs(X_train, y_train)

        labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
                        'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
                        'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
                        'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
                        'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo

```

```

        'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea',
        'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
        'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
        'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
        'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
        'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
                .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                    tf.squeeze(tf.one_hot(y, depth=100, dtype=
                .batch(batch_size)
                .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

```

5/120

```

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                    batch_size=batch_size, callbacks=[early_stopping, checkp

#### <<<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1,len(history.history['loss'])+1)
train_loss = history.history['loss']
val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v1.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

```

```

#### <<<<<<<<Fine-Tune>>>>>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
               loss='categorical_crossentropy',
               metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                           .batch(batch_size)
                           .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset,
                              batch_size=batch_size, callbacks=[early_stopping_monitor])

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract Loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v1.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores: {np.average(test_f1_scores.numpy()*100):.2f}%")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions

```

```

X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist

```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz>

169001437/169001437 ————— 2s 0us/step

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

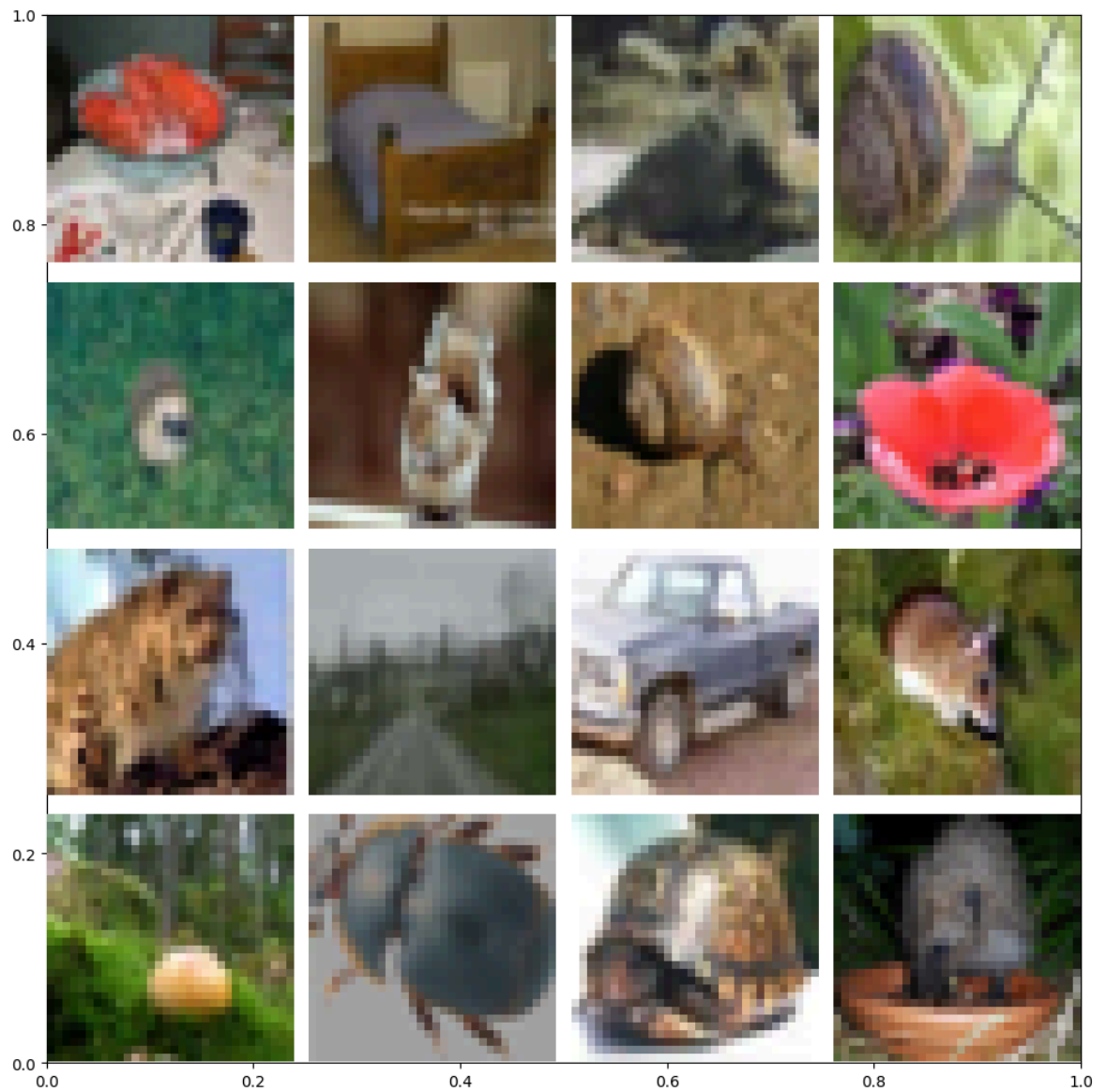
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

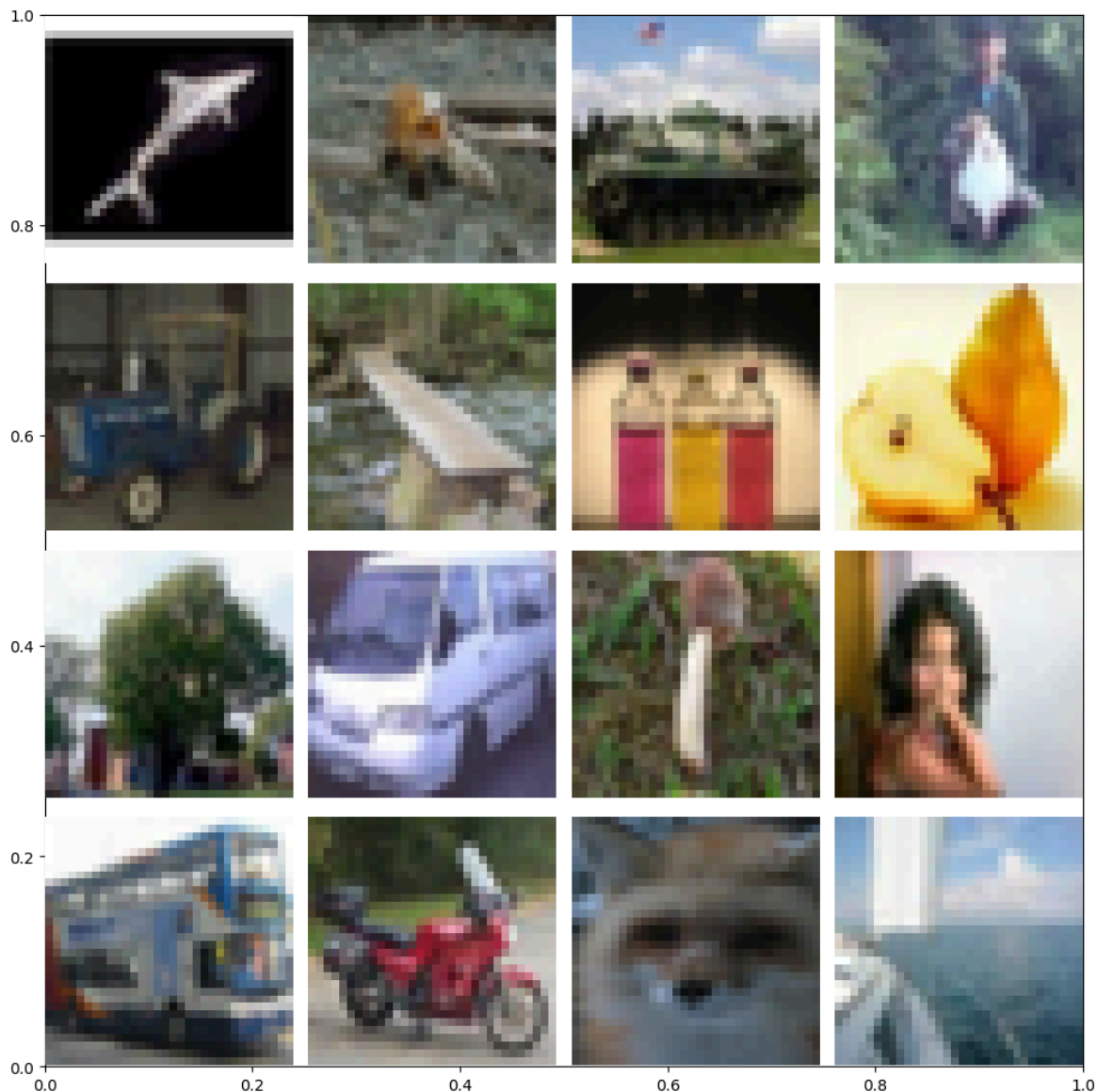
y_test.shape: (9000, 1)

labels[38819].shape: (1,)

imgs[38819].shape: (32, 32, 3)

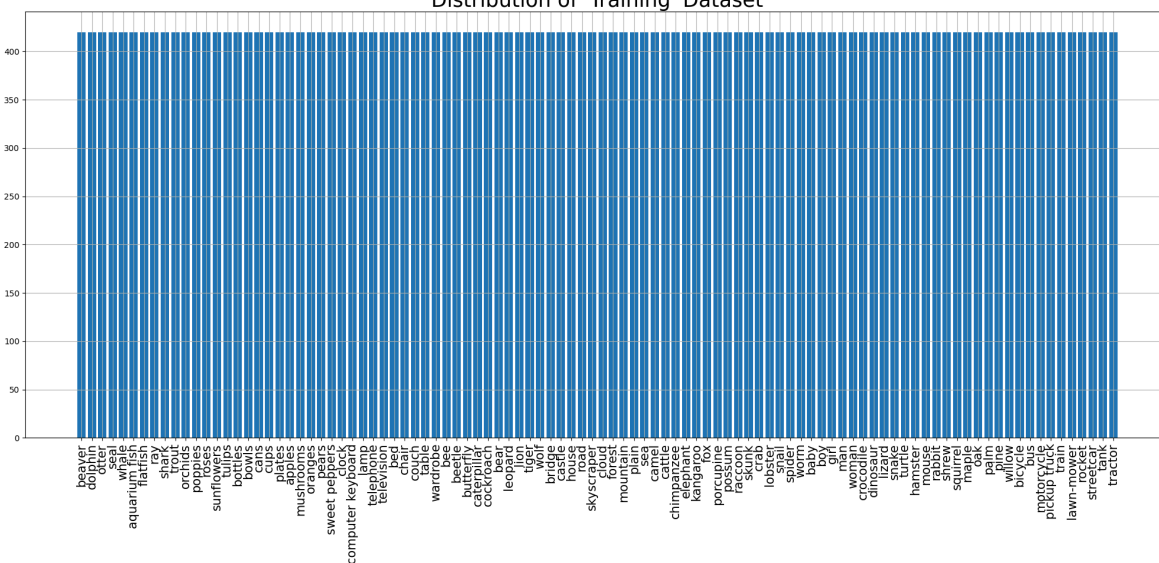


```
labels[8198].shape: (1,)  
imgs[8198].shape: (32, 32, 3)
```

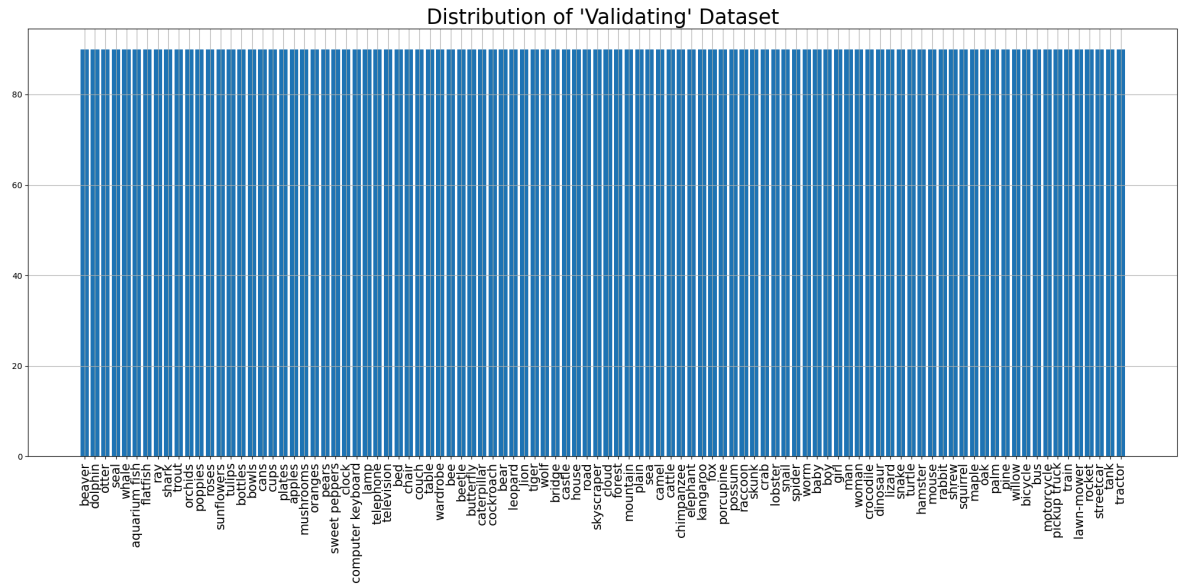


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

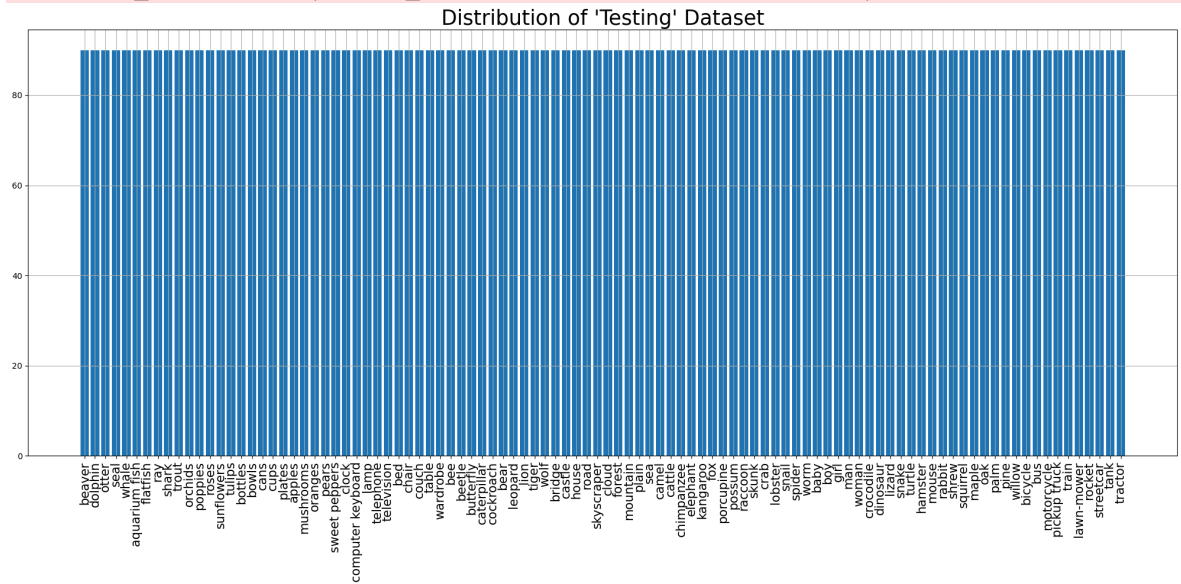
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50v2_weights_tf_dim_ordering_tf_kernels_notop.h5
94668760/94668760 ————— 0s 0us/step
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None , 2048)
dense (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_1 (Dense)	(None , 100)

◀  ▶

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— **0s** 34ms/step - accuracy: 0.3163 - f1_score: 0.3101 - loss: 2.9066 - precision: 0.6639
Epoch 1: val_loss improved from inf to 1.51737, saving model to best_model_v1.keras as
657/657 ————— **57s** 58ms/step - accuracy: 0.3165 - f1_score: 0.3102 - loss: 2.9057 - precision: 0.6640 - val_accuracy: 0.5844 - val_f1_score: 0.5802 - val_loss: 1.5174 - val_precision: 0.8125
Epoch 2/25
656/657 ————— **0s** 23ms/step - accuracy: 0.5261 - f1_score: 0.5223 - loss: 1.7312 - precision: 0.7454
Epoch 2: val_loss improved from 1.51737 to 1.40464, saving model to best_model_v1.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.5262 - f1_score: 0.5223 - loss: 1.7310 - precision: 0.7454 - val_accuracy: 0.6093 - val_f1_score: 0.6047 - val_loss: 1.4046 - val_precision: 0.8088
Epoch 3/25
657/657 ————— **0s** 23ms/step - accuracy: 0.5688 - f1_score: 0.5650 - loss: 1.5514 - precision: 0.7661
Epoch 3: val_loss improved from 1.40464 to 1.35960, saving model to best_model_v1.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.5688 - f1_score: 0.5650 - loss: 1.5513 - precision: 0.7661 - val_accuracy: 0.6207 - val_f1_score: 0.6186 - val_loss: 1.3596 - val_precision: 0.8009
Epoch 4/25
656/657 ————— **0s** 23ms/step - accuracy: 0.5946 - f1_score: 0.5912 - loss: 1.4372 - precision: 0.7789
Epoch 4: val_loss improved from 1.35960 to 1.34488, saving model to best_model_v1.keras
657/657 ————— **19s** 30ms/step - accuracy: 0.5946 - f1_score: 0.5913 - loss: 1.4371 - precision: 0.7789 - val_accuracy: 0.6187 - val_f1_score: 0.6144 - val_loss: 1.3449 - val_precision: 0.8009
Epoch 5/25
657/657 ————— **0s** 23ms/step - accuracy: 0.6120 - f1_score: 0.6088 - loss: 1.3405 - precision: 0.7798
Epoch 5: val_loss improved from 1.34488 to 1.31452, saving model to best_model_v1.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.6120 - f1_score: 0.6088 - loss: 1.3404 - precision: 0.7798 - val_accuracy: 0.6273 - val_f1_score: 0.6252 - val_loss: 1.3145 - val_precision: 0.7963
Epoch 6/25
657/657 ————— **0s** 23ms/step - accuracy: 0.6288 - f1_score: 0.6263 - loss: 1.2742 - precision: 0.7902
Epoch 6: val_loss did not improve from 1.31452
657/657 ————— **18s** 28ms/step - accuracy: 0.6288 - f1_score: 0.6263 - loss: 1.2742 - precision: 0.7902 - val_accuracy: 0.6326 - val_f1_score: 0.6306 - val_loss: 1.3202 - val_precision: 0.7880
Epoch 7/25
655/657 ————— **0s** 23ms/step - accuracy: 0.6411 - f1_score: 0.6380 - loss: 1.2241 - precision: 0.7955
Epoch 7: val_loss improved from 1.31452 to 1.30088, saving model to best_model_v1.keras
657/657 ————— **19s** 29ms/step - accuracy: 0.6411 - f1_score: 0.6380 - loss: 1.2240 - precision: 0.7955 - val_accuracy: 0.6336 - val_f1_score: 0.6321 - val_loss: 1.3009 - val_precision: 0.7878
Epoch 8/25
656/657 ————— **0s** 23ms/step - accuracy: 0.6534 - f1_score: 0.6512 - loss: 1.1700 - precision: 0.8026
Epoch 8: val_loss improved from 1.30088 to 1.30041, saving model to best_model_v1.keras

657/657 ————— **19s** 30ms/step - accuracy: 0.6535 - f1_score: 0.6513
 - loss: 1.1699 - precision: 0.8026 - val_accuracy: 0.6391 - val_f1_score: 0.6390
 - val_loss: 1.3004 - val_precision: 0.7896

Epoch 9/25

656/657 ————— **0s** 23ms/step - accuracy: 0.6675 - f1_score: 0.6644 -
 loss: 1.1145 - precision: 0.8065

Epoch 9: val_loss did not improve from 1.30041

657/657 ————— **19s** 28ms/step - accuracy: 0.6675 - f1_score: 0.6645
 - loss: 1.1144 - precision: 0.8065 - val_accuracy: 0.6369 - val_f1_score: 0.6365
 - val_loss: 1.3185 - val_precision: 0.7779

Epoch 10/25

656/657 ————— **0s** 23ms/step - accuracy: 0.6786 - f1_score: 0.6764 -
 loss: 1.0719 - precision: 0.8099

Epoch 10: val_loss did not improve from 1.30041

657/657 ————— **18s** 28ms/step - accuracy: 0.6786 - f1_score: 0.6765
 - loss: 1.0718 - precision: 0.8099 - val_accuracy: 0.6417 - val_f1_score: 0.6394
 - val_loss: 1.3157 - val_precision: 0.7791

Epoch 11/25

656/657 ————— **0s** 23ms/step - accuracy: 0.6847 - f1_score: 0.6820 -
 loss: 1.0374 - precision: 0.8116

Epoch 11: val_loss did not improve from 1.30041

657/657 ————— **19s** 28ms/step - accuracy: 0.6848 - f1_score: 0.6821
 - loss: 1.0374 - precision: 0.8116 - val_accuracy: 0.6407 - val_f1_score: 0.6407
 - val_loss: 1.3260 - val_precision: 0.7733

Epoch 12/25

655/657 ————— **0s** 23ms/step - accuracy: 0.7003 - f1_score: 0.6975 -
 loss: 0.9909 - precision: 0.8239

Epoch 12: val_loss did not improve from 1.30041

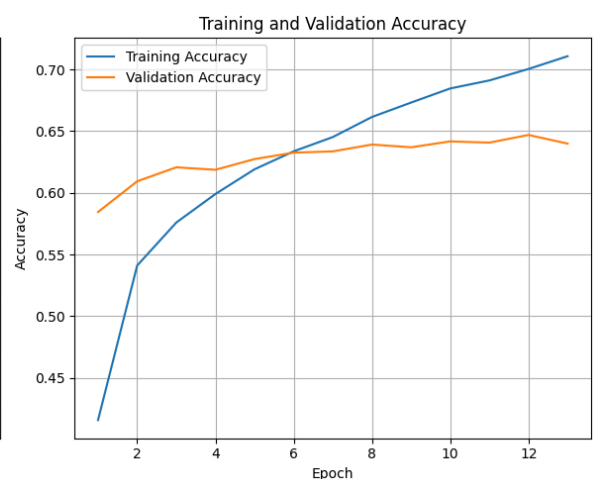
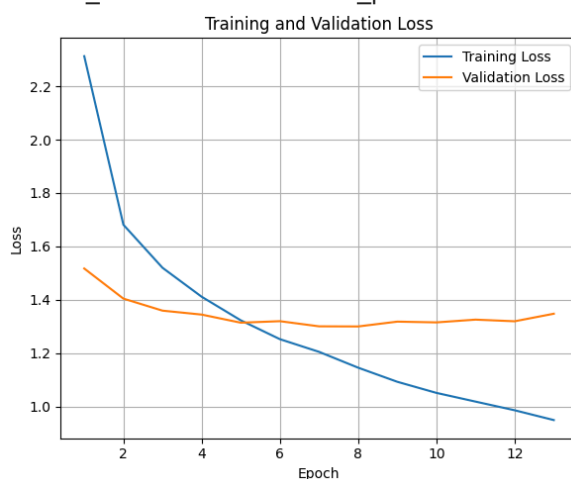
657/657 ————— **18s** 28ms/step - accuracy: 0.7003 - f1_score: 0.6975
 - loss: 0.9909 - precision: 0.8239 - val_accuracy: 0.6469 - val_f1_score: 0.6466
 - val_loss: 1.3198 - val_precision: 0.7764

Epoch 13/25

656/657 ————— **0s** 23ms/step - accuracy: 0.7086 - f1_score: 0.7063 -
 loss: 0.9645 - precision: 0.8292

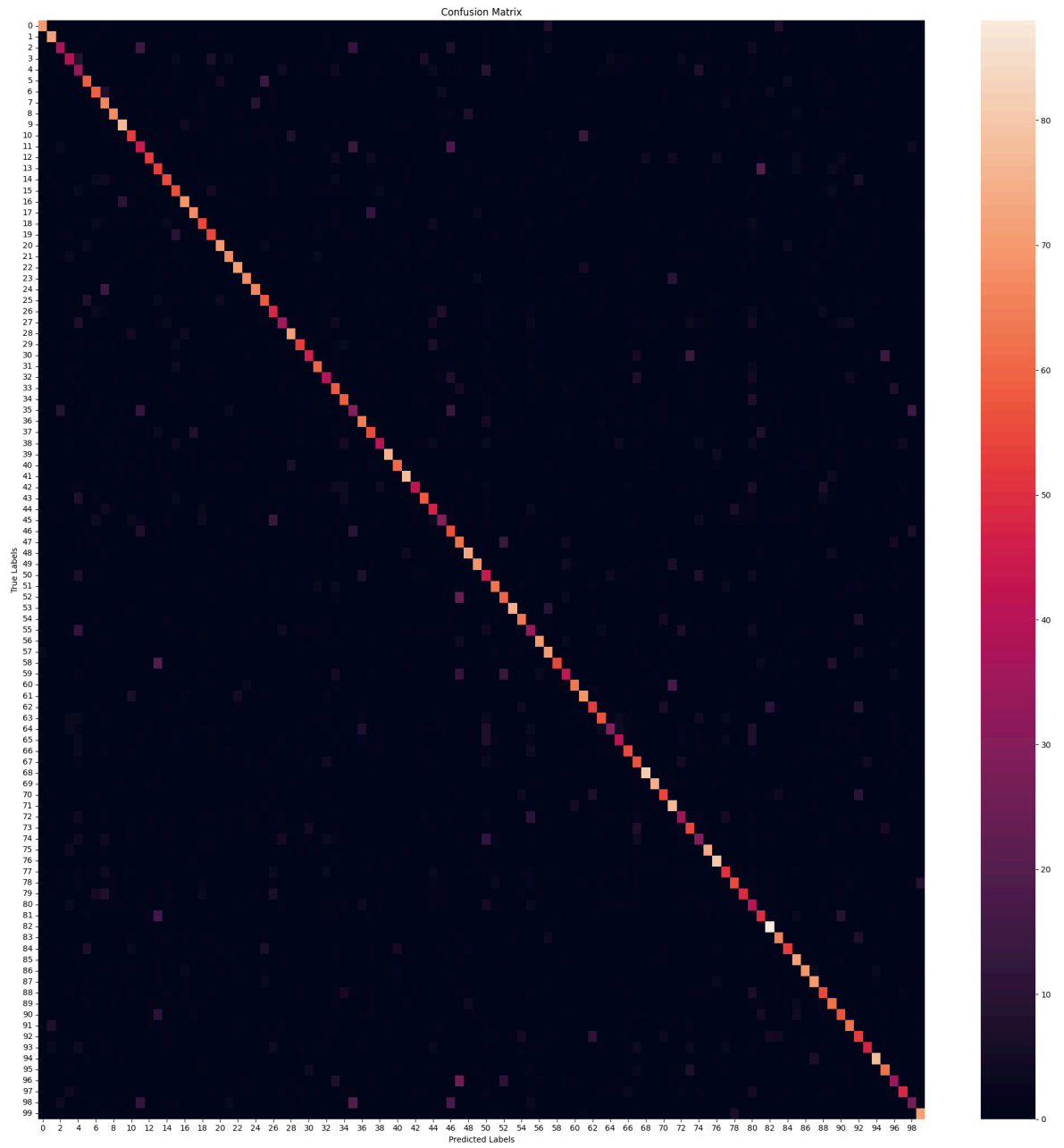
Epoch 13: val_loss did not improve from 1.30041

657/657 ————— **18s** 28ms/step - accuracy: 0.7086 - f1_score: 0.7063
 - loss: 0.9645 - precision: 0.8292 - val_accuracy: 0.6399 - val_f1_score: 0.6390
 - val_loss: 1.3481 - val_precision: 0.7672



141/141  **11s** 40ms/step - accuracy: 0.6264 - f1_score: 0.6223
- loss: 1.3287 - precision: 0.7795
Test Accuracy: 63.61%
Test Loss: 1.2964630126953125
Test Precision: 78.85%
Test F1 Scores (Per Class): [83.33333 79.12087 48.36601 45.0867 30.555552 6
7.04545 62.105255
59.030838 78.61271 79.38144 59.886997 46.39175 65.8385 49.765247
60.773476 57.731956 77.09496 77.27273 60.33519 58.69565 80.459755
76.40449 79.329605 77.27273 76.300575 63.04347 51.6129 44.171776
78.88888 63.855415 60.52631 67.796616 48.520706 51.101315 64.13043
32.584263 70.329666 58.20105 51.94805 87.86127 72.18934 84.32432
58.741253 69.047615 50.53763 38.15789 49.777775 52.991444 78.49462
81.17647 38.260864 71.26436 63.157898 88.2353 68.852455 36.756752
79.09604 70.35175 72.97298 53.08641 76.82927 74.19354 61.9883
66.66666 42.335762 52.564095 64.70587 55.3398 88.52458 84.74575
62.427734 71.962616 39.999996 59.016384 34.11764 81.31868 83.76962
57.627117 67.07316 64.052284 32.78688 51.578945 85.02415 70.27026
62.72189 70.243904 80.23255 76.92307 60.67415 65.957436 61.62162
72.093025 51.70731 59.493668 84.7826 69.27374 44.871788 62.745094
35.616432 79.329605]
Average Test F1 Scores:63.75

282/282  **18s** 38ms/step



	precision	recall	f1-score	support
beaver	0.90	0.78	0.83	90
dolphin	0.78	0.80	0.79	90
otter	0.59	0.41	0.48	90
seal	0.47	0.43	0.45	90
whale	0.26	0.37	0.31	90
aquarium fish	0.69	0.66	0.67	90
flatfish	0.59	0.66	0.62	90
ray	0.49	0.74	0.59	90
shark	0.82	0.76	0.79	90
trout	0.74	0.86	0.79	90
orchids	0.61	0.59	0.60	90
poppies	0.43	0.50	0.46	90
roses	0.75	0.59	0.66	90
sunflowers	0.43	0.59	0.50	90
tulips	0.60	0.61	0.61	90
bottles	0.54	0.62	0.58	90
bowls	0.78	0.77	0.77	90
cans	0.79	0.76	0.77	90
cups	0.61	0.60	0.60	90
plates	0.57	0.60	0.59	90
apples	0.83	0.78	0.80	90
mushrooms	0.77	0.76	0.76	90
oranges	0.80	0.79	0.79	90
pears	0.79	0.76	0.77	90
sweet peppers	0.80	0.73	0.76	90
clock	0.62	0.64	0.63	90
computer keyboard	0.50	0.53	0.52	90
lamp	0.49	0.40	0.44	90
telephone	0.79	0.79	0.79	90
television	0.70	0.59	0.64	90
bed	0.74	0.51	0.61	90
chair	0.69	0.67	0.68	90
couch	0.52	0.46	0.49	90
table	0.42	0.64	0.51	90
wardrobe	0.63	0.66	0.64	90
bee	0.33	0.32	0.33	90
beetle	0.70	0.71	0.70	90
butterfly	0.56	0.61	0.59	90
caterpillar	0.62	0.44	0.52	90
cockroach	0.92	0.84	0.88	90
bear	0.77	0.68	0.72	90
leopard	0.82	0.87	0.84	90
lion	0.79	0.47	0.59	90
tiger	0.74	0.64	0.69	90
wolf	0.49	0.52	0.51	90
bridge	0.47	0.32	0.38	90
castle	0.41	0.62	0.50	90
house	0.43	0.69	0.53	90
road	0.76	0.81	0.78	90
skyscraper	0.86	0.77	0.81	90
cloud	0.31	0.49	0.38	90
forest	0.74	0.69	0.71	90
mountain	0.60	0.67	0.63	90
plain	0.94	0.83	0.88	90
sea	0.68	0.70	0.69	90
camel	0.36	0.38	0.37	90
cattle	0.80	0.78	0.79	90
chimpanzee	0.64	0.78	0.70	90

elephant	0.93	0.60	0.73	90
kangaroo	0.60	0.48	0.53	90
fox	0.85	0.70	0.77	90
porcupine	0.72	0.77	0.74	90
possum	0.65	0.59	0.62	90
raccoon	0.70	0.63	0.67	90
skunk	0.62	0.32	0.42	90
crab	0.62	0.46	0.53	90
lobster	0.69	0.61	0.65	90
snail	0.49	0.63	0.55	90
spider	0.86	0.90	0.88	90
worm	0.86	0.83	0.85	90
baby	0.65	0.60	0.62	90
boy	0.62	0.86	0.72	90
girl	0.44	0.37	0.40	90
man	0.58	0.60	0.59	90
woman	0.36	0.32	0.34	90
crocodile	0.80	0.82	0.81	90
dinosaur	0.79	0.89	0.84	90
lizard	0.59	0.57	0.58	90
snake	0.74	0.61	0.67	90
turtle	0.78	0.54	0.64	90
hamster	0.26	0.44	0.33	90
mouse	0.50	0.56	0.53	90
rabbit	0.75	0.98	0.85	90
shrew	0.68	0.72	0.70	90
squirrel	0.67	0.59	0.63	90
maple	0.63	0.80	0.70	90
oak	0.84	0.77	0.80	90
palm	0.76	0.78	0.77	90
pine	0.61	0.60	0.61	90
willow	0.63	0.69	0.66	90
bicycle	0.60	0.63	0.62	90
bus	0.76	0.69	0.72	90
motorcycle	0.46	0.59	0.52	90
pickup truck	0.69	0.52	0.59	90
train	0.83	0.87	0.85	90
lawn-mower	0.70	0.69	0.69	90
rocket	0.53	0.39	0.45	90
streetcar	0.76	0.53	0.63	90
tank	0.46	0.29	0.36	90
tractor	0.80	0.79	0.79	90
accuracy			0.64	9000
macro avg	0.65	0.64	0.64	9000
weighted avg	0.65	0.64	0.64	9000

Model: "sequential"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d (GlobalAveragePooling2D)	(None , 2048)
dense (Dense)	(None , 512)
dropout (Dropout)	(None , 512)
dense_1 (Dense)	(None , 100)



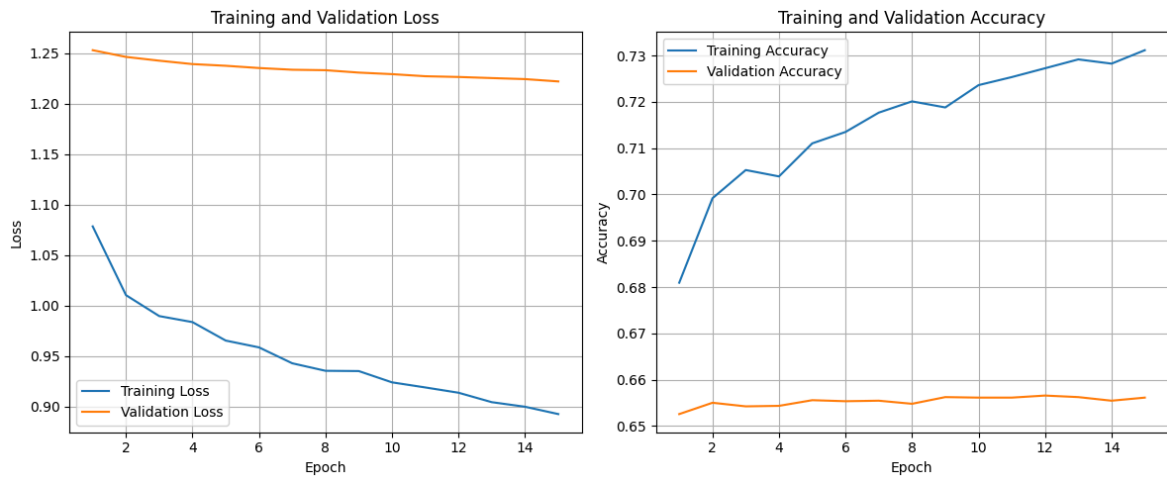
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1313/1314 ————— 0s 26ms/step - accuracy: 0.6793 - f1_score: 0.6779
- loss: 1.0716 - precision: 0.8211
Epoch 1: val_loss improved from inf to 1.25322, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 57s 35ms/step - accuracy: 0.6793 - f1_score: 0.6779 - loss: 1.0716 - precision: 0.8211 - val_accuracy: 0.6526 - val_f1_score: 0.6520 - val_loss: 1.2532 - val_precision: 0.7981
Epoch 2/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7028 - f1_score: 0.7014 - loss: 0.9859 - precision: 0.8391
Epoch 2: val_loss improved from 1.25322 to 1.24656, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7028 - f1_score: 0.7013 - loss: 0.9859 - precision: 0.8391 - val_accuracy: 0.6550 - val_f1_score: 0.6544 - val_loss: 1.2466 - val_precision: 0.7967
Epoch 3/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7128 - f1_score: 0.7110 - loss: 0.9560 - precision: 0.8423
Epoch 3: val_loss improved from 1.24656 to 1.24282, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7128 - f1_score: 0.7110 - loss: 0.9561 - precision: 0.8423 - val_accuracy: 0.6542 - val_f1_score: 0.6535 - val_loss: 1.2428 - val_precision: 0.7969
Epoch 4/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7118 - f1_score: 0.7100 - loss: 0.9454 - precision: 0.8418
Epoch 4: val_loss improved from 1.24282 to 1.23945, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7118 - f1_score: 0.7100 - loss: 0.9454 - precision: 0.8418 - val_accuracy: 0.6543 - val_f1_score: 0.6535 - val_loss: 1.2394 - val_precision: 0.7973
Epoch 5/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7185 - f1_score: 0.7166 - loss: 0.9311 - precision: 0.8462
Epoch 5: val_loss improved from 1.23945 to 1.23778, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7185 - f1_score: 0.7165 - loss: 0.9312 - precision: 0.8462 - val_accuracy: 0.6556 - val_f1_score: 0.6545 - val_loss: 1.2378 - val_precision: 0.7963
Epoch 6/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7221 - f1_score: 0.7203 - loss: 0.9219 - precision: 0.8490
Epoch 6: val_loss improved from 1.23778 to 1.23554, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7221 - f1_score: 0.7203 - loss: 0.9219 - precision: 0.8490 - val_accuracy: 0.6553 - val_f1_score: 0.6542 - val_loss: 1.2355 - val_precision: 0.7953
Epoch 7/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7273 - f1_score: 0.7254 - loss: 0.9023 - precision: 0.8505
Epoch 7: val_loss improved from 1.23554 to 1.23385, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7273 - f1_score: 0.7254 - loss: 0.9024 - precision: 0.8505 - val_accuracy: 0.6554 - val_f1_score: 0.6543 - val_loss: 1.2338 - val_precision: 0.7954
Epoch 8/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7307 - f1_score: 0.7290 - loss: 0.8977 - precision: 0.8525
Epoch 8: val_loss improved from 1.23385 to 1.23339, saving model to best_model_fine_tune_v1.keras

```
ne_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7307 - f1_score: 0.729
0 - loss: 0.8978 - precision: 0.8525 - val_accuracy: 0.6548 - val_f1_score: 0.653
6 - val_loss: 1.2334 - val_precision: 0.7946
Epoch 9/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7283 - f1_score: 0.7265
- loss: 0.8971 - precision: 0.8519
Epoch 9: val_loss improved from 1.23339 to 1.23102, saving model to best_model_fi
ne_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7283 - f1_score: 0.726
4 - loss: 0.8972 - precision: 0.8519 - val_accuracy: 0.6562 - val_f1_score: 0.654
9 - val_loss: 1.2310 - val_precision: 0.7948
Epoch 10/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7323 - f1_score: 0.7303
- loss: 0.8823 - precision: 0.8544
Epoch 10: val_loss improved from 1.23102 to 1.22948, saving model to best_model_f
ine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7322 - f1_score: 0.730
3 - loss: 0.8823 - precision: 0.8544 - val_accuracy: 0.6561 - val_f1_score: 0.655
1 - val_loss: 1.2295 - val_precision: 0.7949
Epoch 11/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7332 - f1_score: 0.7313
- loss: 0.8820 - precision: 0.8569
Epoch 11: val_loss improved from 1.22948 to 1.22740, saving model to best_model_f
ine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7332 - f1_score: 0.731
3 - loss: 0.8821 - precision: 0.8569 - val_accuracy: 0.6561 - val_f1_score: 0.655
1 - val_loss: 1.2274 - val_precision: 0.7940
Epoch 12/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7374 - f1_score: 0.7352
- loss: 0.8736 - precision: 0.8563
Epoch 12: val_loss improved from 1.22740 to 1.22667, saving model to best_model_f
ine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7374 - f1_score: 0.735
2 - loss: 0.8737 - precision: 0.8563 - val_accuracy: 0.6566 - val_f1_score: 0.655
5 - val_loss: 1.2267 - val_precision: 0.7944
Epoch 13/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7377 - f1_score: 0.7358
- loss: 0.8666 - precision: 0.8587
Epoch 13: val_loss improved from 1.22667 to 1.22558, saving model to best_model_f
ine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7377 - f1_score: 0.735
8 - loss: 0.8667 - precision: 0.8587 - val_accuracy: 0.6562 - val_f1_score: 0.655
4 - val_loss: 1.2256 - val_precision: 0.7942
Epoch 14/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7393 - f1_score: 0.7377
- loss: 0.8568 - precision: 0.8592
Epoch 14: val_loss improved from 1.22558 to 1.22456, saving model to best_model_f
ine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7392 - f1_score: 0.737
7 - loss: 0.8569 - precision: 0.8592 - val_accuracy: 0.6554 - val_f1_score: 0.654
5 - val_loss: 1.2246 - val_precision: 0.7939
Epoch 15/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7412 - f1_score: 0.7395
- loss: 0.8462 - precision: 0.8612
Epoch 15: val_loss improved from 1.22456 to 1.22225, saving model to best_model_f
ine_tune_v1.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7412 - f1_score: 0.739
4 - loss: 0.8463 - precision: 0.8612 - val_accuracy: 0.6561 - val_f1_score: 0.655
0 - val_loss: 1.2222 - val_precision: 0.7947
```



141/141 ————— **11s 40ms/step** - accuracy: 0.6513 - f1_score: 0.6443

- loss: 1.2445 - precision: 0.7985

Test Accuracy: 66.03%

Test Loss: 1.2187669277191162

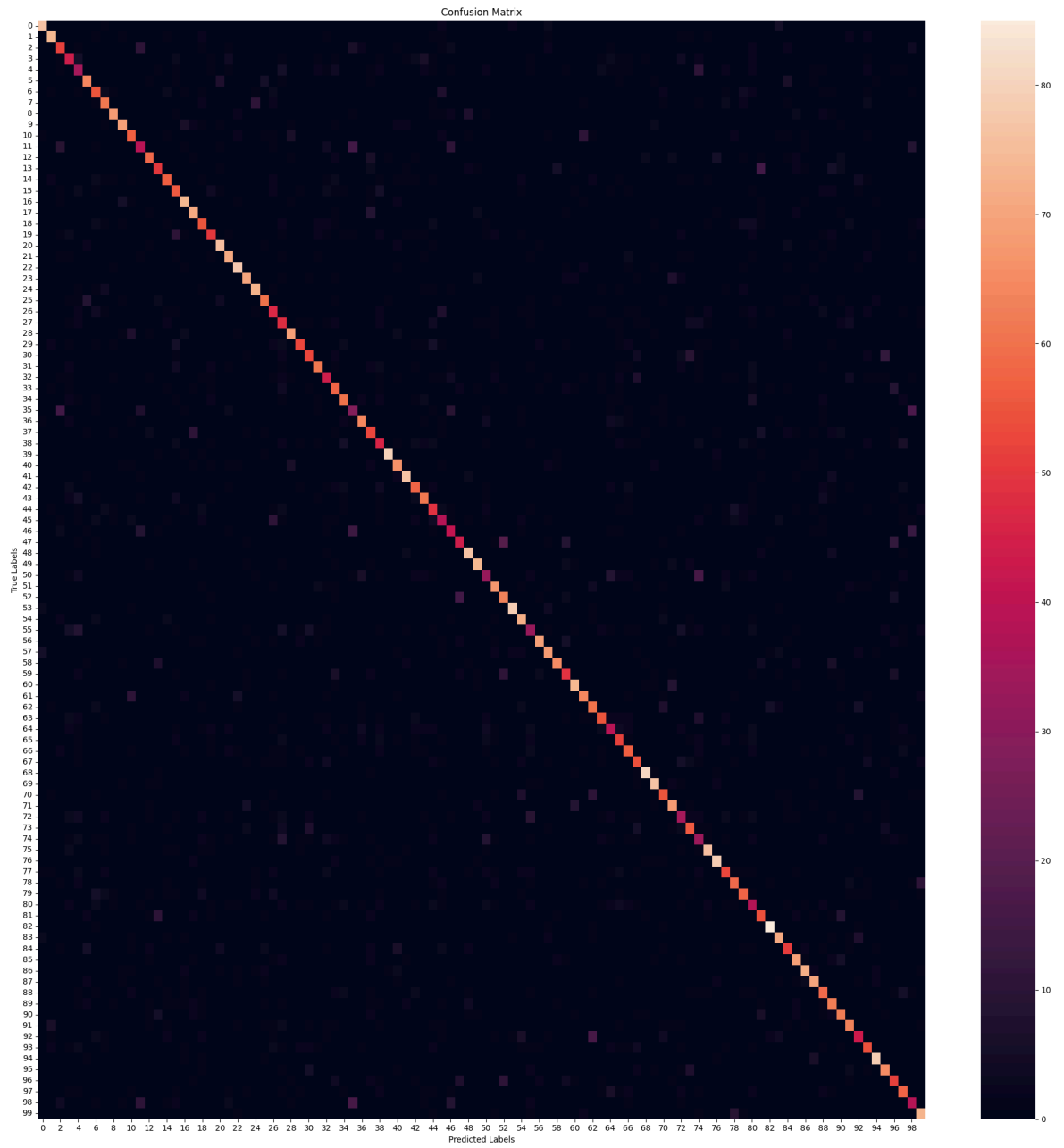
Test Precision: 80.41%

Test F1 Scores (Per Class): [85.08286 80.87432 54.838707 49.999992 37.433147 7
1.1111 57.591618

68.53932	80.473366	79.31035	62.983418	45.882347	69.41176	58.139526
66.27907	57.731956	77.48691	75.53191	60.439552	59.880234	81.72042
76.34408	82.97872	79.09604	79.78141	70.17543	50.53763	45.933006
77.9661	67.51592	61.6279	71.764694	50.28571	59.18367	67.796616
32.369934	73.14285	56.989246	49.729725	91.95402	70.588234	83.69564
64.80447	68.53932	51.041664	44.18604	46.590904	54.32098	80.628265
82.68156	38.55421	74.03315	64.94845	90.28571	73.09644	39.999996
80.701744	74.725266	77.8443	56.97674	81.31868	73.56321	63.157898
65.868256	45.61403	57.142853	62.29508	59.999996	89.01098	83.24323
63.218384	73.11827	41.420113	61.202175	34.736835	84.44444	84.946236
62.72189	67.42856	65.90909	39.79057	59.016384	90.90908	77.173904
59.999992	74.59459	81.35593	76.92307	64.13043	64.58333	66.31015
68.1081	51.807224	61.01694	85.86956	72.625694	55.026455	64.44444
42.222218	77.24867					

Average Test F1 Scores:65.98

282/282 ————— **11s 28ms/step**



	precision	recall	f1-score	support
beaver	0.85	0.86	0.85	90
dolphin	0.80	0.82	0.81	90
otter	0.54	0.58	0.56	90
seal	0.51	0.49	0.50	90
whale	0.36	0.39	0.37	90
aquarium fish	0.71	0.71	0.71	90
flatfish	0.54	0.61	0.58	90
ray	0.69	0.68	0.69	90
shark	0.86	0.76	0.80	90
trout	0.82	0.77	0.79	90
orchids	0.63	0.63	0.63	90
poppies	0.49	0.43	0.46	90
roses	0.74	0.66	0.69	90
sunflowers	0.61	0.56	0.58	90
tulips	0.70	0.63	0.66	90
bottles	0.54	0.62	0.58	90
bowls	0.73	0.82	0.77	90
cans	0.72	0.79	0.76	90
cups	0.60	0.61	0.60	90
plates	0.65	0.56	0.60	90
apples	0.79	0.84	0.82	90
mushrooms	0.74	0.79	0.76	90
oranges	0.80	0.87	0.83	90
pears	0.80	0.78	0.79	90
sweet peppers	0.78	0.81	0.80	90
clock	0.74	0.67	0.70	90
computer keyboard	0.49	0.52	0.51	90
lamp	0.40	0.53	0.46	90
telephone	0.79	0.77	0.78	90
television	0.79	0.59	0.68	90
bed	0.65	0.59	0.62	90
chair	0.76	0.68	0.72	90
couch	0.52	0.49	0.50	90
table	0.55	0.64	0.59	90
wardrobe	0.69	0.67	0.68	90
bee	0.34	0.31	0.32	90
beetle	0.75	0.71	0.73	90
butterfly	0.55	0.59	0.57	90
caterpillar	0.48	0.51	0.50	90
cockroach	0.95	0.89	0.92	90
bear	0.68	0.73	0.71	90
leopard	0.82	0.86	0.84	90
lion	0.65	0.64	0.65	90
tiger	0.69	0.68	0.69	90
wolf	0.48	0.54	0.51	90
bridge	0.46	0.42	0.44	90
castle	0.48	0.46	0.47	90
house	0.61	0.49	0.54	90
road	0.76	0.86	0.81	90
skyscraper	0.83	0.82	0.83	90
cloud	0.42	0.36	0.39	90
forest	0.74	0.74	0.74	90
mountain	0.61	0.70	0.65	90
plain	0.93	0.88	0.90	90
sea	0.67	0.80	0.73	90
camel	0.43	0.36	0.39	90
cattle	0.85	0.77	0.81	90
chimpanzee	0.74	0.76	0.75	90

elephant	0.84	0.72	0.78	90
kangaroo	0.60	0.54	0.57	90
fox	0.80	0.82	0.81	90
porcupine	0.76	0.71	0.74	90
possum	0.60	0.67	0.63	90
raccoon	0.71	0.61	0.66	90
skunk	0.48	0.43	0.46	90
crab	0.57	0.58	0.57	90
lobster	0.61	0.63	0.62	90
snail	0.60	0.60	0.60	90
spider	0.88	0.90	0.89	90
worm	0.81	0.86	0.83	90
baby	0.65	0.61	0.63	90
boy	0.71	0.76	0.73	90
girl	0.44	0.39	0.41	90
man	0.60	0.62	0.61	90
woman	0.33	0.37	0.35	90
crocodile	0.84	0.84	0.84	90
dinosaur	0.82	0.88	0.85	90
lizard	0.67	0.59	0.63	90
snake	0.69	0.66	0.67	90
turtle	0.67	0.64	0.66	90
hamster	0.38	0.42	0.40	90
mouse	0.58	0.60	0.59	90
rabbit	0.88	0.94	0.91	90
shrew	0.76	0.79	0.77	90
squirrel	0.64	0.57	0.60	90
maple	0.73	0.77	0.75	90
oak	0.83	0.80	0.81	90
palm	0.76	0.78	0.77	90
pine	0.63	0.66	0.64	90
willow	0.61	0.69	0.65	90
bicycle	0.64	0.69	0.66	90
bus	0.66	0.70	0.68	90
motorcycle	0.57	0.48	0.52	90
pickup truck	0.62	0.60	0.61	90
train	0.84	0.88	0.86	90
lawn-mower	0.73	0.72	0.73	90
rocket	0.53	0.58	0.55	90
streetcar	0.64	0.64	0.64	90
tank	0.42	0.42	0.42	90
tractor	0.74	0.81	0.77	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

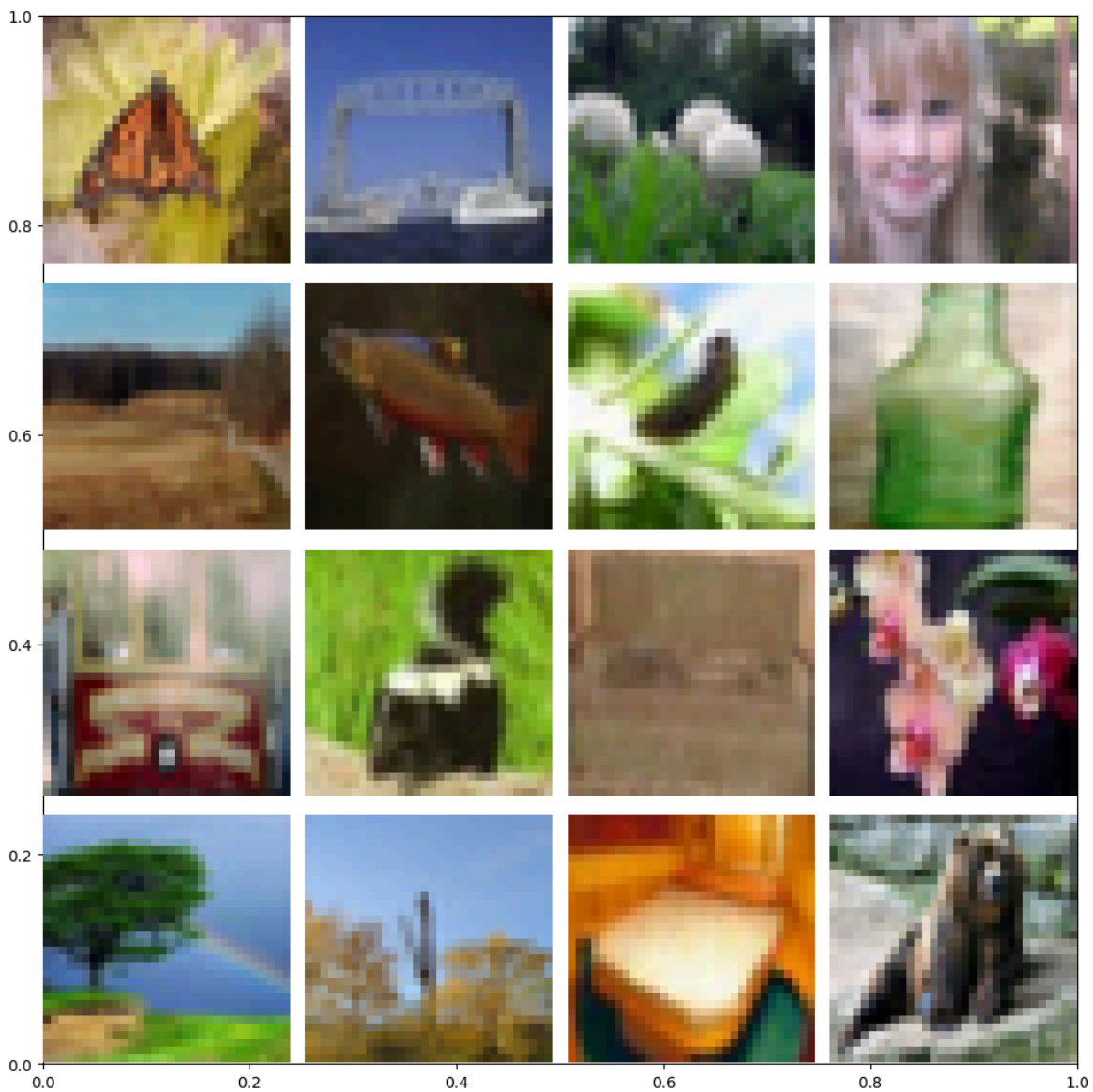
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

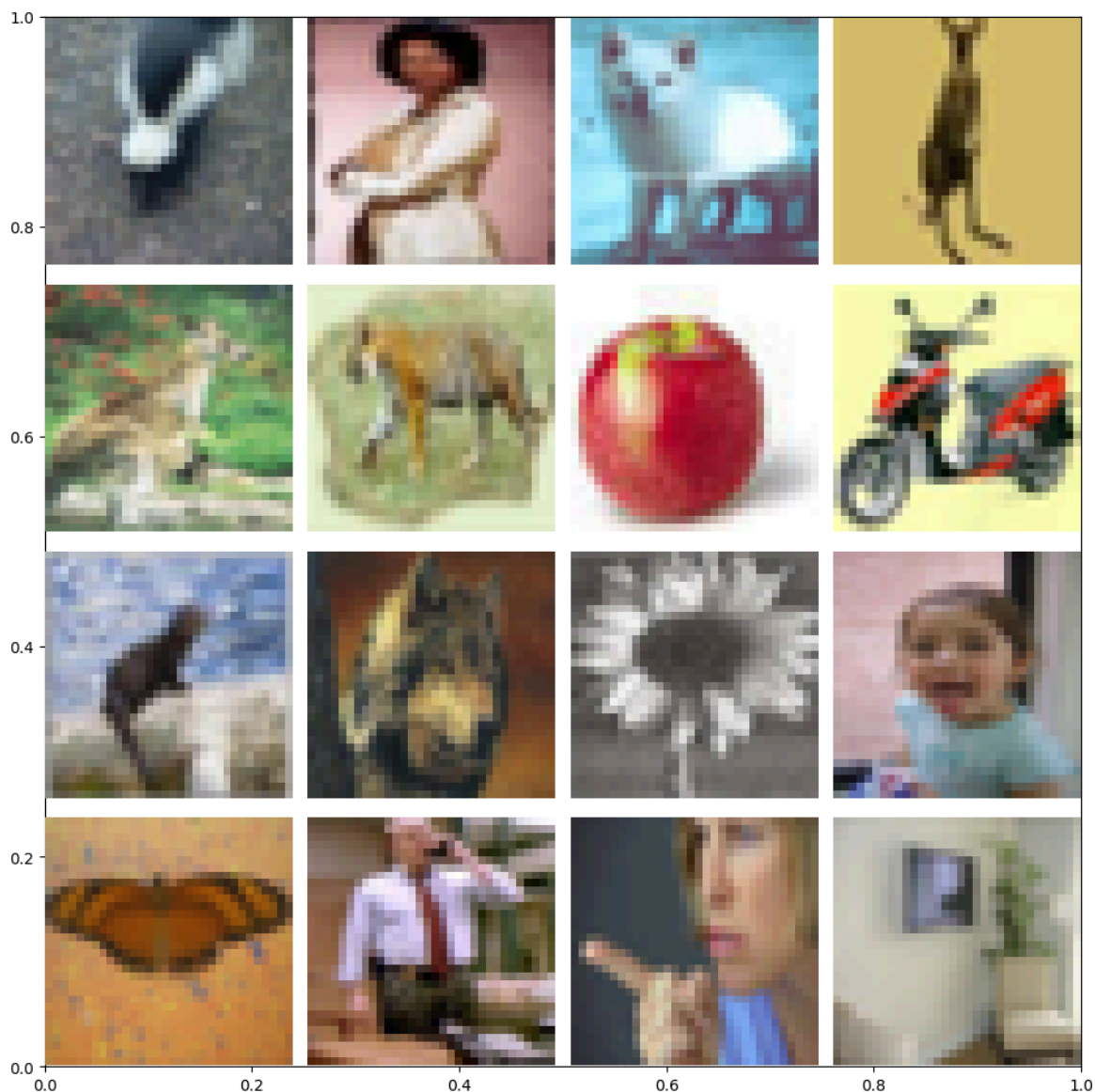
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[39724].shape: (1,)  
imgs[39724].shape: (32, 32, 3)
```

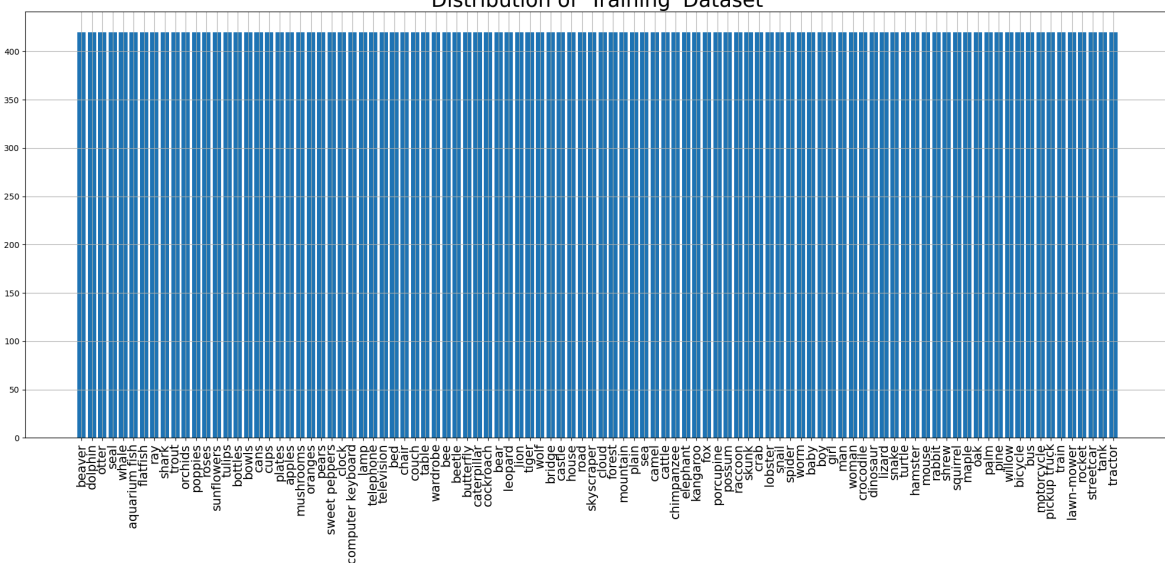


```
labels[27017].shape: (1,)  
imgs[27017].shape: (32, 32, 3)
```

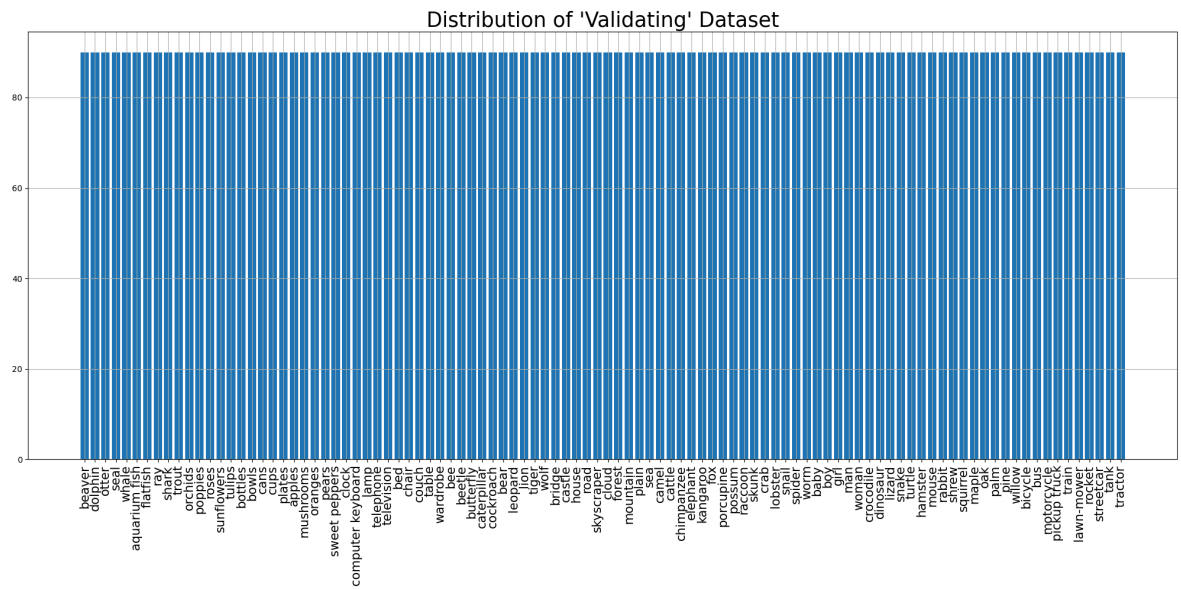


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

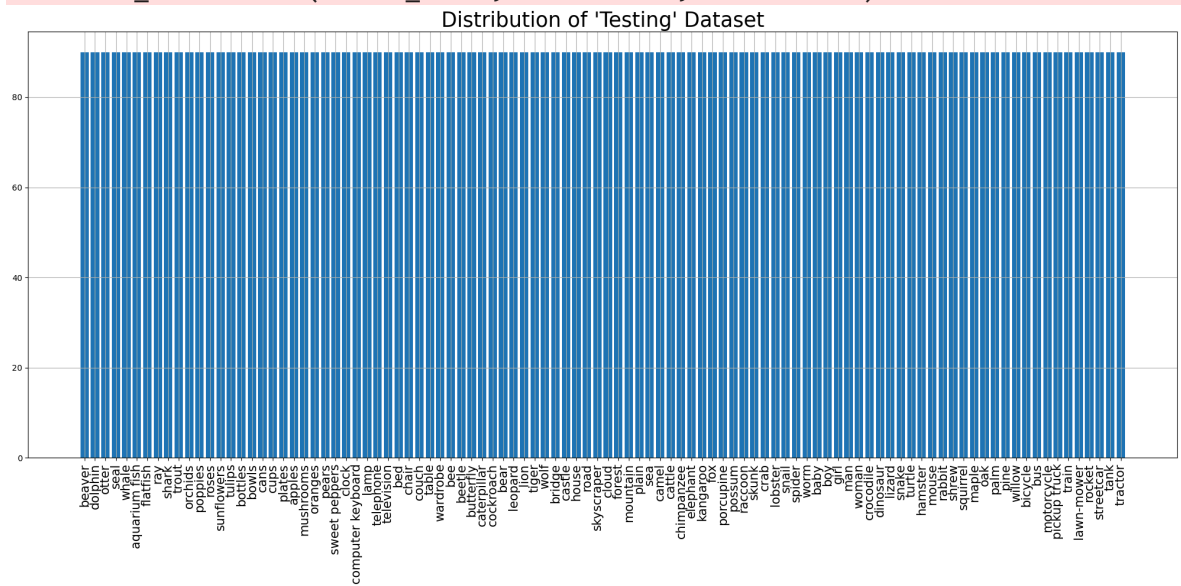
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_3 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— **0s** 29ms/step - accuracy: 0.3139 - f1_score: 0.3082 - loss: 2.9117 - precision: 0.6544
Epoch 1: val_loss improved from inf to 1.49852, saving model to best_model_v1.keras
657/657 ————— **40s** 46ms/step - accuracy: 0.3141 - f1_score: 0.3084 - loss: 2.9108 - precision: 0.6545 - val_accuracy: 0.5829 - val_f1_score: 0.5788 - val_loss: 1.4985 - val_precision: 0.8196
Epoch 2/25
655/657 ————— **0s** 24ms/step - accuracy: 0.5216 - f1_score: 0.5179 - loss: 1.7409 - precision: 0.7447
Epoch 2: val_loss improved from 1.49852 to 1.39130, saving model to best_model_v1.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.5217 - f1_score: 0.5180 - loss: 1.7406 - precision: 0.7448 - val_accuracy: 0.6111 - val_f1_score: 0.6073 - val_loss: 1.3913 - val_precision: 0.8062
Epoch 3/25
656/657 ————— **0s** 24ms/step - accuracy: 0.5652 - f1_score: 0.5614 - loss: 1.5490 - precision: 0.7578
Epoch 3: val_loss improved from 1.39130 to 1.35460, saving model to best_model_v1.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.5652 - f1_score: 0.5615 - loss: 1.5489 - precision: 0.7578 - val_accuracy: 0.6172 - val_f1_score: 0.6146 - val_loss: 1.3546 - val_precision: 0.8024
Epoch 4/25
656/657 ————— **0s** 24ms/step - accuracy: 0.5870 - f1_score: 0.5838 - loss: 1.4380 - precision: 0.7672
Epoch 4: val_loss improved from 1.35460 to 1.33845, saving model to best_model_v1.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.5870 - f1_score: 0.5838 - loss: 1.4380 - precision: 0.7672 - val_accuracy: 0.6256 - val_f1_score: 0.6214 - val_loss: 1.3384 - val_precision: 0.8126
Epoch 5/25
656/657 ————— **0s** 24ms/step - accuracy: 0.6090 - f1_score: 0.6056 - loss: 1.3547 - precision: 0.7806
Epoch 5: val_loss improved from 1.33845 to 1.32144, saving model to best_model_v1.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.6090 - f1_score: 0.6056 - loss: 1.3546 - precision: 0.7806 - val_accuracy: 0.6284 - val_f1_score: 0.6272 - val_loss: 1.3214 - val_precision: 0.7958
Epoch 6/25
657/657 ————— **0s** 24ms/step - accuracy: 0.6326 - f1_score: 0.6297 - loss: 1.2700 - precision: 0.7957
Epoch 6: val_loss did not improve from 1.32144
657/657 ————— **19s** 29ms/step - accuracy: 0.6326 - f1_score: 0.6298 - loss: 1.2699 - precision: 0.7957 - val_accuracy: 0.6262 - val_f1_score: 0.6230 - val_loss: 1.3283 - val_precision: 0.7957
Epoch 7/25
655/657 ————— **0s** 24ms/step - accuracy: 0.6419 - f1_score: 0.6393 - loss: 1.2145 - precision: 0.7966
Epoch 7: val_loss improved from 1.32144 to 1.31593, saving model to best_model_v1.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.6419 - f1_score: 0.6394 - loss: 1.2144 - precision: 0.7966 - val_accuracy: 0.6371 - val_f1_score: 0.6362 - val_loss: 1.3159 - val_precision: 0.7849
Epoch 8/25
657/657 ————— **0s** 24ms/step - accuracy: 0.6524 - f1_score: 0.6500 - loss: 1.1629 - precision: 0.7954
Epoch 8: val_loss did not improve from 1.31593
657/657 ————— **19s** 29ms/step - accuracy: 0.6524 - f1_score: 0.6500

- loss: 1.1629 - precision: 0.7954 - val_accuracy: 0.6370 - val_f1_score: 0.6348
 - val_loss: 1.3240 - val_precision: 0.7801

Epoch 9/25

655/657 ————— 0s 24ms/step - accuracy: 0.6706 - f1_score: 0.6674 -
 loss: 1.1072 - precision: 0.8063

Epoch 9: val_loss did not improve from 1.31593

657/657 ————— 19s 29ms/step - accuracy: 0.6706 - f1_score: 0.6674
 - loss: 1.1071 - precision: 0.8063 - val_accuracy: 0.6334 - val_f1_score: 0.6314
 - val_loss: 1.3338 - val_precision: 0.7753

Epoch 10/25

655/657 ————— 0s 24ms/step - accuracy: 0.6836 - f1_score: 0.6812 -
 loss: 1.0687 - precision: 0.8092

Epoch 10: val_loss did not improve from 1.31593

657/657 ————— 19s 29ms/step - accuracy: 0.6836 - f1_score: 0.6812
 - loss: 1.0687 - precision: 0.8092 - val_accuracy: 0.6427 - val_f1_score: 0.6403
 - val_loss: 1.3220 - val_precision: 0.7806

Epoch 11/25

656/657 ————— 0s 24ms/step - accuracy: 0.6907 - f1_score: 0.6880 -
 loss: 1.0260 - precision: 0.8172

Epoch 11: val_loss did not improve from 1.31593

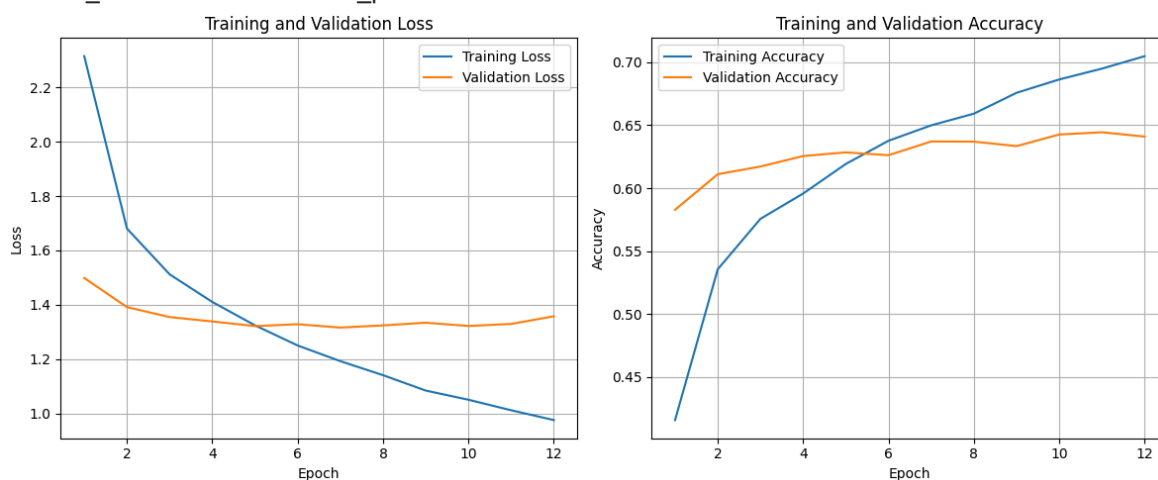
657/657 ————— 19s 28ms/step - accuracy: 0.6907 - f1_score: 0.6880
 - loss: 1.0260 - precision: 0.8172 - val_accuracy: 0.6444 - val_f1_score: 0.6438
 - val_loss: 1.3293 - val_precision: 0.7765

Epoch 12/25

655/657 ————— 0s 24ms/step - accuracy: 0.7007 - f1_score: 0.6982 -
 loss: 1.0002 - precision: 0.8194

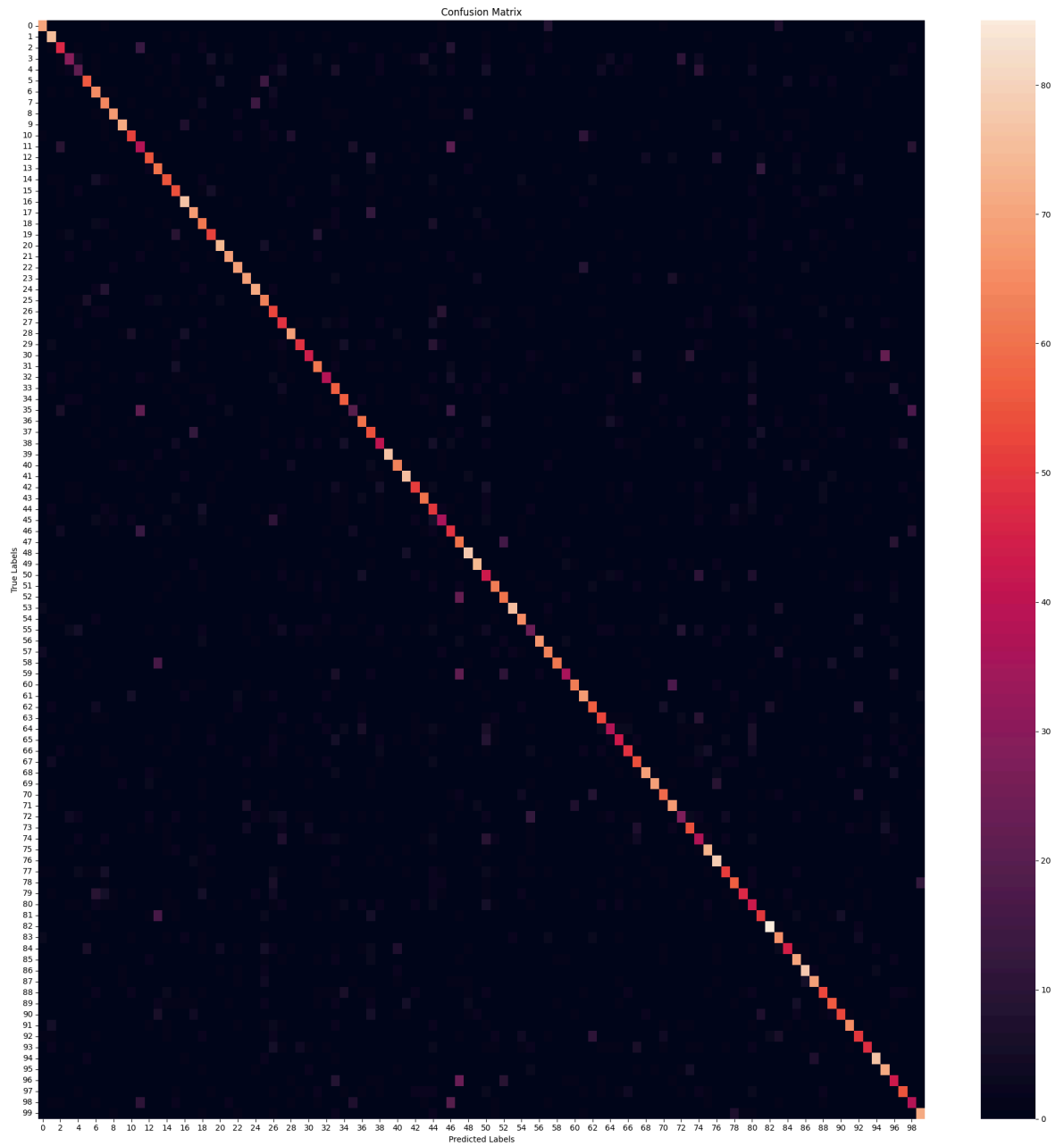
Epoch 12: val_loss did not improve from 1.31593

657/657 ————— 19s 28ms/step - accuracy: 0.7007 - f1_score: 0.6982
 - loss: 1.0001 - precision: 0.8194 - val_accuracy: 0.6410 - val_f1_score: 0.6400
 - val_loss: 1.3576 - val_precision: 0.7645



141/141  **11s** 38ms/step - accuracy: 0.6268 - f1_score: 0.6216
- loss: 1.3299 - precision: 0.7835
Test Accuracy: 63.76%
Test Loss: 1.3040658235549927
Test Precision: 78.92%
Test F1 Scores (Per Class): [80.92485 74.99999 56.28742 39.72602 29.729725 6
6.666664 63.414627
65.97938 77.71428 84.02366 63.803684 39.58333 63.5838 59.40594
65.088745 56.25 78.172585 73.91304 58.653843 61.90476 82.22221
79.768776 78.85714 77.9661 76.34408 60.869564 48.623848 51.308887
77.09496 62.025307 59.459454 64.55026 50.32258 50.220257 61.956512
29.230764 67.0391 52.427177 48.19277 90.05848 72.72727 81.28342
62.576683 71.00591 42.553185 46.15384 44.14414 49.999992 80.41236
80.43478 38.05309 72.093025 63.82978 88.372086 71.8232 29.870125
81.707306 69.99999 77.707 49.999992 74.69879 73.11827 62.63736
63.09523 47.43589 52.439022 60.869564 56.25 81.6568 83.63635
67.42856 71.57894 34.17721 64.70587 38.341965 76.8421 76.69902
61.53846 66.27907 64.38356 36.134453 57.142853 89.94709 65.346535
56.96202 72.16493 80.41236 75.40983 59.21787 65.88235 65.43209
73.0337 52.631577 58.895695 87.49999 70.29703 44.897953 67.07316
44.18604 78.453026]
Average Test F1 Scores:63.82

282/282  **12s** 28ms/step



	precision	recall	f1-score	support
beaver	0.84	0.78	0.81	90
dolphin	0.68	0.83	0.75	90
otter	0.61	0.52	0.56	90
seal	0.52	0.32	0.40	90
whale	0.38	0.23	0.29	90
aquarium fish	0.72	0.62	0.67	90
flatfish	0.57	0.72	0.63	90
ray	0.62	0.71	0.66	90
shark	0.80	0.76	0.78	90
trout	0.90	0.79	0.84	90
orchids	0.71	0.58	0.64	90
poppies	0.37	0.42	0.40	90
roses	0.66	0.61	0.64	90
sunflowers	0.54	0.67	0.59	90
tulips	0.70	0.61	0.65	90
bottles	0.53	0.60	0.56	90
bowls	0.72	0.86	0.78	90
cans	0.72	0.76	0.74	90
cups	0.52	0.68	0.59	90
plates	0.67	0.58	0.62	90
apples	0.82	0.82	0.82	90
mushrooms	0.83	0.77	0.80	90
oranges	0.81	0.77	0.79	90
pears	0.79	0.77	0.78	90
sweet peppers	0.74	0.79	0.76	90
clock	0.54	0.70	0.61	90
computer keyboard	0.41	0.59	0.49	90
lamp	0.48	0.54	0.51	90
telephone	0.78	0.77	0.77	90
television	0.72	0.54	0.62	90
bed	0.76	0.49	0.59	90
chair	0.62	0.68	0.65	90
couch	0.59	0.42	0.49	90
table	0.42	0.63	0.50	90
wardrobe	0.61	0.63	0.62	90
bee	0.47	0.21	0.29	90
beetle	0.67	0.67	0.67	90
butterfly	0.47	0.60	0.52	90
caterpillar	0.53	0.44	0.48	90
cockroach	0.95	0.86	0.90	90
bear	0.74	0.70	0.72	90
leopard	0.78	0.84	0.81	90
lion	0.70	0.57	0.63	90
tiger	0.76	0.67	0.71	90
wolf	0.34	0.56	0.43	90
bridge	0.55	0.40	0.46	90
castle	0.37	0.54	0.44	90
house	0.40	0.67	0.50	90
road	0.75	0.87	0.80	90
skyscraper	0.79	0.82	0.80	90
cloud	0.32	0.48	0.38	90
forest	0.76	0.69	0.72	90
mountain	0.61	0.67	0.63	90
plain	0.93	0.84	0.88	90
sea	0.71	0.72	0.72	90
camel	0.36	0.26	0.30	90
cattle	0.91	0.74	0.82	90
chimpanzee	0.69	0.70	0.70	90

elephant	0.91	0.68	0.78	90
kangaroo	0.67	0.40	0.50	90
fox	0.82	0.69	0.75	90
porcupine	0.71	0.76	0.73	90
possum	0.62	0.63	0.63	90
raccoon	0.68	0.59	0.63	90
skunk	0.56	0.41	0.47	90
crab	0.58	0.48	0.52	90
lobster	0.69	0.54	0.61	90
snail	0.51	0.60	0.55	90
spider	0.87	0.77	0.82	90
worm	0.92	0.77	0.84	90
baby	0.69	0.66	0.67	90
boy	0.68	0.76	0.72	90
girl	0.40	0.30	0.34	90
man	0.68	0.60	0.64	90
woman	0.36	0.41	0.38	90
crocodile	0.73	0.81	0.77	90
dinosaur	0.68	0.88	0.76	90
lizard	0.65	0.57	0.61	90
snake	0.70	0.63	0.66	90
turtle	0.84	0.52	0.64	90
hamster	0.29	0.48	0.36	90
mouse	0.59	0.56	0.57	90
rabbit	0.86	0.94	0.90	90
shrew	0.59	0.73	0.65	90
squirrel	0.66	0.50	0.57	90
maple	0.67	0.78	0.72	90
oak	0.75	0.87	0.80	90
palm	0.74	0.77	0.75	90
pine	0.60	0.59	0.59	90
willow	0.70	0.62	0.66	90
bicycle	0.74	0.59	0.65	90
bus	0.74	0.72	0.73	90
motorcycle	0.50	0.56	0.53	90
pickup truck	0.66	0.53	0.59	90
train	0.90	0.86	0.88	90
lawn-mower	0.63	0.79	0.70	90
rocket	0.41	0.48	0.44	90
streetcar	0.74	0.61	0.67	90
tank	0.46	0.42	0.44	90
tractor	0.78	0.79	0.78	90
accuracy			0.64	9000
macro avg	0.65	0.64	0.64	9000
weighted avg	0.65	0.64	0.64	9000

Model: "sequential_1"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None , 2048)
dense_2 (Dense)	(None , 512)
dropout_1 (Dropout)	(None , 512)
dense_3 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1313/1314 ————— 0s 27ms/step - accuracy: 0.6698 - f1_score: 0.6681
- loss: 1.1204 - precision: 0.8110
Epoch 1: val_loss improved from inf to 1.27476, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 57s 35ms/step - accuracy: 0.6698 - f1_score: 0.6681 - loss: 1.1204 - precision: 0.8110 - val_accuracy: 0.6466 - val_f1_score: 0.6449 - val_loss: 1.2748 - val_precision: 0.7966
Epoch 2/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6925 - f1_score: 0.6904
- loss: 1.0296 - precision: 0.8278
Epoch 2: val_loss improved from 1.27476 to 1.26837, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.6925 - f1_score: 0.6904 - loss: 1.0296 - precision: 0.8278 - val_accuracy: 0.6497 - val_f1_score: 0.6480 - val_loss: 1.2684 - val_precision: 0.7961
Epoch 3/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.6985 - f1_score: 0.6965
- loss: 1.0061 - precision: 0.8351
Epoch 3: val_loss improved from 1.26837 to 1.26447, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.6985 - f1_score: 0.6965 - loss: 1.0062 - precision: 0.8351 - val_accuracy: 0.6523 - val_f1_score: 0.6508 - val_loss: 1.2645 - val_precision: 0.7961
Epoch 4/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7047 - f1_score: 0.7028
- loss: 0.9839 - precision: 0.8371
Epoch 4: val_loss improved from 1.26447 to 1.26030, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7047 - f1_score: 0.7028 - loss: 0.9839 - precision: 0.8371 - val_accuracy: 0.6539 - val_f1_score: 0.6523 - val_loss: 1.2603 - val_precision: 0.7967
Epoch 5/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7045 - f1_score: 0.7026
- loss: 0.9736 - precision: 0.8380
Epoch 5: val_loss improved from 1.26030 to 1.25761, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7045 - f1_score: 0.7026 - loss: 0.9736 - precision: 0.8380 - val_accuracy: 0.6539 - val_f1_score: 0.6524 - val_loss: 1.2576 - val_precision: 0.7968
Epoch 6/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7102 - f1_score: 0.7084
- loss: 0.9597 - precision: 0.8418
Epoch 6: val_loss improved from 1.25761 to 1.25463, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7102 - f1_score: 0.7084 - loss: 0.9597 - precision: 0.8417 - val_accuracy: 0.6540 - val_f1_score: 0.6526 - val_loss: 1.2546 - val_precision: 0.7976
Epoch 7/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7161 - f1_score: 0.7142
- loss: 0.9475 - precision: 0.8478
Epoch 7: val_loss improved from 1.25463 to 1.25245, saving model to best_model_fine_tune_v1.keras
1314/1314 ————— 35s 27ms/step - accuracy: 0.7161 - f1_score: 0.7142 - loss: 0.9475 - precision: 0.8478 - val_accuracy: 0.6556 - val_f1_score: 0.6542 - val_loss: 1.2524 - val_precision: 0.7977
Epoch 8/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7211 - f1_score: 0.7190
- loss: 0.9323 - precision: 0.8445
Epoch 8: val_loss improved from 1.25245 to 1.24975, saving model to best_model_fine_tune_v1.keras

ne_tune_v1.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.7211 - f1_score: 0.7189 - loss: 0.9324 - precision: 0.8445 - val_accuracy: 0.6569 - val_f1_score: 0.6555 - val_loss: 1.2497 - val_precision: 0.7983

Epoch 9/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.7194 - f1_score: 0.7175 - loss: 0.9282 - precision: 0.8466

Epoch 9: val_loss improved from 1.24975 to 1.24771, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.7194 - f1_score: 0.7175 - loss: 0.9282 - precision: 0.8466 - val_accuracy: 0.6573 - val_f1_score: 0.6559 - val_loss: 1.2477 - val_precision: 0.7970

Epoch 10/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.7211 - f1_score: 0.7191 - loss: 0.9250 - precision: 0.8515

Epoch 10: val_loss improved from 1.24771 to 1.24567, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.7211 - f1_score: 0.7191 - loss: 0.9250 - precision: 0.8515 - val_accuracy: 0.6570 - val_f1_score: 0.6555 - val_loss: 1.2457 - val_precision: 0.7990

Epoch 11/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.7299 - f1_score: 0.7282 - loss: 0.9073 - precision: 0.8543

Epoch 11: val_loss improved from 1.24567 to 1.24432, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.7299 - f1_score: 0.7282 - loss: 0.9073 - precision: 0.8543 - val_accuracy: 0.6586 - val_f1_score: 0.6573 - val_loss: 1.2443 - val_precision: 0.7994

Epoch 12/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.7296 - f1_score: 0.7279 - loss: 0.8956 - precision: 0.8550

Epoch 12: val_loss improved from 1.24432 to 1.24287, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7296 - f1_score: 0.7279 - loss: 0.8957 - precision: 0.8550 - val_accuracy: 0.6583 - val_f1_score: 0.6571 - val_loss: 1.2429 - val_precision: 0.7982

Epoch 13/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.7341 - f1_score: 0.7323 - loss: 0.8919 - precision: 0.8564

Epoch 13: val_loss improved from 1.24287 to 1.24147, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7341 - f1_score: 0.7323 - loss: 0.8919 - precision: 0.8564 - val_accuracy: 0.6591 - val_f1_score: 0.6578 - val_loss: 1.2415 - val_precision: 0.7975

Epoch 14/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.7319 - f1_score: 0.7302 - loss: 0.8912 - precision: 0.8584

Epoch 14: val_loss improved from 1.24147 to 1.23958, saving model to best_model_fine_tune_v1.keras

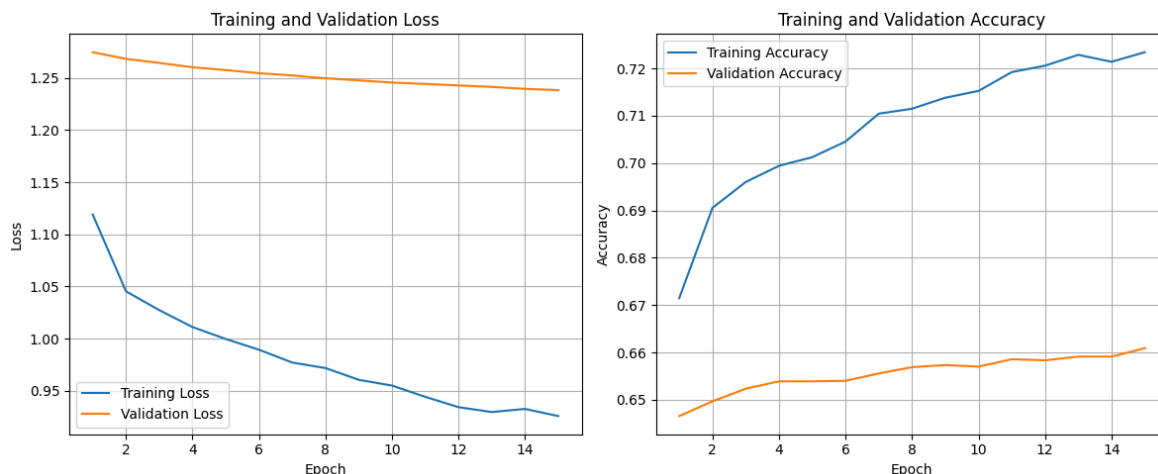
1314/1314 ————— 35s 27ms/step - accuracy: 0.7318 - f1_score: 0.7302 - loss: 0.8912 - precision: 0.8584 - val_accuracy: 0.6591 - val_f1_score: 0.6578 - val_loss: 1.2396 - val_precision: 0.7999

Epoch 15/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.7334 - f1_score: 0.7316 - loss: 0.8856 - precision: 0.8575

Epoch 15: val_loss improved from 1.23958 to 1.23838, saving model to best_model_fine_tune_v1.keras

1314/1314 ————— 35s 27ms/step - accuracy: 0.7334 - f1_score: 0.7316 - loss: 0.8857 - precision: 0.8575 - val_accuracy: 0.6609 - val_f1_score: 0.6595 - val_loss: 1.2384 - val_precision: 0.7985



141/141 ————— **12s 40ms/step** - accuracy: 0.6568 - f1_score: 0.6484

- loss: 1.2409 - precision: 0.7961

Test Accuracy: 66.28%

Test Loss: 1.2180702686309814

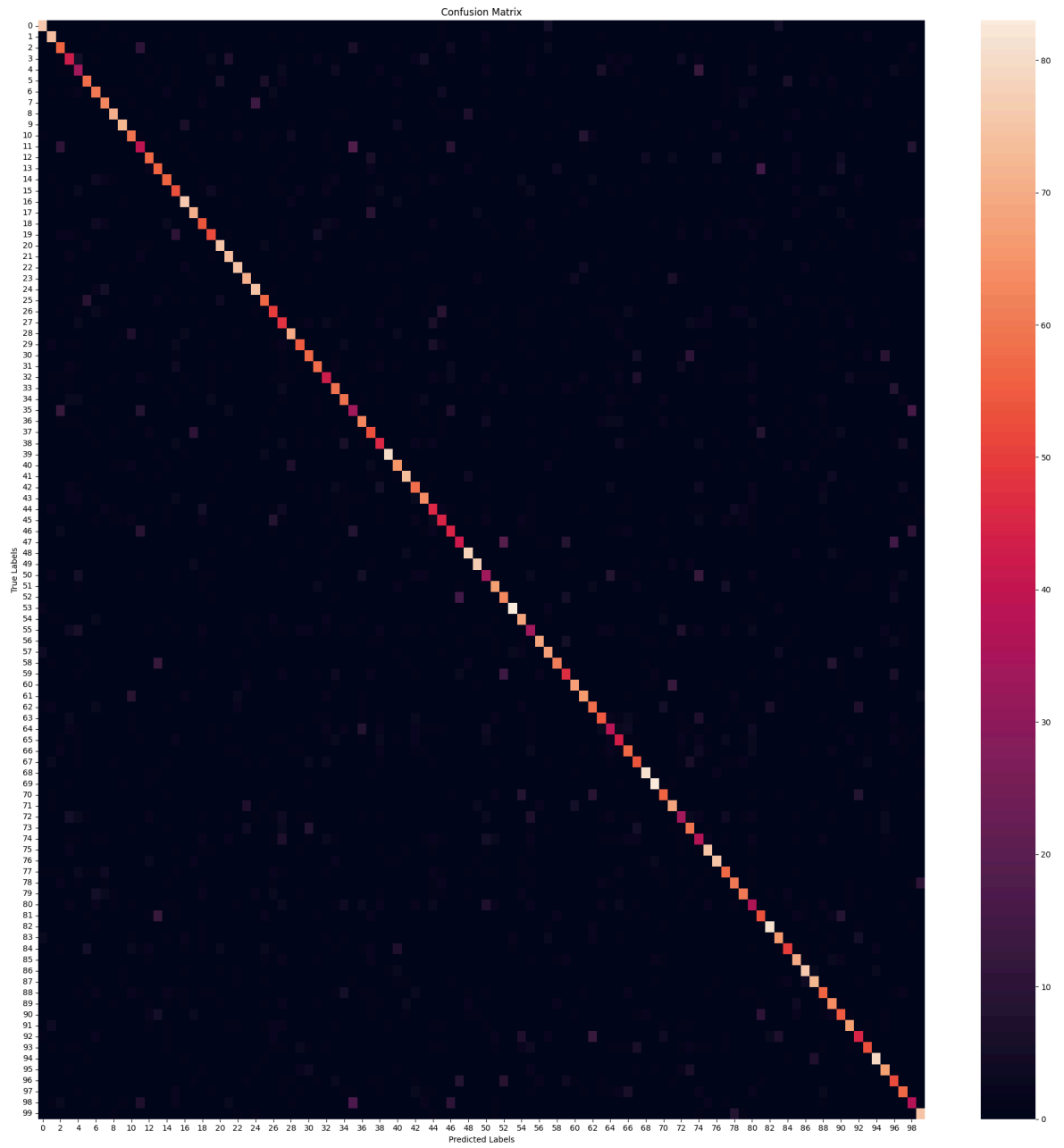
Test Precision: 80.19%

Test F1 Scores (Per Class): [84.26965 77.89473 58.63874 48.863632 37.988823 67.05202 62.827217

71.18643 81.14285 83.236984 64.83515 44.571423 65.14285 61.538452
65.11627 57.297295 80.42328 77.173904 60.33519 61.90476 81.96721
78.723404 81.96721 78.88888 79.787224 65.517235 55.555553 48.2412
77.77777 65.47618 66.66666 66.66666 51.807224 62.105255 66.29213
37.988823 69.61325 57.297295 51.396645 90.90909 68.06282 82.48587
62.702698 72.31638 49.197857 52.631577 50.847454 52.1739 81.675385
84.61538 39.534874 73.62636 64.2857 91.208786 70.05075 38.418076
81.6568 73.91304 75.30864 55.294113 78.16091 74.57626 63.73626
63.953484 45.23809 51.461983 61.62162 58.69565 88.04347 86.910995
64.739876 74.19354 42.23602 62.76595 38.341965 79.787224 83.798874
63.687145 68.57143 68.181816 38.095238 57.60869 89.01098 70.967735
59.171593 73.68421 83.51647 78.260864 61.202175 66.321236 61.111103
74.86033 53.254433 63.473045 87.77777 73.62636 53.06122 64.77273
42.28571 81.08108]

Average Test F1 Scores:66.18

282/282 ————— **11s 28ms/step**



	precision	recall	f1-score	support
beaver	0.85	0.84	0.85	90
dolphin	0.74	0.82	0.78	90
otter	0.55	0.62	0.59	90
seal	0.50	0.48	0.49	90
whale	0.38	0.38	0.38	90
aquarium fish	0.70	0.64	0.67	90
flatfish	0.59	0.67	0.63	90
ray	0.73	0.70	0.72	90
shark	0.84	0.79	0.81	90
trout	0.87	0.80	0.83	90
orchids	0.64	0.66	0.65	90
poppies	0.46	0.43	0.45	90
roses	0.67	0.63	0.65	90
sunflowers	0.61	0.62	0.62	90
tulips	0.68	0.62	0.65	90
bottles	0.56	0.59	0.57	90
bowls	0.77	0.84	0.80	90
cans	0.76	0.79	0.77	90
cups	0.61	0.60	0.61	90
plates	0.67	0.58	0.62	90
apples	0.81	0.83	0.82	90
mushrooms	0.76	0.82	0.79	90
oranges	0.81	0.83	0.82	90
pears	0.79	0.79	0.79	90
sweet peppers	0.77	0.83	0.80	90
clock	0.68	0.63	0.66	90
computer keyboard	0.55	0.56	0.55	90
lamp	0.44	0.53	0.48	90
telephone	0.78	0.78	0.78	90
television	0.71	0.61	0.65	90
bed	0.70	0.63	0.67	90
chair	0.71	0.64	0.67	90
couch	0.57	0.48	0.52	90
table	0.59	0.66	0.62	90
wardrobe	0.67	0.66	0.66	90
bee	0.38	0.38	0.38	90
beetle	0.69	0.70	0.70	90
butterfly	0.56	0.59	0.57	90
caterpillar	0.52	0.51	0.51	90
cockroach	0.93	0.89	0.91	90
bear	0.64	0.72	0.68	90
leopard	0.84	0.81	0.82	90
lion	0.61	0.64	0.63	90
tiger	0.74	0.71	0.72	90
wolf	0.48	0.51	0.49	90
bridge	0.56	0.50	0.53	90
castle	0.52	0.50	0.51	90
house	0.59	0.47	0.52	90
road	0.77	0.87	0.82	90
skyscraper	0.84	0.86	0.85	90
cloud	0.41	0.38	0.40	90
forest	0.73	0.74	0.74	90
mountain	0.59	0.70	0.64	90
plain	0.91	0.92	0.92	90
sea	0.64	0.77	0.70	90
camel	0.39	0.38	0.38	90
cattle	0.87	0.77	0.82	90
chimpanzee	0.72	0.76	0.74	90

elephant	0.85	0.68	0.75	90
kangaroo	0.59	0.52	0.55	90
fox	0.81	0.76	0.78	90
porcupine	0.76	0.73	0.75	90
possum	0.63	0.64	0.64	90
raccoon	0.67	0.61	0.64	90
skunk	0.49	0.42	0.45	90
crab	0.54	0.49	0.51	90
lobster	0.60	0.63	0.62	90
snail	0.57	0.60	0.59	90
spider	0.86	0.90	0.88	90
worm	0.82	0.92	0.87	90
baby	0.67	0.62	0.65	90
boy	0.72	0.77	0.74	90
girl	0.48	0.38	0.42	90
man	0.60	0.66	0.63	90
woman	0.36	0.41	0.38	90
crocodile	0.77	0.83	0.80	90
dinosaur	0.84	0.83	0.84	90
lizard	0.64	0.63	0.64	90
snake	0.71	0.67	0.69	90
turtle	0.70	0.67	0.68	90
hamster	0.36	0.40	0.38	90
mouse	0.56	0.59	0.58	90
rabbit	0.88	0.90	0.89	90
shrew	0.69	0.73	0.71	90
squirrel	0.63	0.56	0.59	90
maple	0.70	0.78	0.74	90
oak	0.83	0.84	0.84	90
palm	0.77	0.80	0.78	90
pine	0.60	0.62	0.61	90
willow	0.62	0.71	0.66	90
bicycle	0.61	0.61	0.61	90
bus	0.75	0.74	0.75	90
motorcycle	0.57	0.50	0.53	90
pickup truck	0.69	0.59	0.63	90
train	0.88	0.88	0.88	90
lawn-mower	0.73	0.74	0.74	90
rocket	0.49	0.58	0.53	90
streetcar	0.66	0.63	0.65	90
tank	0.44	0.41	0.42	90
tractor	0.79	0.83	0.81	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

Dropout(0.6), weight_decay(3e-2), l2(3e-2)

```
In [6]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
```

```

X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
                'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
                'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
                'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
                'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
                'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
                'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
                'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
                'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
                'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
                'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
#train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))

```

```

# .map(augment_dataset, num_parallel_calls=tf.data.experimental.AUTOTUNE)
# .batch(batch_size)
# .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset#.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-2) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.6), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):

```

```

print(type(sample)) # Should be <class 'tuple'>
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'
print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
#for x, y in test_dataset.take(1):
#    print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
#for x_batch, y_batch in test_dataset.take(1):
#    test_loss, test_acc = model.evaluate(x_batch, y_batch)
#    print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
#tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=3e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)

#reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                               factor=0.1, # Reduce lr by a factor
#                               patience=3, # Num epochs w/ no improvement
#                               min_lr=1e-6, # Min lr
#                               verbose=1)

#tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                            histogram_freq=1, # Logs histograms for weights/acc
#                            write_graph=True, # Logs graph of model
#                            write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model_v2.keras', # Path where the best model is saved
                             monitor='val_loss', # What to monitor (validation loss)
                             save_best_only=True, # Only save model when val loss is best
                             mode='min', # Looking to minimize the validation loss
                             verbose=1) # Print out message when saving model
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v2.keras', # Path where the best model is saved
                                       monitor='val_loss', # What to monitor (validation loss)
                                       save_best_only=True, # Only save model when val loss is best
                                       mode='min', # Looking to minimize the validation loss
                                       verbose=1) # Print out message when saving model
#csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                   batch_size=batch_size, callbacks=[early_stopping, checkpoint])

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']

```

```

val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v2.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n

#### <<<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<<Fine-Tune>>>>>>>>>>>>

# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

```

```

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                           .batch(batch_size)
                           .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v2.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores: {np.average(test_f1_scores.numpy()*100):.2f}")

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))

```

```
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

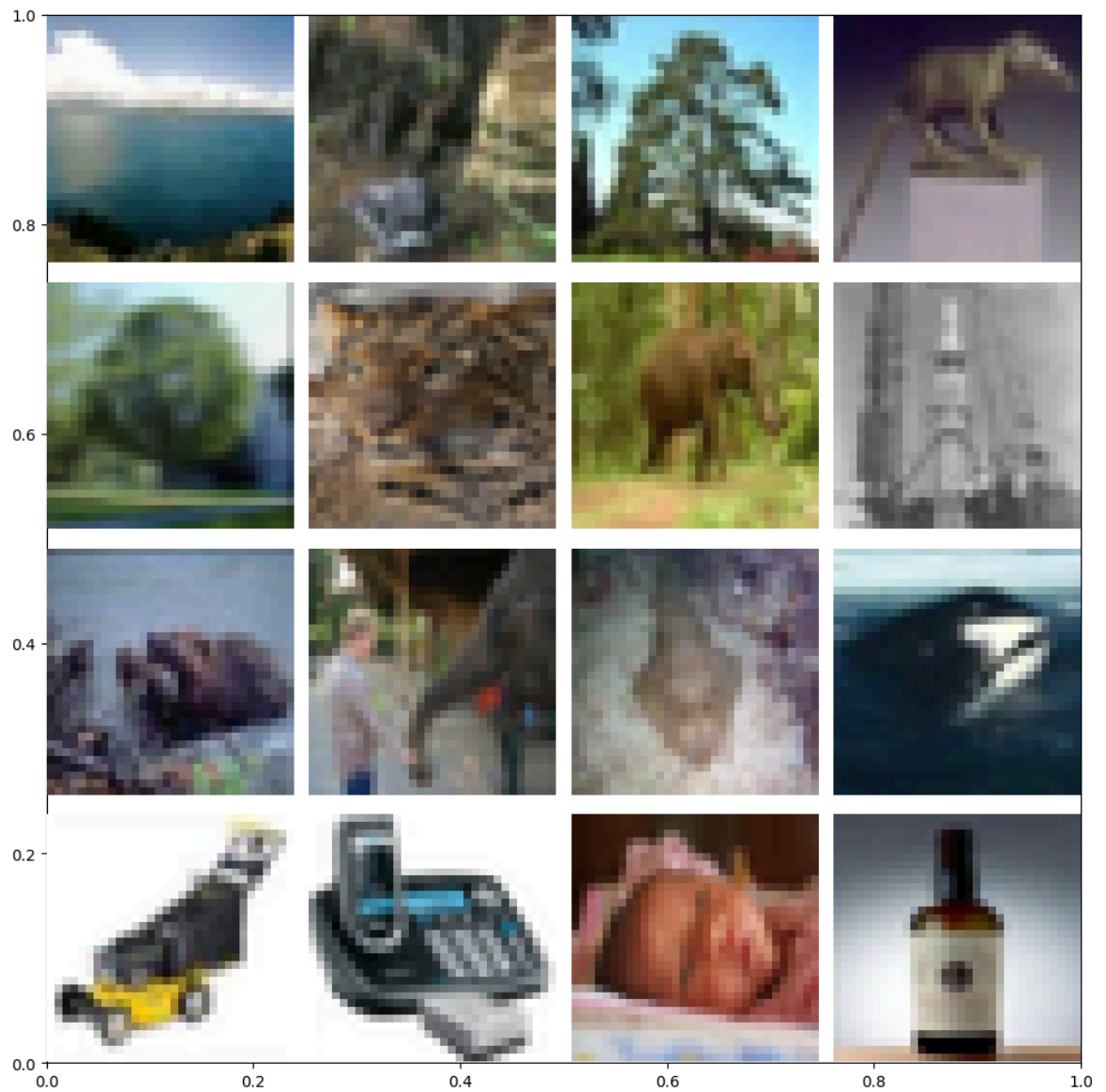
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

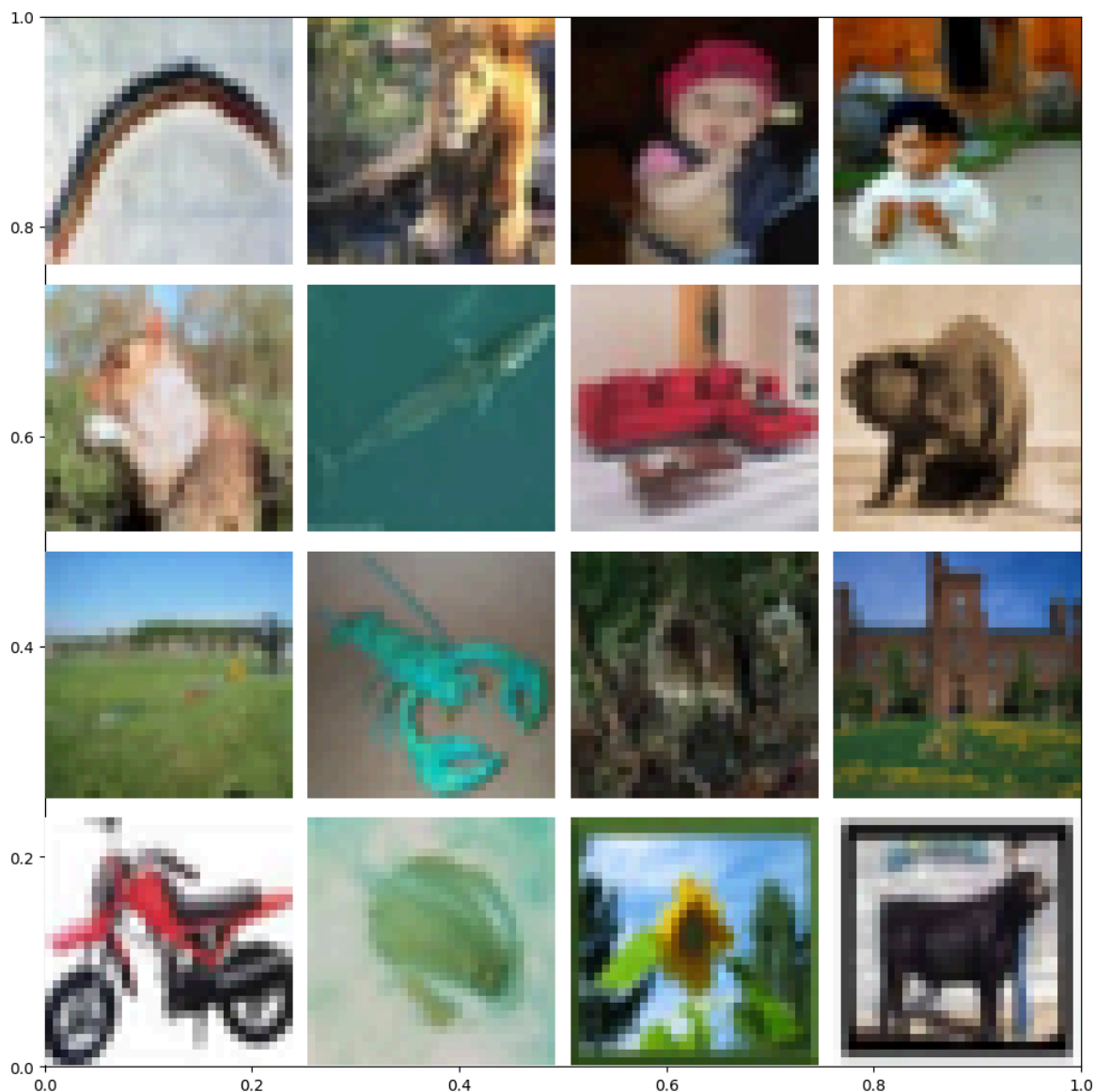
y_test.shape: (9000, 1)

labels[9538].shape: (1,)

imgs[9538].shape: (32, 32, 3)

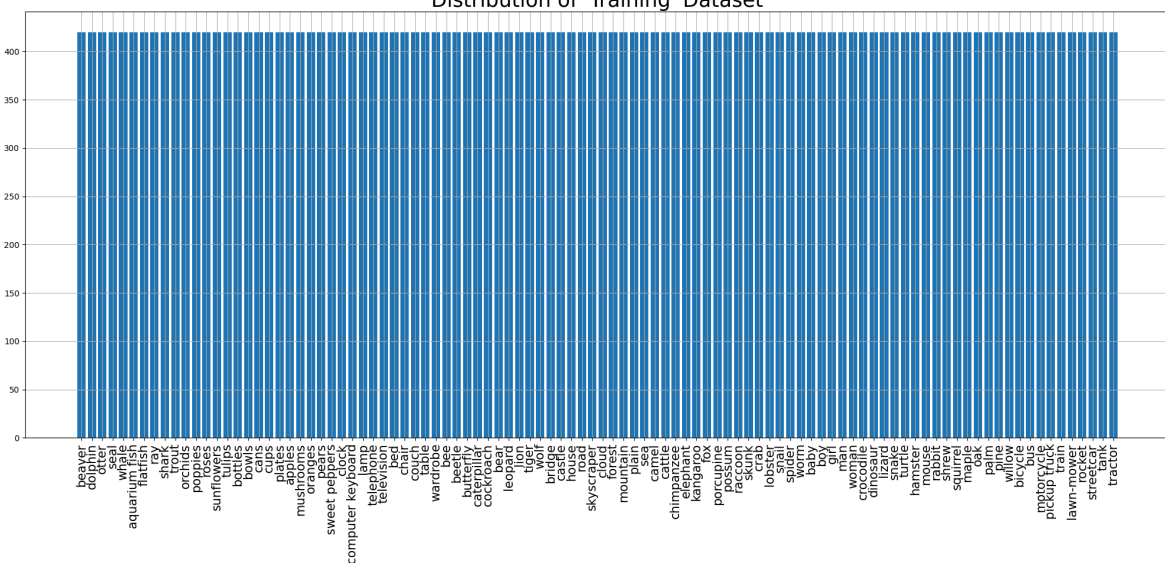


labels[24543].shape: (1,)
imgs[24543].shape: (32, 32, 3)

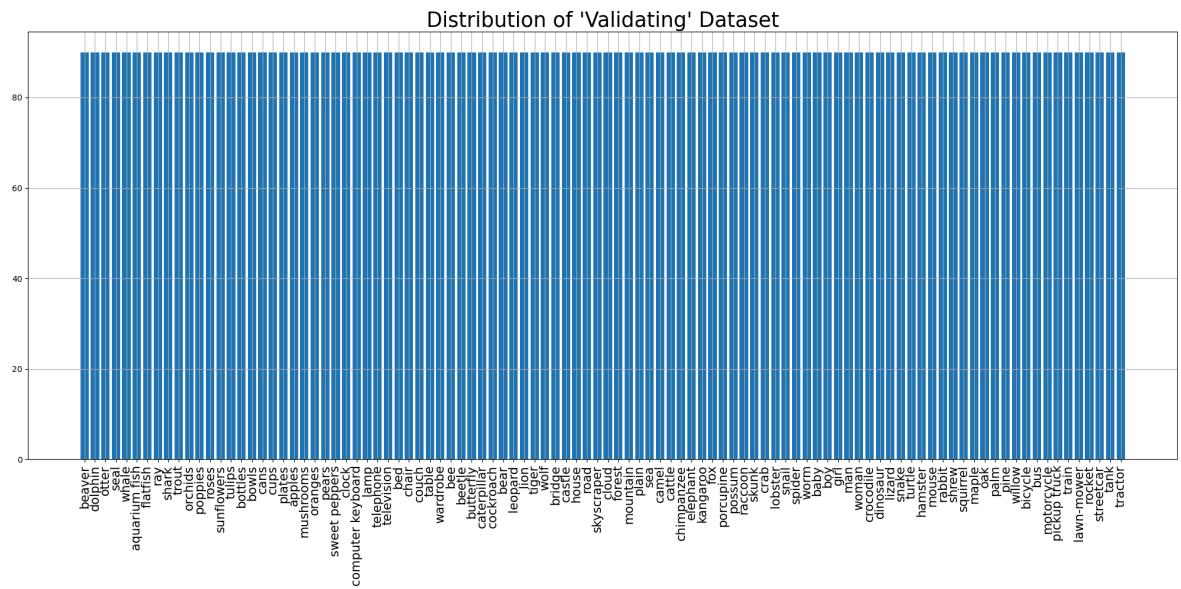


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

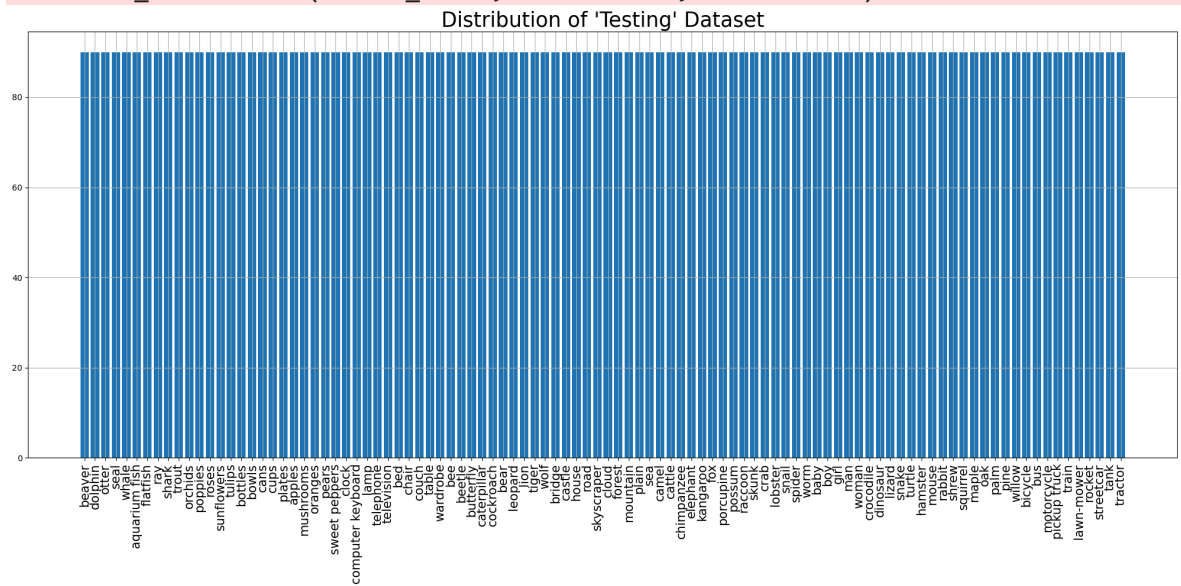
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_2"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None , 2048)
dense_4 (Dense)	(None , 512)
dropout_2 (Dropout)	(None , 512)
dense_5 (Dense)	(None , 100)



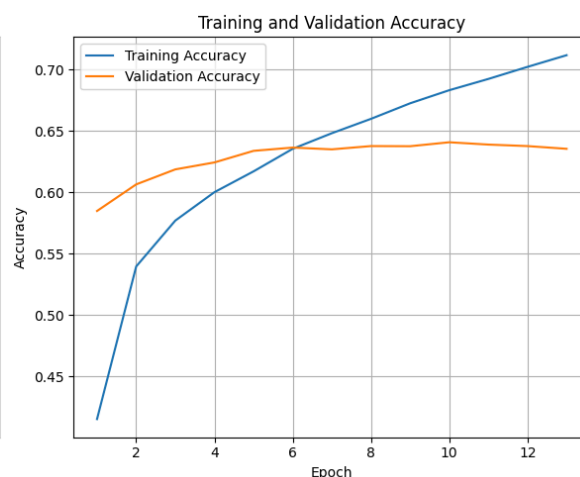
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

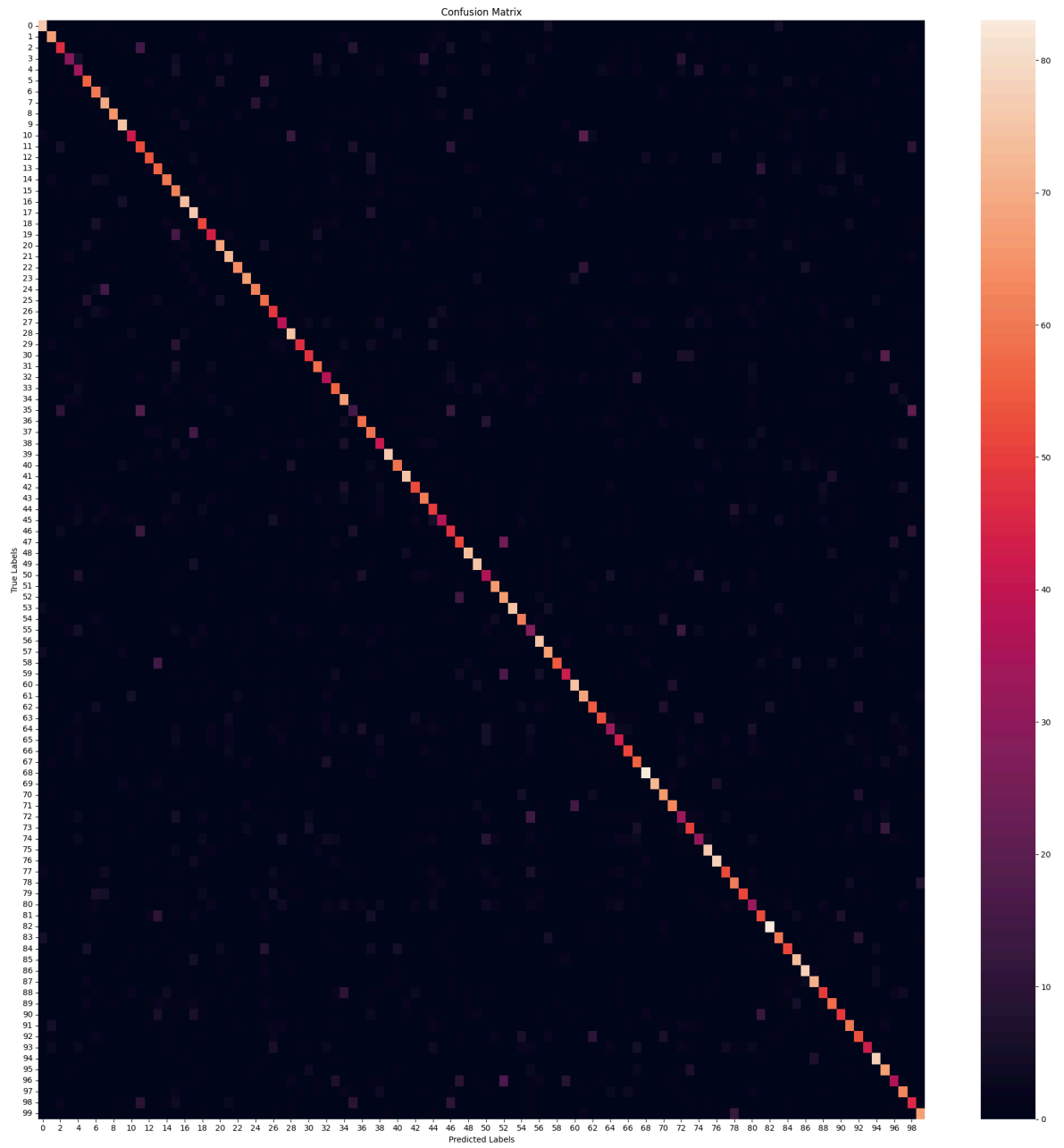
Epoch 1/25
657/657 ————— **0s** 30ms/step - accuracy: 0.3145 - f1_score: 0.3081 - loss: 2.9021 - precision: 0.6628
Epoch 1: val_loss improved from inf to 1.50664, saving model to best_model_v2.keras as
657/657 ————— **41s** 46ms/step - accuracy: 0.3147 - f1_score: 0.3083 - loss: 2.9012 - precision: 0.6628 - val_accuracy: 0.5849 - val_f1_score: 0.5795 - val_loss: 1.5066 - val_precision: 0.8154
Epoch 2/25
655/657 ————— **0s** 24ms/step - accuracy: 0.5236 - f1_score: 0.5197 - loss: 1.7418 - precision: 0.7417
Epoch 2: val_loss improved from 1.50664 to 1.39411, saving model to best_model_v2.keras
657/657 ————— **20s** 31ms/step - accuracy: 0.5236 - f1_score: 0.5198 - loss: 1.7415 - precision: 0.7417 - val_accuracy: 0.6066 - val_f1_score: 0.6033 - val_loss: 1.3941 - val_precision: 0.8063
Epoch 3/25
655/657 ————— **0s** 24ms/step - accuracy: 0.5685 - f1_score: 0.5656 - loss: 1.5453 - precision: 0.7607
Epoch 3: val_loss improved from 1.39411 to 1.35685, saving model to best_model_v2.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.5686 - f1_score: 0.5656 - loss: 1.5451 - precision: 0.7608 - val_accuracy: 0.6188 - val_f1_score: 0.6162 - val_loss: 1.3568 - val_precision: 0.7995
Epoch 4/25
657/657 ————— **0s** 24ms/step - accuracy: 0.5931 - f1_score: 0.5892 - loss: 1.4381 - precision: 0.7772
Epoch 4: val_loss improved from 1.35685 to 1.33942, saving model to best_model_v2.keras
657/657 ————— **20s** 31ms/step - accuracy: 0.5931 - f1_score: 0.5892 - loss: 1.4380 - precision: 0.7772 - val_accuracy: 0.6244 - val_f1_score: 0.6228 - val_loss: 1.3394 - val_precision: 0.7968
Epoch 5/25
655/657 ————— **0s** 24ms/step - accuracy: 0.6074 - f1_score: 0.6042 - loss: 1.3618 - precision: 0.7783
Epoch 5: val_loss improved from 1.33942 to 1.31299, saving model to best_model_v2.keras
657/657 ————— **20s** 31ms/step - accuracy: 0.6074 - f1_score: 0.6043 - loss: 1.3617 - precision: 0.7783 - val_accuracy: 0.6339 - val_f1_score: 0.6320 - val_loss: 1.3130 - val_precision: 0.7921
Epoch 6/25
655/657 ————— **0s** 24ms/step - accuracy: 0.6280 - f1_score: 0.6246 - loss: 1.2792 - precision: 0.7863
Epoch 6: val_loss did not improve from 1.31299
657/657 ————— **19s** 29ms/step - accuracy: 0.6281 - f1_score: 0.6247 - loss: 1.2791 - precision: 0.7864 - val_accuracy: 0.6366 - val_f1_score: 0.6347 - val_loss: 1.3222 - val_precision: 0.7961
Epoch 7/25
657/657 ————— **0s** 24ms/step - accuracy: 0.6426 - f1_score: 0.6397 - loss: 1.2230 - precision: 0.7913
Epoch 7: val_loss did not improve from 1.31299
657/657 ————— **19s** 29ms/step - accuracy: 0.6426 - f1_score: 0.6397 - loss: 1.2230 - precision: 0.7913 - val_accuracy: 0.6351 - val_f1_score: 0.6318 - val_loss: 1.3159 - val_precision: 0.7850
Epoch 8/25
655/657 ————— **0s** 24ms/step - accuracy: 0.6556 - f1_score: 0.6525 - loss: 1.1718 - precision: 0.8004
Epoch 8: val_loss improved from 1.31299 to 1.30995, saving model to best_model_v2.keras
657/657 ————— **20s** 31ms/step - accuracy: 0.6557 - f1_score: 0.6525

- loss: 1.1717 - precision: 0.8004 - val_accuracy: 0.6378 - val_f1_score: 0.6354
 - val_loss: 1.3099 - val_precision: 0.7843
 Epoch 9/25
655/657 ————— 0s 24ms/step - accuracy: 0.6695 - f1_score: 0.6661 -
 loss: 1.1156 - precision: 0.8040
 Epoch 9: val_loss did not improve from 1.30995
657/657 ————— 19s 29ms/step - accuracy: 0.6695 - f1_score: 0.6662
 - loss: 1.1155 - precision: 0.8040 - val_accuracy: 0.6377 - val_f1_score: 0.6357
 - val_loss: 1.3151 - val_precision: 0.7800
 Epoch 10/25
656/657 ————— 0s 24ms/step - accuracy: 0.6743 - f1_score: 0.6717 -
 loss: 1.0779 - precision: 0.8088
 Epoch 10: val_loss did not improve from 1.30995
657/657 ————— 19s 29ms/step - accuracy: 0.6744 - f1_score: 0.6718
 - loss: 1.0778 - precision: 0.8088 - val_accuracy: 0.6409 - val_f1_score: 0.6398
 - val_loss: 1.3132 - val_precision: 0.7793
 Epoch 11/25
657/657 ————— 0s 24ms/step - accuracy: 0.6892 - f1_score: 0.6866 -
 loss: 1.0283 - precision: 0.8139
 Epoch 11: val_loss did not improve from 1.30995
657/657 ————— 19s 29ms/step - accuracy: 0.6893 - f1_score: 0.6866
 - loss: 1.0282 - precision: 0.8139 - val_accuracy: 0.6390 - val_f1_score: 0.6387
 - val_loss: 1.3374 - val_precision: 0.7655
 Epoch 12/25
657/657 ————— 0s 24ms/step - accuracy: 0.7005 - f1_score: 0.6979 -
 loss: 0.9880 - precision: 0.8214
 Epoch 12: val_loss did not improve from 1.30995
657/657 ————— 19s 29ms/step - accuracy: 0.7005 - f1_score: 0.6979
 - loss: 0.9880 - precision: 0.8214 - val_accuracy: 0.6378 - val_f1_score: 0.6352
 - val_loss: 1.3378 - val_precision: 0.7708
 Epoch 13/25
656/657 ————— 0s 24ms/step - accuracy: 0.7118 - f1_score: 0.7092 -
 loss: 0.9508 - precision: 0.8265
 Epoch 13: val_loss did not improve from 1.30995
657/657 ————— 19s 29ms/step - accuracy: 0.7118 - f1_score: 0.7092
 - loss: 0.9508 - precision: 0.8265 - val_accuracy: 0.6356 - val_f1_score: 0.6336
 - val_loss: 1.3466 - val_precision: 0.7645



141/141  **11s** 37ms/step - accuracy: 0.6260 - f1_score: 0.6178
- loss: 1.3274 - precision: 0.7766
Test Accuracy: 63.84%
Test Loss: 1.2974097728729248
Test Precision: 78.98%
Test F1 Scores (Per Class): [83.51647 75.97765 54.335255 41.134747 37.569054 64.36781 61.53846
66.98565 79.26829 81.52173 56.37583 51.485146 66.25767 58.031082
69.36415 49.599995 78.723404 70.046074 56.66666 54.32098 78.40909
73.09644 75.73964 77.456635 74.69879 61.052628 54.143642 46.913574
75.75757 60.645157 61.63522 62.702698 43.786976 56.852787 65.07177
20.689651 68.63905 56.730766 46.857132 88.888885 69.82249 81.52173
63.030296 68.53932 50.505043 46.540874 48.48484 58.620686 80.21977
78.53403 37.499992 71.73912 61.187206 89.2857 67.77777 30.93922
75.37688 72.04301 73.33333 55.84415 78.53403 69.346725 64.70587
67.948715 42.307686 52.439022 62.195118 57.868015 86.01036 88.34355
70.588234 72.83237 36.263733 58.139526 36.47058 81.28342 83.422455
60.465115 65.24063 65.80644 33.507847 54.450256 90.217384 67.796616
59.64912 72.27722 80.82901 76.34408 60.24096 64.80447 59.036137
65.24063 57.297295 56.209152 82.97872 65.36585 43.113766 64.2857
48.677242 76.57142]
Average Test F1 Scores:63.67

282/282  **12s** 28ms/step



	precision	recall	f1-score	support
beaver	0.83	0.84	0.84	90
dolphin	0.76	0.76	0.76	90
otter	0.57	0.52	0.54	90
seal	0.57	0.32	0.41	90
whale	0.37	0.38	0.37	90
aquarium fish	0.67	0.62	0.64	90
flatfish	0.57	0.67	0.62	90
ray	0.59	0.78	0.67	90
shark	0.88	0.72	0.79	90
trout	0.80	0.83	0.82	90
orchids	0.71	0.47	0.56	90
poppies	0.46	0.58	0.51	90
roses	0.74	0.60	0.66	90
sunflowers	0.54	0.62	0.58	90
tulips	0.72	0.67	0.69	90
bottles	0.39	0.69	0.50	90
bowls	0.76	0.82	0.79	90
cans	0.60	0.84	0.70	90
cups	0.57	0.57	0.57	90
plates	0.61	0.49	0.54	90
apples	0.80	0.77	0.78	90
mushrooms	0.67	0.80	0.73	90
oranges	0.81	0.71	0.76	90
pears	0.81	0.74	0.77	90
sweet peppers	0.82	0.69	0.75	90
clock	0.58	0.64	0.61	90
computer keyboard	0.54	0.54	0.54	90
lamp	0.52	0.42	0.47	90
telephone	0.69	0.83	0.76	90
television	0.72	0.52	0.61	90
bed	0.71	0.53	0.61	90
chair	0.61	0.64	0.63	90
couch	0.47	0.41	0.44	90
table	0.52	0.62	0.57	90
wardrobe	0.57	0.76	0.65	90
bee	0.27	0.17	0.21	90
beetle	0.73	0.64	0.69	90
butterfly	0.50	0.66	0.57	90
caterpillar	0.48	0.46	0.47	90
cockroach	0.94	0.84	0.89	90
bear	0.75	0.66	0.70	90
leopard	0.80	0.83	0.82	90
lion	0.69	0.58	0.63	90
tiger	0.69	0.68	0.69	90
wolf	0.46	0.56	0.51	90
bridge	0.54	0.41	0.47	90
castle	0.44	0.53	0.48	90
house	0.61	0.57	0.59	90
road	0.79	0.81	0.80	90
skyscraper	0.74	0.83	0.79	90
cloud	0.35	0.40	0.38	90
forest	0.70	0.73	0.72	90
mountain	0.52	0.74	0.61	90
plain	0.96	0.83	0.89	90
sea	0.68	0.68	0.68	90
camel	0.31	0.31	0.31	90
cattle	0.69	0.83	0.75	90
chimpanzee	0.70	0.74	0.72	90

elephant	0.92	0.61	0.73	90
kangaroo	0.67	0.48	0.56	90
fox	0.74	0.83	0.79	90
porcupine	0.63	0.77	0.69	90
possum	0.69	0.61	0.65	90
raccoon	0.80	0.59	0.68	90
skunk	0.50	0.37	0.42	90
crab	0.59	0.48	0.53	90
lobster	0.69	0.57	0.62	90
snail	0.53	0.62	0.57	90
spider	0.81	0.92	0.86	90
worm	0.99	0.80	0.88	90
baby	0.68	0.73	0.71	90
boy	0.76	0.70	0.73	90
girl	0.35	0.37	0.36	90
man	0.61	0.56	0.58	90
woman	0.39	0.34	0.36	90
crocodile	0.78	0.84	0.81	90
dinosaur	0.80	0.87	0.83	90
lizard	0.63	0.58	0.60	90
snake	0.63	0.68	0.65	90
turtle	0.78	0.57	0.66	90
hamster	0.31	0.36	0.33	90
mouse	0.51	0.58	0.54	90
rabbit	0.88	0.92	0.90	90
shrew	0.69	0.67	0.68	90
squirrel	0.63	0.57	0.60	90
maple	0.65	0.81	0.72	90
oak	0.76	0.87	0.81	90
palm	0.74	0.79	0.76	90
pine	0.66	0.56	0.60	90
willow	0.65	0.64	0.65	90
bicycle	0.64	0.54	0.59	90
bus	0.62	0.67	0.65	90
motorcycle	0.56	0.59	0.57	90
pickup truck	0.68	0.48	0.56	90
train	0.80	0.87	0.83	90
lawn-mower	0.58	0.74	0.65	90
rocket	0.47	0.40	0.43	90
streetcar	0.59	0.70	0.64	90
tank	0.46	0.51	0.49	90
tractor	0.79	0.74	0.77	90
accuracy			0.64	9000
macro avg	0.65	0.64	0.64	9000
weighted avg	0.65	0.64	0.64	9000

Model: "sequential_2"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None , 2048)
dense_4 (Dense)	(None , 512)
dropout_2 (Dropout)	(None , 512)
dense_5 (Dense)	(None , 100)



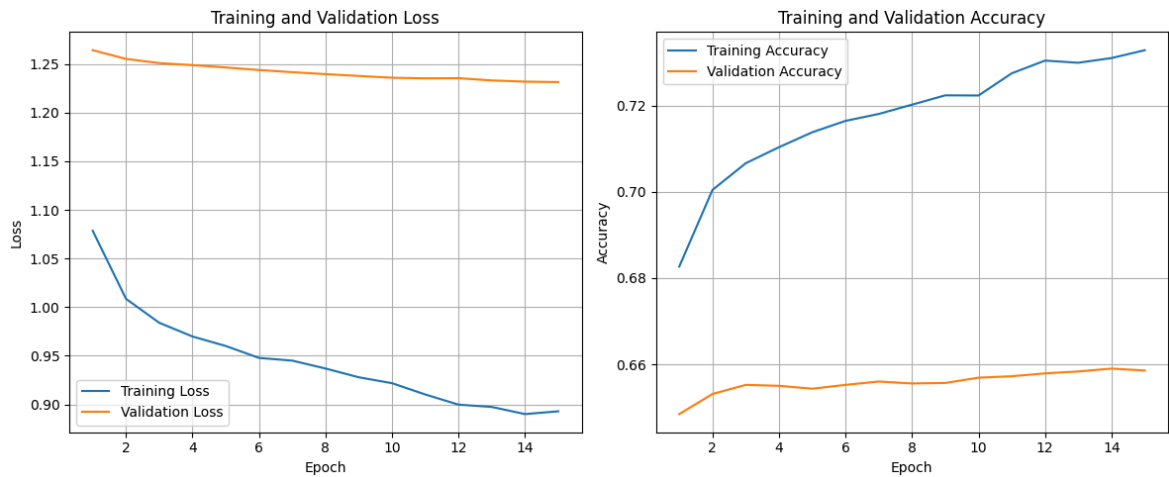
Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1313/1314 ————— 0s 27ms/step - accuracy: 0.6781 - f1_score: 0.6761
- loss: 1.0822 - precision: 0.8136
Epoch 1: val_loss improved from inf to 1.26418, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 57s 35ms/step - accuracy: 0.6781 - f1_score: 0.6761
1 - loss: 1.0822 - precision: 0.8137 - val_accuracy: 0.6484 - val_f1_score: 0.6466
6 - val_loss: 1.2642 - val_precision: 0.7936
Epoch 2/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7044 - f1_score: 0.7022
- loss: 0.9880 - precision: 0.8342
Epoch 2: val_loss improved from 1.26418 to 1.25522, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.7044 - f1_score: 0.7022
2 - loss: 0.9880 - precision: 0.8342 - val_accuracy: 0.6531 - val_f1_score: 0.6515
5 - val_loss: 1.2552 - val_precision: 0.7943
Epoch 3/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7119 - f1_score: 0.7099
- loss: 0.9557 - precision: 0.8363
Epoch 3: val_loss improved from 1.25522 to 1.25095, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.7119 - f1_score: 0.7099
9 - loss: 0.9557 - precision: 0.8363 - val_accuracy: 0.6552 - val_f1_score: 0.6537
7 - val_loss: 1.2509 - val_precision: 0.7948
Epoch 4/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7159 - f1_score: 0.7141
- loss: 0.9432 - precision: 0.8433
Epoch 4: val_loss improved from 1.25095 to 1.24881, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.7159 - f1_score: 0.7141
1 - loss: 0.9432 - precision: 0.8433 - val_accuracy: 0.6550 - val_f1_score: 0.6534
4 - val_loss: 1.2488 - val_precision: 0.7940
Epoch 5/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7204 - f1_score: 0.7186
- loss: 0.9349 - precision: 0.8445
Epoch 5: val_loss improved from 1.24881 to 1.24642, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.7204 - f1_score: 0.7185
5 - loss: 0.9349 - precision: 0.8445 - val_accuracy: 0.6543 - val_f1_score: 0.6530
0 - val_loss: 1.2464 - val_precision: 0.7927
Epoch 6/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7242 - f1_score: 0.7221
- loss: 0.9162 - precision: 0.8473
Epoch 6: val_loss improved from 1.24642 to 1.24380, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.7242 - f1_score: 0.7221
1 - loss: 0.9163 - precision: 0.8473 - val_accuracy: 0.6552 - val_f1_score: 0.6538
8 - val_loss: 1.2438 - val_precision: 0.7923
Epoch 7/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7262 - f1_score: 0.7242
- loss: 0.9090 - precision: 0.8491
Epoch 7: val_loss improved from 1.24380 to 1.24157, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.7262 - f1_score: 0.7242
2 - loss: 0.9091 - precision: 0.8490 - val_accuracy: 0.6560 - val_f1_score: 0.6546
6 - val_loss: 1.2416 - val_precision: 0.7924
Epoch 8/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7296 - f1_score: 0.7278
- loss: 0.8979 - precision: 0.8505
Epoch 8: val_loss improved from 1.24157 to 1.23956, saving model to best_model_fine_tune_v2.keras

```
ne_tune_v2.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.7296 - f1_score: 0.727
8 - loss: 0.8979 - precision: 0.8505 - val_accuracy: 0.6556 - val_f1_score: 0.654
2 - val_loss: 1.2396 - val_precision: 0.7917
Epoch 9/15
1312/1314 ————— 0s 25ms/step - accuracy: 0.7321 - f1_score: 0.7301
- loss: 0.8890 - precision: 0.8532
Epoch 9: val_loss improved from 1.23956 to 1.23769, saving model to best_model_fi
ne_tune_v2.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.7320 - f1_score: 0.730
1 - loss: 0.8891 - precision: 0.8532 - val_accuracy: 0.6557 - val_f1_score: 0.654
6 - val_loss: 1.2377 - val_precision: 0.7923
Epoch 10/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7290 - f1_score: 0.7274
- loss: 0.8854 - precision: 0.8535
Epoch 10: val_loss improved from 1.23769 to 1.23586, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.7290 - f1_score: 0.727
4 - loss: 0.8854 - precision: 0.8535 - val_accuracy: 0.6569 - val_f1_score: 0.655
6 - val_loss: 1.2359 - val_precision: 0.7916
Epoch 11/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7381 - f1_score: 0.7366
- loss: 0.8718 - precision: 0.8596
Epoch 11: val_loss improved from 1.23586 to 1.23525, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 36s 28ms/step - accuracy: 0.7381 - f1_score: 0.736
6 - loss: 0.8718 - precision: 0.8596 - val_accuracy: 0.6572 - val_f1_score: 0.655
9 - val_loss: 1.2353 - val_precision: 0.7906
Epoch 12/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7386 - f1_score: 0.7367
- loss: 0.8616 - precision: 0.8579
Epoch 12: val_loss did not improve from 1.23525
1314/1314 ————— 35s 27ms/step - accuracy: 0.7386 - f1_score: 0.736
7 - loss: 0.8617 - precision: 0.8579 - val_accuracy: 0.6579 - val_f1_score: 0.656
8 - val_loss: 1.2354 - val_precision: 0.7901
Epoch 13/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7379 - f1_score: 0.7361
- loss: 0.8634 - precision: 0.8548
Epoch 13: val_loss improved from 1.23525 to 1.23309, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7379 - f1_score: 0.736
1 - loss: 0.8634 - precision: 0.8548 - val_accuracy: 0.6583 - val_f1_score: 0.656
9 - val_loss: 1.2331 - val_precision: 0.7913
Epoch 14/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7417 - f1_score: 0.7398
- loss: 0.8496 - precision: 0.8596
Epoch 14: val_loss improved from 1.23309 to 1.23186, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7417 - f1_score: 0.739
8 - loss: 0.8497 - precision: 0.8596 - val_accuracy: 0.6590 - val_f1_score: 0.657
7 - val_loss: 1.2319 - val_precision: 0.7908
Epoch 15/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7423 - f1_score: 0.7402
- loss: 0.8550 - precision: 0.8618
Epoch 15: val_loss improved from 1.23186 to 1.23141, saving model to best_model_f
ine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7423 - f1_score: 0.740
2 - loss: 0.8550 - precision: 0.8618 - val_accuracy: 0.6586 - val_f1_score: 0.657
3 - val_loss: 1.2314 - val_precision: 0.7894
```



141/141 ————— **11s** 38ms/step - accuracy: 0.6510 - f1_score: 0.6435

- loss: 1.2580 - precision: 0.7801

Test Accuracy: 66.28%

Test Loss: 1.225183129310608

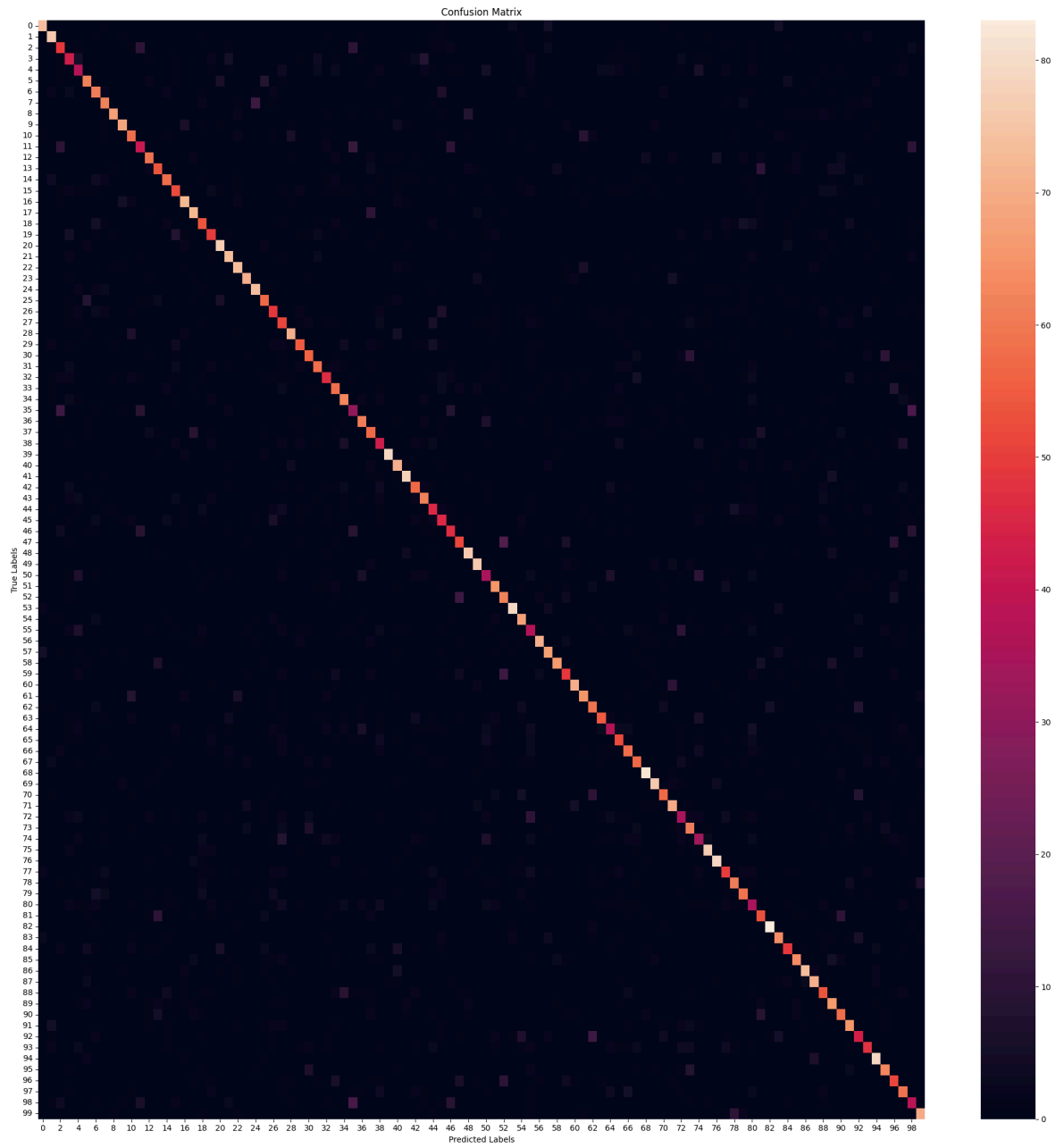
Test Precision: 79.53%

Test F1 Scores (Per Class): [83.14606 78.35052 52.68817 47.513805 41.807903 67.75956 62.5641

69.27374 82.42424 79.768776 63.73626 48.235287 68.604645 61.01694
67.44185 58.757053 77.41935 76.190475 59.66851 58.959538 81.91489
76.041664 78.49462 81.60919 80.87432 65.89595 55.05617 50.246292
79.329605 65.088745 64. 67.44185 55.621304 60.73298 67.02127
35.428566 68.926544 60.63829 48.044685 91.86046 69.95074 84.7826
65.14285 68.508286 50.82873 50.549446 51.72413 59.30232 81.72042
82.16216 40.229874 72.222206 62.944157 88.26815 69.74358 39.999996
83.5294 72.04301 79.51807 57.988155 82.08092 73.44632 62.76595
67.90123 43.636356 59.770107 63.04347 65.517235 86.77248 84.44444
68.263466 74.8663 40.46242 62.827217 39.30635 83.69564 85.86956
58.620686 69.71428 64.480865 37.894733 56.382973 91.7127 69.94535
59.756096 73.988434 82.95453 79.329605 60.67415 66.33166 63.04347
68.42105 51.764698 56.470577 85.405396 70.39106 56.35359 63.440853
41.758232 77.77777]

Average Test F1 Scores:66.22

282/282 ————— **12s** 29ms/step



	precision	recall	f1-score	support
beaver	0.84	0.82	0.83	90
dolphin	0.73	0.84	0.78	90
otter	0.51	0.54	0.53	90
seal	0.47	0.48	0.47	90
whale	0.43	0.41	0.42	90
aquarium fish	0.67	0.69	0.68	90
flatfish	0.58	0.68	0.63	90
ray	0.70	0.69	0.69	90
shark	0.91	0.76	0.82	90
trout	0.83	0.77	0.80	90
orchids	0.63	0.64	0.64	90
poppies	0.52	0.46	0.49	90
roses	0.72	0.66	0.69	90
sunflowers	0.62	0.60	0.61	90
tulips	0.71	0.64	0.67	90
bottles	0.60	0.58	0.59	90
bowls	0.75	0.80	0.77	90
cans	0.73	0.80	0.76	90
cups	0.59	0.60	0.60	90
plates	0.61	0.56	0.58	90
apples	0.79	0.86	0.82	90
mushrooms	0.72	0.81	0.76	90
oranges	0.76	0.81	0.78	90
pears	0.85	0.79	0.82	90
sweet peppers	0.80	0.82	0.81	90
clock	0.68	0.63	0.66	90
computer keyboard	0.55	0.54	0.55	90
lamp	0.45	0.57	0.50	90
telephone	0.80	0.79	0.79	90
television	0.70	0.61	0.65	90
bed	0.66	0.62	0.64	90
chair	0.71	0.64	0.67	90
couch	0.59	0.52	0.56	90
table	0.57	0.64	0.61	90
wardrobe	0.64	0.70	0.67	90
bee	0.36	0.34	0.35	90
beetle	0.70	0.68	0.69	90
butterfly	0.58	0.63	0.61	90
caterpillar	0.48	0.48	0.48	90
cockroach	0.96	0.88	0.92	90
bear	0.63	0.79	0.70	90
leopard	0.83	0.87	0.85	90
lion	0.67	0.63	0.65	90
tiger	0.68	0.69	0.69	90
wolf	0.51	0.51	0.51	90
bridge	0.49	0.51	0.50	90
castle	0.55	0.52	0.53	90
house	0.62	0.57	0.59	90
road	0.79	0.84	0.82	90
skyscraper	0.80	0.84	0.82	90
cloud	0.42	0.39	0.40	90
forest	0.72	0.72	0.72	90
mountain	0.58	0.69	0.63	90
plain	0.89	0.88	0.88	90
sea	0.65	0.76	0.70	90
camel	0.39	0.41	0.40	90
cattle	0.89	0.79	0.84	90
chimpanzee	0.70	0.74	0.72	90

elephant	0.87	0.73	0.80	90
kangaroo	0.62	0.54	0.58	90
fox	0.86	0.79	0.82	90
porcupine	0.75	0.72	0.73	90
possum	0.60	0.66	0.63	90
raccoon	0.76	0.61	0.68	90
skunk	0.48	0.40	0.44	90
crab	0.62	0.58	0.60	90
lobster	0.62	0.64	0.63	90
snail	0.69	0.63	0.66	90
spider	0.83	0.91	0.87	90
worm	0.84	0.84	0.84	90
baby	0.74	0.63	0.68	90
boy	0.72	0.78	0.75	90
girl	0.42	0.39	0.40	90
man	0.60	0.68	0.64	90
woman	0.41	0.38	0.39	90
crocodile	0.82	0.86	0.84	90
dinosaur	0.84	0.88	0.86	90
lizard	0.60	0.56	0.58	90
snake	0.72	0.68	0.70	90
turtle	0.63	0.66	0.64	90
hamster	0.35	0.39	0.37	90
mouse	0.54	0.59	0.56	90
rabbit	0.91	0.92	0.92	90
shrew	0.69	0.71	0.70	90
squirrel	0.66	0.54	0.60	90
maple	0.77	0.71	0.74	90
oak	0.85	0.81	0.83	90
palm	0.80	0.79	0.79	90
pine	0.61	0.60	0.61	90
willow	0.61	0.73	0.66	90
bicycle	0.62	0.64	0.63	90
bus	0.65	0.72	0.68	90
motorcycle	0.55	0.49	0.52	90
pickup truck	0.60	0.53	0.56	90
train	0.83	0.88	0.85	90
lawn-mower	0.71	0.70	0.70	90
rocket	0.56	0.57	0.56	90
streetcar	0.61	0.66	0.63	90
tank	0.41	0.42	0.42	90
tractor	0.78	0.78	0.78	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

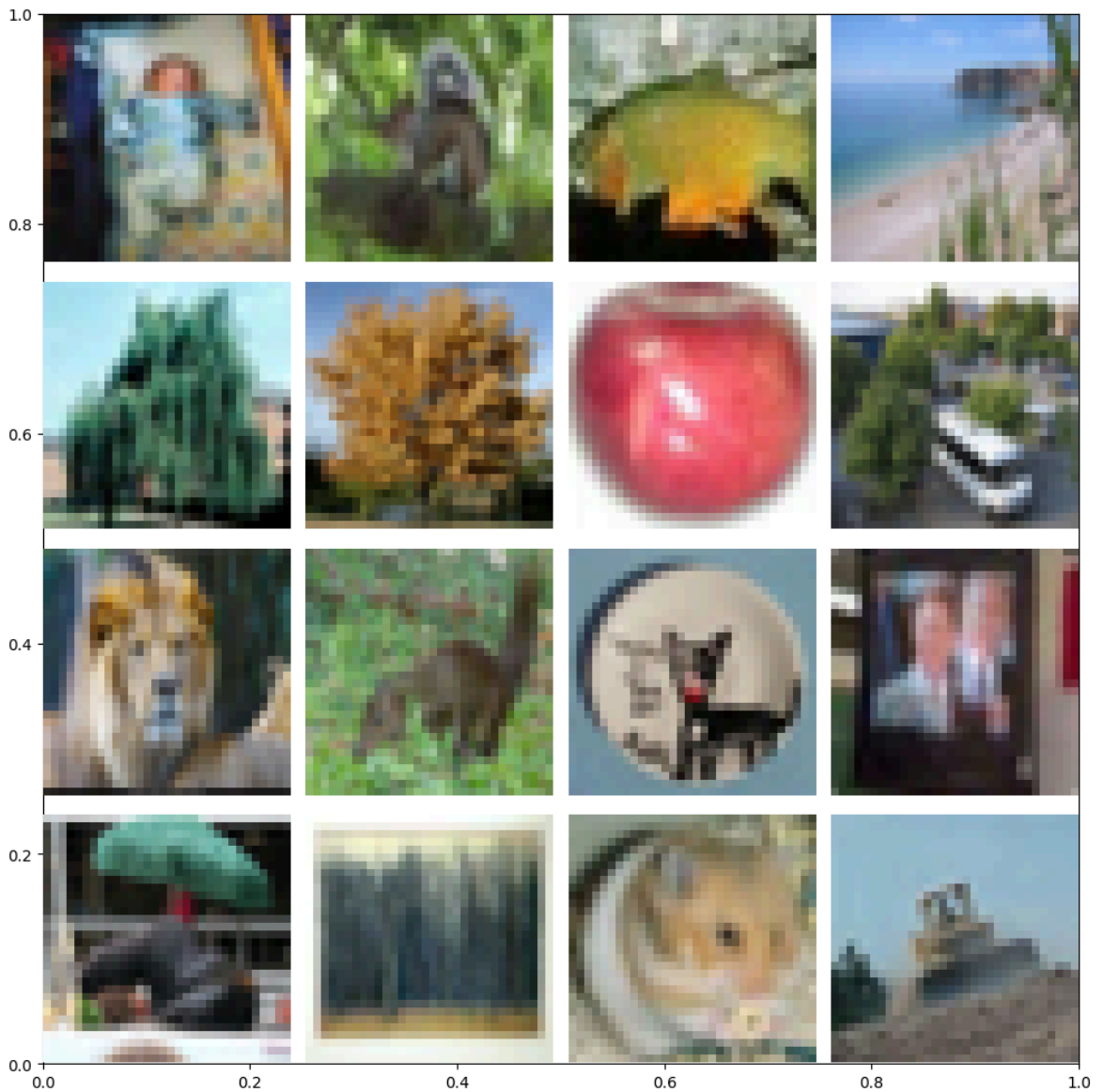
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

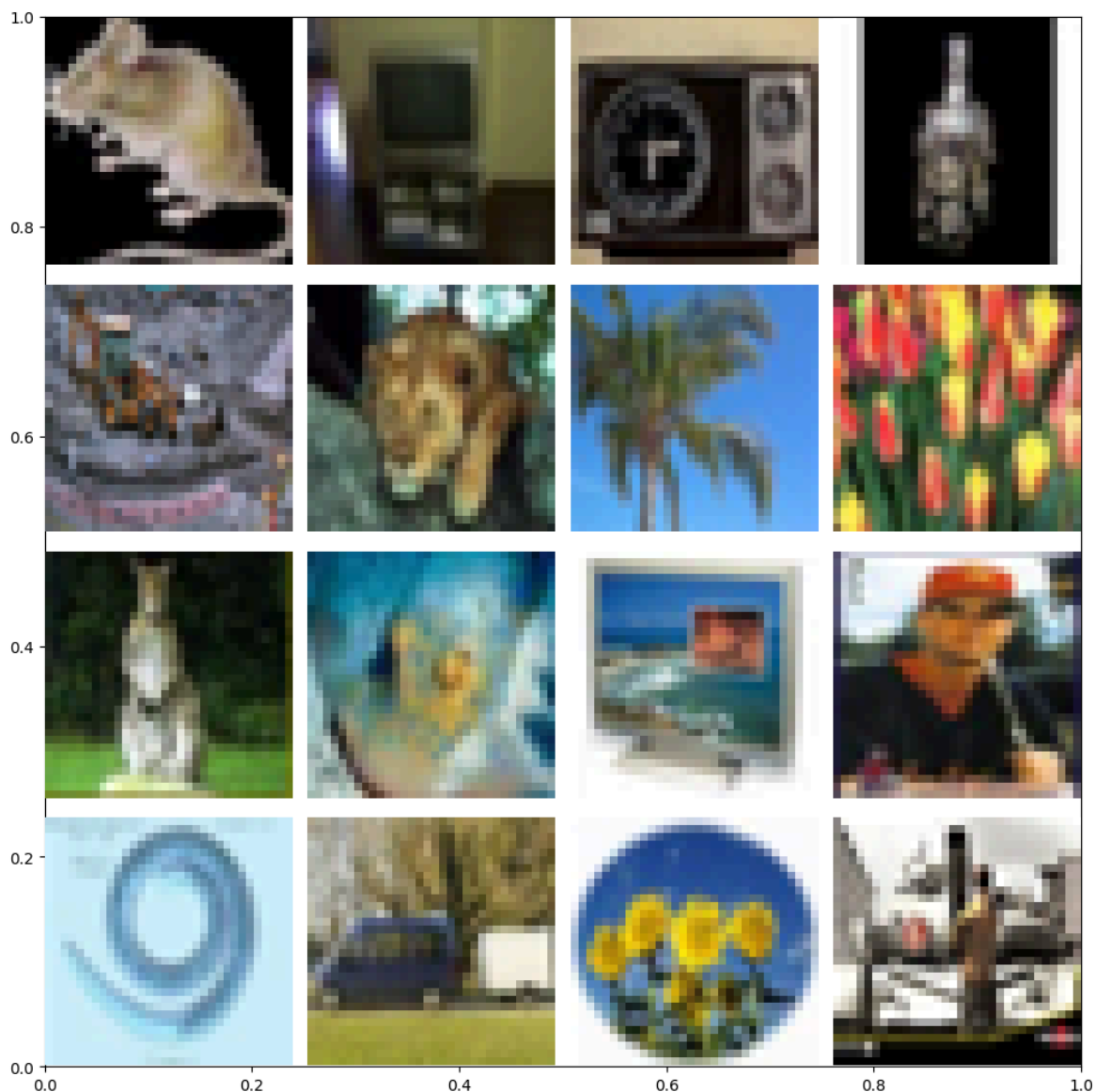
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

```
labels[25736].shape: (1,)  
imgs[25736].shape: (32, 32, 3)
```

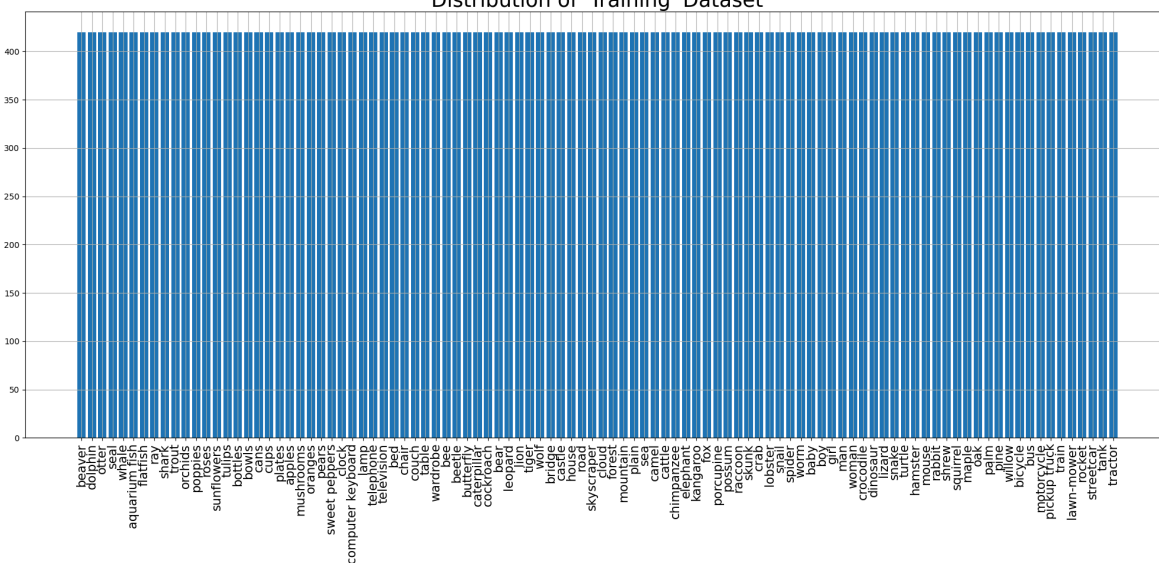


```
labels[24513].shape: (1,)  
imgs[24513].shape: (32, 32, 3)
```

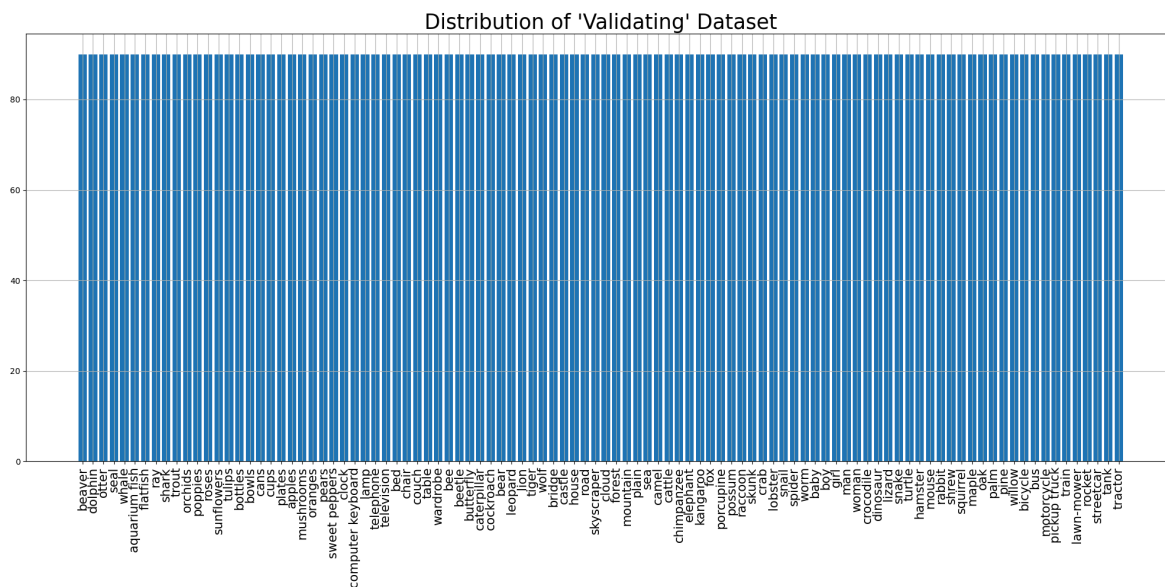


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

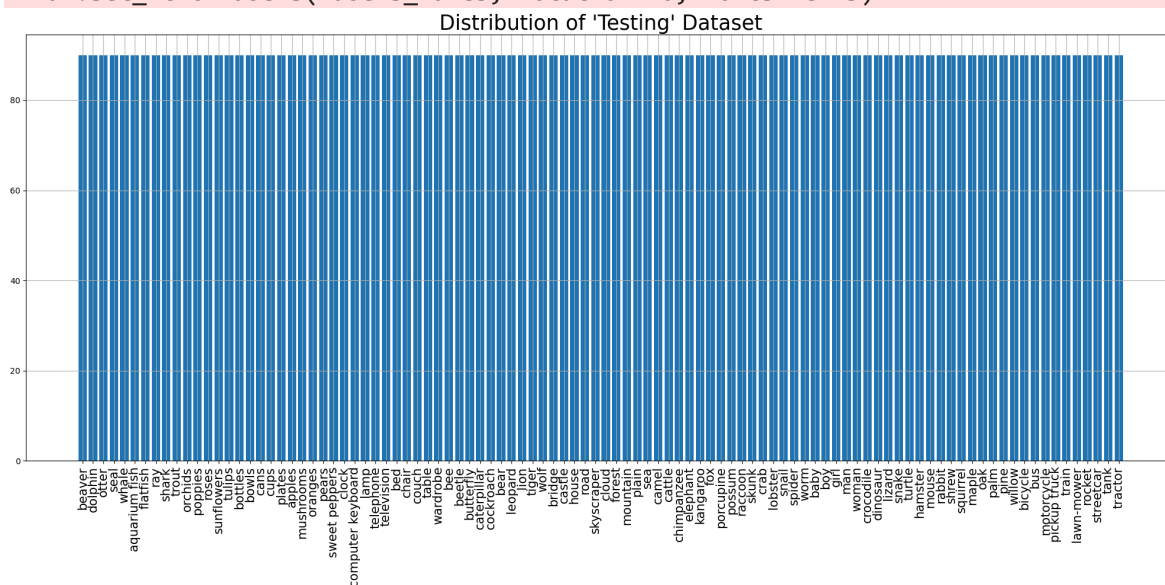
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
```

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
Image shape: (64, 224, 224, 3)
Label shape: (64, 100)
<class 'tuple'>
2
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
(64, 224, 224, 3)
(64, 100)
Model input shape: (None, 224, 224, 3)
Model output shape: (None, 100)
2
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
(64, 224, 224, 3) (64, 100)
```

Model: "sequential_3"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None , 2048)
dense_6 (Dense)	(None , 512)
dropout_3 (Dropout)	(None , 512)
dense_7 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
657/657 ————— **0s** 29ms/step - accuracy: 0.3158 - f1_score: 0.3120 - loss: 2.9066 - precision: 0.6664
Epoch 1: val_loss improved from inf to 1.51815, saving model to best_model_v2.keras as
657/657 ————— **41s** 45ms/step - accuracy: 0.3159 - f1_score: 0.3121 - loss: 2.9057 - precision: 0.6665 - val_accuracy: 0.5829 - val_f1_score: 0.5788 - val_loss: 1.5181 - val_precision: 0.8111
Epoch 2/25
656/657 ————— **0s** 24ms/step - accuracy: 0.5305 - f1_score: 0.5266 - loss: 1.7396 - precision: 0.7495
Epoch 2: val_loss improved from 1.51815 to 1.40917, saving model to best_model_v2.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.5305 - f1_score: 0.5266 - loss: 1.7395 - precision: 0.7495 - val_accuracy: 0.6060 - val_f1_score: 0.6011 - val_loss: 1.4092 - val_precision: 0.8091
Epoch 3/25
657/657 ————— **0s** 24ms/step - accuracy: 0.5699 - f1_score: 0.5671 - loss: 1.5622 - precision: 0.7648
Epoch 3: val_loss improved from 1.40917 to 1.34699, saving model to best_model_v2.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.5699 - f1_score: 0.5671 - loss: 1.5621 - precision: 0.7648 - val_accuracy: 0.6253 - val_f1_score: 0.6225 - val_loss: 1.3470 - val_precision: 0.8017
Epoch 4/25
656/657 ————— **0s** 24ms/step - accuracy: 0.5966 - f1_score: 0.5937 - loss: 1.4423 - precision: 0.7737
Epoch 4: val_loss improved from 1.34699 to 1.33248, saving model to best_model_v2.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.5966 - f1_score: 0.5938 - loss: 1.4422 - precision: 0.7737 - val_accuracy: 0.6207 - val_f1_score: 0.6182 - val_loss: 1.3325 - val_precision: 0.8042
Epoch 5/25
656/657 ————— **0s** 24ms/step - accuracy: 0.6150 - f1_score: 0.6119 - loss: 1.3477 - precision: 0.7825
Epoch 5: val_loss improved from 1.33248 to 1.31719, saving model to best_model_v2.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.6150 - f1_score: 0.6119 - loss: 1.3476 - precision: 0.7825 - val_accuracy: 0.6324 - val_f1_score: 0.6298 - val_loss: 1.3172 - val_precision: 0.7988
Epoch 6/25
655/657 ————— **0s** 24ms/step - accuracy: 0.6285 - f1_score: 0.6252 - loss: 1.2887 - precision: 0.7894
Epoch 6: val_loss improved from 1.31719 to 1.30927, saving model to best_model_v2.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.6285 - f1_score: 0.6253 - loss: 1.2886 - precision: 0.7894 - val_accuracy: 0.6350 - val_f1_score: 0.6321 - val_loss: 1.3093 - val_precision: 0.7919
Epoch 7/25
656/657 ————— **0s** 24ms/step - accuracy: 0.6414 - f1_score: 0.6384 - loss: 1.2247 - precision: 0.7956
Epoch 7: val_loss improved from 1.30927 to 1.30741, saving model to best_model_v2.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.6414 - f1_score: 0.6384 - loss: 1.2246 - precision: 0.7956 - val_accuracy: 0.6376 - val_f1_score: 0.6360 - val_loss: 1.3074 - val_precision: 0.7922
Epoch 8/25
657/657 ————— **0s** 24ms/step - accuracy: 0.6517 - f1_score: 0.6491 - loss: 1.1769 - precision: 0.7969
Epoch 8: val_loss did not improve from 1.30741

657/657 ————— **19s** 28ms/step - accuracy: 0.6517 - f1_score: 0.6491
- loss: 1.1769 - precision: 0.7969 - val_accuracy: 0.6388 - val_f1_score: 0.6366
- val_loss: 1.3166 - val_precision: 0.7788
Epoch 9/25
655/657 ————— **0s** 24ms/step - accuracy: 0.6613 - f1_score: 0.6584 -
loss: 1.1373 - precision: 0.8049
Epoch 9: val_loss improved from 1.30741 to 1.30513, saving model to best_model_v
2.keras
657/657 ————— **20s** 30ms/step - accuracy: 0.6613 - f1_score: 0.6584
- loss: 1.1372 - precision: 0.8049 - val_accuracy: 0.6428 - val_f1_score: 0.6414
- val_loss: 1.3051 - val_precision: 0.7857
Epoch 10/25
655/657 ————— **0s** 24ms/step - accuracy: 0.6804 - f1_score: 0.6783 -
loss: 1.0646 - precision: 0.8099
Epoch 10: val_loss did not improve from 1.30513
657/657 ————— **19s** 28ms/step - accuracy: 0.6804 - f1_score: 0.6783
- loss: 1.0646 - precision: 0.8099 - val_accuracy: 0.6442 - val_f1_score: 0.6408
- val_loss: 1.3135 - val_precision: 0.7838
Epoch 11/25
655/657 ————— **0s** 24ms/step - accuracy: 0.6870 - f1_score: 0.6844 -
loss: 1.0420 - precision: 0.8160
Epoch 11: val_loss did not improve from 1.30513
657/657 ————— **19s** 28ms/step - accuracy: 0.6870 - f1_score: 0.6845
- loss: 1.0418 - precision: 0.8160 - val_accuracy: 0.6473 - val_f1_score: 0.6456
- val_loss: 1.3203 - val_precision: 0.7735
Epoch 12/25
657/657 ————— **0s** 24ms/step - accuracy: 0.6998 - f1_score: 0.6968 -
loss: 0.9933 - precision: 0.8154
Epoch 12: val_loss did not improve from 1.30513
657/657 ————— **19s** 28ms/step - accuracy: 0.6998 - f1_score: 0.6968
- loss: 0.9933 - precision: 0.8154 - val_accuracy: 0.6434 - val_f1_score: 0.6425
- val_loss: 1.3293 - val_precision: 0.7679
Epoch 13/25
657/657 ————— **0s** 24ms/step - accuracy: 0.7030 - f1_score: 0.7005 -
loss: 0.9695 - precision: 0.8217
Epoch 13: val_loss did not improve from 1.30513
657/657 ————— **19s** 28ms/step - accuracy: 0.7030 - f1_score: 0.7005
- loss: 0.9695 - precision: 0.8217 - val_accuracy: 0.6426 - val_f1_score: 0.6410
- val_loss: 1.3400 - val_precision: 0.7669
Epoch 14/25
656/657 ————— **0s** 24ms/step - accuracy: 0.7154 - f1_score: 0.7134 -
loss: 0.9287 - precision: 0.8287
Epoch 14: val_loss did not improve from 1.30513
657/657 ————— **19s** 28ms/step - accuracy: 0.7154 - f1_score: 0.7135
- loss: 0.9287 - precision: 0.8287 - val_accuracy: 0.6453 - val_f1_score: 0.6446
- val_loss: 1.3499 - val_precision: 0.7671



141/141 ————— **11s** 37ms/step - accuracy: 0.6294 - f1_score: 0.6204

- loss: 1.3492 - precision: 0.7660

Test Accuracy: 63.93%

Test Loss: 1.3208928108215332

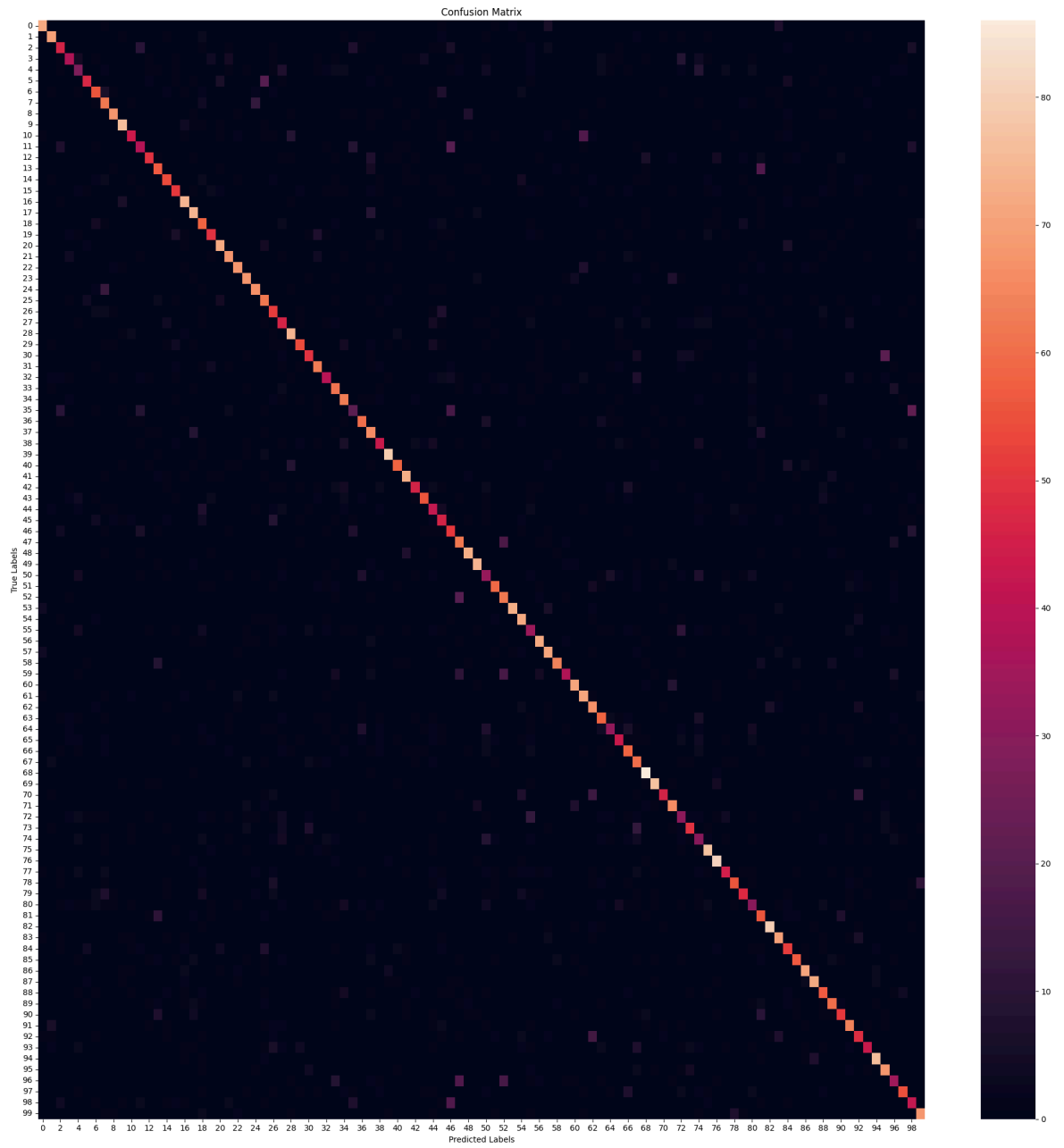
Test Precision: 78.08%

Test F1 Scores (Per Class): [81.81818 76.92307 51.977394 48.684204 33.93939 6
1.93548 61.538452

59.330147 79.53216 83.060104 58.278145 45.508976 65.359474 59.685856
63.905323 60.355026 77.08333 73.63183 54.2056 60.24096 78.260864
75.138115 77.71428 76.66666 74.03315 58.767765 51.74129 47.179485
77.89473 66.666664 61.349686 69.99999 51.315792 60.396034 62.944157
28.378374 67.796616 60.273968 51.497 85.86956 69.461075 78.49462
60.52631 70.88607 44.444443 46.46464 47.663544 57.67442 80.21977
80.87432 36.464085 69.82249 60.78431 86.904755 70.243904 36.464085
77.83783 73.19587 78.048775 52.482265 78.88888 71.35678 65.04854
62.032085 43.243237 54.77706 60.73298 54.05405 83.49514 85.08286
63.013695 71.73912 34.44444 62.893074 31.746027 78.57142 81.81817
58.064507 69.135796 60.75949 35.29411 53.846146 88.888885 72.251305
55.026455 70.37037 79.775276 78.68852 61.290314 68.965515 62.576683
68.449196 53.551907 55.48386 84.44444 69.03552 42.682922 66.66666
43.97905 75.55555]

Average Test F1 Scores:63.78

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.84	0.80	0.82	90
dolphin	0.76	0.78	0.77	90
otter	0.53	0.51	0.52	90
seal	0.60	0.41	0.49	90
whale	0.37	0.31	0.34	90
aquarium fish	0.73	0.52	0.61	90
flatfish	0.61	0.62	0.62	90
ray	0.52	0.69	0.59	90
shark	0.84	0.76	0.80	90
trout	0.82	0.84	0.83	90
orchids	0.72	0.49	0.58	90
poppies	0.50	0.42	0.46	90
roses	0.78	0.56	0.65	90
sunflowers	0.56	0.63	0.60	90
tulips	0.68	0.60	0.64	90
bottles	0.65	0.57	0.60	90
bowls	0.73	0.82	0.77	90
cans	0.67	0.82	0.74	90
cups	0.47	0.64	0.54	90
plates	0.66	0.56	0.60	90
apples	0.77	0.80	0.78	90
mushrooms	0.75	0.76	0.75	90
oranges	0.81	0.76	0.78	90
pears	0.76	0.76	0.76	90
sweet peppers	0.74	0.74	0.74	90
clock	0.51	0.69	0.59	90
computer keyboard	0.46	0.58	0.51	90
lamp	0.44	0.51	0.47	90
telephone	0.74	0.82	0.78	90
television	0.75	0.60	0.67	90
bed	0.68	0.56	0.61	90
chair	0.70	0.70	0.70	90
couch	0.64	0.43	0.52	90
table	0.54	0.68	0.60	90
wardrobe	0.58	0.70	0.64	90
bee	0.36	0.23	0.28	90
beetle	0.69	0.67	0.68	90
butterfly	0.52	0.73	0.61	90
caterpillar	0.57	0.48	0.52	90
cockroach	0.84	0.88	0.86	90
bear	0.75	0.64	0.69	90
leopard	0.76	0.81	0.78	90
lion	0.74	0.51	0.61	90
tiger	0.82	0.62	0.71	90
wolf	0.42	0.47	0.44	90
bridge	0.43	0.51	0.46	90
castle	0.41	0.57	0.47	90
house	0.50	0.69	0.58	90
road	0.79	0.81	0.80	90
skyscraper	0.80	0.82	0.81	90
cloud	0.37	0.37	0.37	90
forest	0.75	0.66	0.70	90
mountain	0.54	0.69	0.61	90
plain	0.94	0.81	0.87	90
sea	0.63	0.80	0.70	90
camel	0.37	0.38	0.37	90
cattle	0.76	0.80	0.78	90
chimpanzee	0.68	0.79	0.73	90

elephant	0.86	0.71	0.78	90
kangaroo	0.73	0.41	0.52	90
fox	0.78	0.79	0.78	90
porcupine	0.65	0.79	0.71	90
possum	0.58	0.74	0.65	90
raccoon	0.60	0.64	0.62	90
skunk	0.55	0.36	0.43	90
crab	0.63	0.48	0.54	90
lobster	0.57	0.64	0.61	90
snail	0.45	0.67	0.54	90
spider	0.74	0.96	0.83	90
worm	0.85	0.86	0.85	90
baby	0.82	0.51	0.63	90
boy	0.70	0.73	0.72	90
girl	0.35	0.34	0.35	90
man	0.72	0.56	0.63	90
woman	0.30	0.33	0.32	90
crocodile	0.73	0.86	0.79	90
dinosaur	0.75	0.90	0.82	90
lizard	0.69	0.50	0.58	90
snake	0.78	0.62	0.69	90
turtle	0.71	0.53	0.61	90
hamster	0.37	0.33	0.35	90
mouse	0.47	0.62	0.54	90
rabbit	0.89	0.89	0.89	90
shrew	0.69	0.78	0.73	90
squirrel	0.52	0.58	0.55	90
maple	0.79	0.63	0.70	90
oak	0.81	0.79	0.80	90
palm	0.77	0.80	0.79	90
pine	0.59	0.63	0.61	90
willow	0.71	0.67	0.69	90
bicycle	0.70	0.57	0.63	90
bus	0.66	0.71	0.68	90
motorcycle	0.53	0.54	0.54	90
pickup truck	0.66	0.48	0.55	90
train	0.84	0.84	0.84	90
lawn-mower	0.64	0.76	0.69	90
rocket	0.47	0.39	0.42	90
streetcar	0.73	0.61	0.67	90
tank	0.42	0.47	0.44	90
tractor	0.76	0.76	0.76	90
accuracy			0.64	9000
macro avg	0.65	0.64	0.64	9000
weighted avg	0.65	0.64	0.64	9000

Model: "sequential_3"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None , 2048)
dense_6 (Dense)	(None , 512)
dropout_3 (Dropout)	(None , 512)
dense_7 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1312/1314 ————— 0s 27ms/step - accuracy: 0.6848 - f1_score: 0.6822
- loss: 1.0585 - precision: 0.8185
Epoch 1: val_loss improved from inf to 1.25814, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 57s 35ms/step - accuracy: 0.6848 - f1_score: 0.6822 - loss: 1.0586 - precision: 0.8185 - val_accuracy: 0.6562 - val_f1_score: 0.6554 - val_loss: 1.2581 - val_precision: 0.7968
Epoch 2/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7103 - f1_score: 0.7084 - loss: 0.9666 - precision: 0.8365
Epoch 2: val_loss improved from 1.25814 to 1.24949, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7103 - f1_score: 0.7084 - loss: 0.9666 - precision: 0.8365 - val_accuracy: 0.6564 - val_f1_score: 0.6557 - val_loss: 1.2495 - val_precision: 0.7962
Epoch 3/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7182 - f1_score: 0.7163 - loss: 0.9272 - precision: 0.8421
Epoch 3: val_loss improved from 1.24949 to 1.24600, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7182 - f1_score: 0.7163 - loss: 0.9272 - precision: 0.8420 - val_accuracy: 0.6593 - val_f1_score: 0.6587 - val_loss: 1.2460 - val_precision: 0.7979
Epoch 4/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7232 - f1_score: 0.7215 - loss: 0.9145 - precision: 0.8476
Epoch 4: val_loss improved from 1.24600 to 1.24323, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7232 - f1_score: 0.7215 - loss: 0.9146 - precision: 0.8476 - val_accuracy: 0.6583 - val_f1_score: 0.6576 - val_loss: 1.2432 - val_precision: 0.7985
Epoch 5/15
1312/1314 ————— 0s 24ms/step - accuracy: 0.7264 - f1_score: 0.7247 - loss: 0.9031 - precision: 0.8481
Epoch 5: val_loss improved from 1.24323 to 1.24121, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7264 - f1_score: 0.7247 - loss: 0.9032 - precision: 0.8481 - val_accuracy: 0.6604 - val_f1_score: 0.6596 - val_loss: 1.2412 - val_precision: 0.7964
Epoch 6/15
1314/1314 ————— 0s 24ms/step - accuracy: 0.7315 - f1_score: 0.7295 - loss: 0.8909 - precision: 0.8507
Epoch 6: val_loss improved from 1.24121 to 1.23858, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7314 - f1_score: 0.7295 - loss: 0.8909 - precision: 0.8507 - val_accuracy: 0.6601 - val_f1_score: 0.6595 - val_loss: 1.2386 - val_precision: 0.7977
Epoch 7/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7303 - f1_score: 0.7286 - loss: 0.8819 - precision: 0.8497
Epoch 7: val_loss improved from 1.23858 to 1.23620, saving model to best_model_fine_tune_v2.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.7303 - f1_score: 0.7286 - loss: 0.8820 - precision: 0.8497 - val_accuracy: 0.6599 - val_f1_score: 0.6592 - val_loss: 1.2362 - val_precision: 0.7971
Epoch 8/15
1313/1314 ————— 0s 24ms/step - accuracy: 0.7355 - f1_score: 0.7339 - loss: 0.8673 - precision: 0.8537
Epoch 8: val_loss improved from 1.23620 to 1.23477, saving model to best_model_fine_tune_v2.keras

ne_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7355 - f1_score: 0.7339 - loss: 0.8674 - precision: 0.8537 - val_accuracy: 0.6606 - val_f1_score: 0.6600 - val_loss: 1.2348 - val_precision: 0.7975

Epoch 9/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.7418 - f1_score: 0.7402 - loss: 0.8516 - precision: 0.8593

Epoch 9: val_loss improved from 1.23477 to 1.23318, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7418 - f1_score: 0.7402 - loss: 0.8517 - precision: 0.8593 - val_accuracy: 0.6602 - val_f1_score: 0.6597 - val_loss: 1.2332 - val_precision: 0.7977

Epoch 10/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.7408 - f1_score: 0.7391 - loss: 0.8532 - precision: 0.8564

Epoch 10: val_loss improved from 1.23318 to 1.23202, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7408 - f1_score: 0.7391 - loss: 0.8533 - precision: 0.8564 - val_accuracy: 0.6620 - val_f1_score: 0.6614 - val_loss: 1.2320 - val_precision: 0.7967

Epoch 11/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.7451 - f1_score: 0.7435 - loss: 0.8411 - precision: 0.8586

Epoch 11: val_loss improved from 1.23202 to 1.23152, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7451 - f1_score: 0.7435 - loss: 0.8412 - precision: 0.8585 - val_accuracy: 0.6622 - val_f1_score: 0.6614 - val_loss: 1.2315 - val_precision: 0.7951

Epoch 12/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.7484 - f1_score: 0.7466 - loss: 0.8285 - precision: 0.8627

Epoch 12: val_loss improved from 1.23152 to 1.23089, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7484 - f1_score: 0.7466 - loss: 0.8285 - precision: 0.8627 - val_accuracy: 0.6627 - val_f1_score: 0.6620 - val_loss: 1.2309 - val_precision: 0.7955

Epoch 13/15

1314/1314 ————— 0s 24ms/step - accuracy: 0.7471 - f1_score: 0.7454 - loss: 0.8300 - precision: 0.8628

Epoch 13: val_loss improved from 1.23089 to 1.22943, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7471 - f1_score: 0.7454 - loss: 0.8300 - precision: 0.8628 - val_accuracy: 0.6632 - val_f1_score: 0.6626 - val_loss: 1.2294 - val_precision: 0.7966

Epoch 14/15

1312/1314 ————— 0s 24ms/step - accuracy: 0.7516 - f1_score: 0.7499 - loss: 0.8227 - precision: 0.8658

Epoch 14: val_loss improved from 1.22943 to 1.22802, saving model to best_model_fine_tune_v2.keras

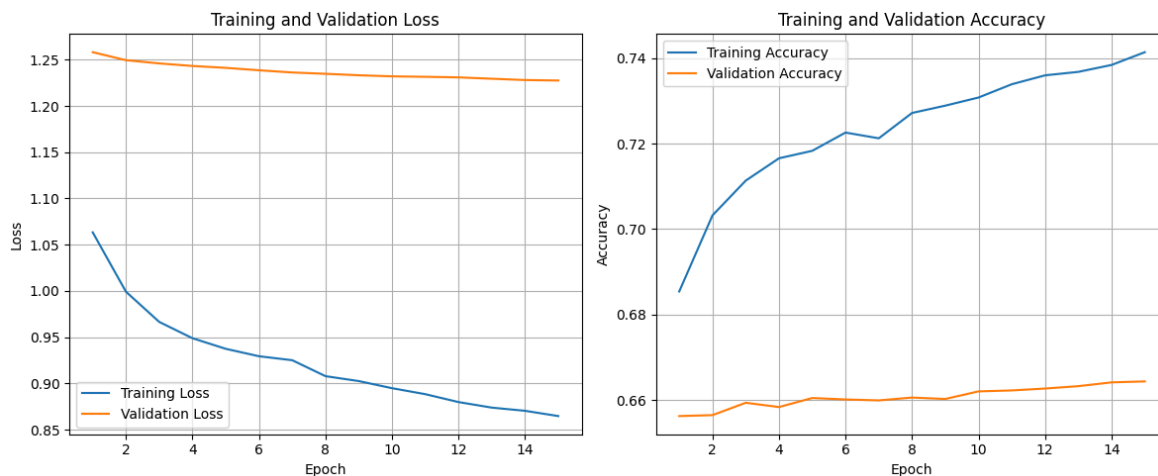
1314/1314 ————— 36s 27ms/step - accuracy: 0.7516 - f1_score: 0.7499 - loss: 0.8228 - precision: 0.8657 - val_accuracy: 0.6641 - val_f1_score: 0.6632 - val_loss: 1.2280 - val_precision: 0.7969

Epoch 15/15

1313/1314 ————— 0s 24ms/step - accuracy: 0.7552 - f1_score: 0.7535 - loss: 0.8176 - precision: 0.8654

Epoch 15: val_loss improved from 1.22802 to 1.22752, saving model to best_model_fine_tune_v2.keras

1314/1314 ————— 36s 27ms/step - accuracy: 0.7551 - f1_score: 0.7535 - loss: 0.8177 - precision: 0.8654 - val_accuracy: 0.6643 - val_f1_score: 0.6637 - val_loss: 1.2275 - val_precision: 0.7972



141/141 ————— **11s** 37ms/step - accuracy: 0.6514 - f1_score: 0.6439

- loss: 1.2624 - precision: 0.7851

Test Accuracy: 66.33%

Test Loss: 1.235614538192749

Test Precision: 79.41%

Test F1 Scores (Per Class): [87.49999 80.42328 55.737698 51.64835 38.150284 68.888885 63.87434

68.13186 79.99999 84.090904 61.176468 46.15384 67.816086 61.87845

68.208084 57.627117 78.947365 77.08333 58.959538 56.80473 82.16216

75.89743 79.12087 79.329605 78.918915 64.739876 51.111107 48.73096

80. 70.44025 64.77273 70.520226 51.219505 60.10362 67.3913

34.319523 67.77777 62.499992 54.545444 89.017334 71.13402 80.87432

63.82978 71.67629 45.026173 48.044685 46.32768 60.919537 80.20833

80.66298 40.909084 74.157295 63.21243 90.39548 69.99999 39.3617

82.84023 75.82417 79.51807 56.80473 78.02197 72.625694 63.04347

67.85713 45.12195 49.71751 59.574467 65.11627 87.23404 85.41666

70.58823 75.675674 38.787876 64.921455 36.871502 82.22221 87.91208

59.523804 67.0391 67.75956 38.20224 59.574467 91.7127 71.50259

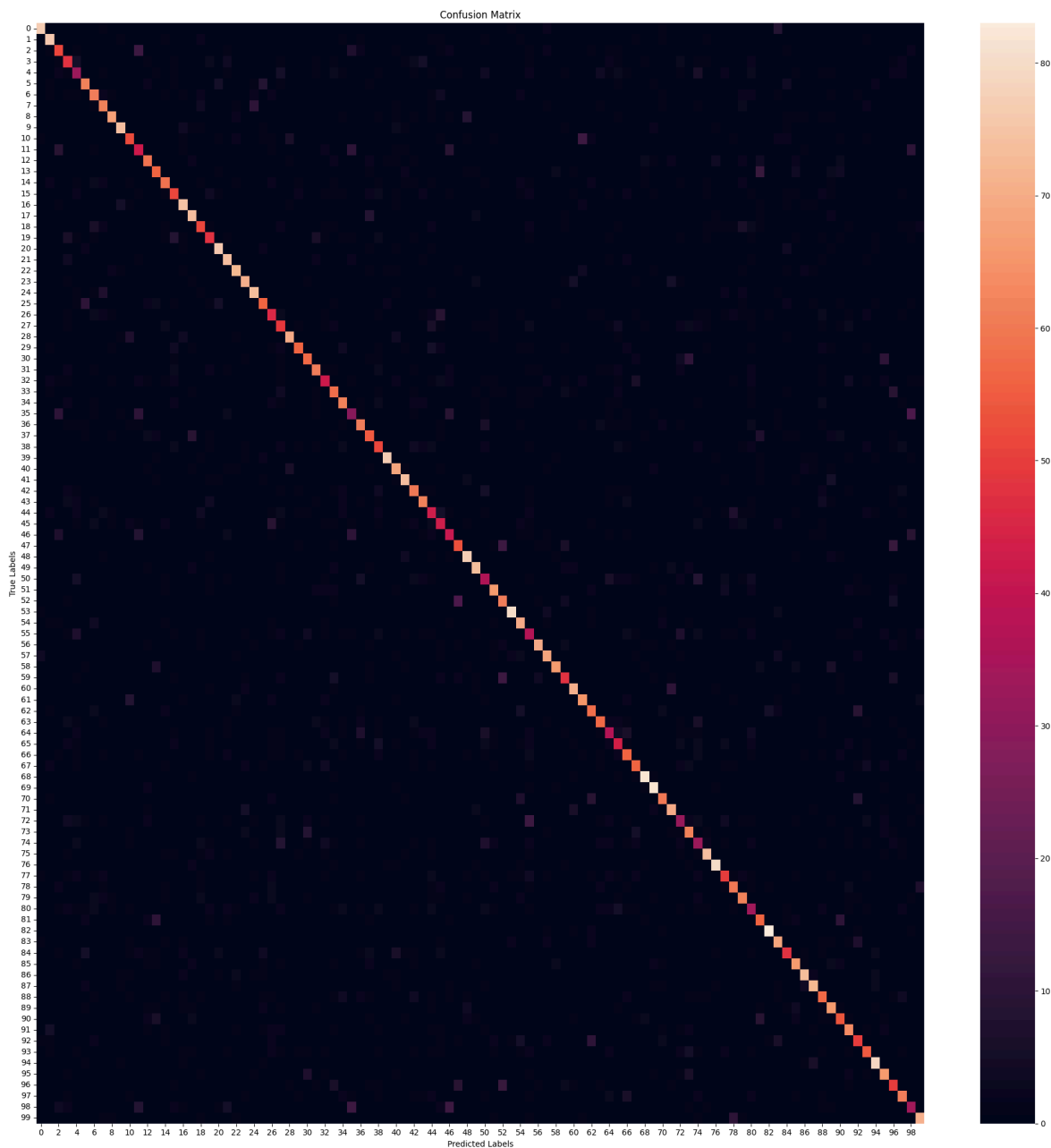
58.895695 71.35134 81.56424 80.89887 61.290314 68.39378 59.999996

69.94535 57.471264 63.218384 84.946236 72.13114 52.910053 67.03297

39.30635 79.775276]

Average Test F1 Scores:66.23

282/282 ————— **11s** 28ms/step



	precision	recall	f1-score	support
beaver	0.90	0.86	0.88	90
dolphin	0.77	0.84	0.80	90
otter	0.55	0.57	0.56	90
seal	0.52	0.53	0.52	90
whale	0.40	0.37	0.38	90
aquarium fish	0.69	0.69	0.69	90
flatfish	0.60	0.68	0.64	90
ray	0.67	0.69	0.68	90
shark	0.85	0.76	0.80	90
trout	0.86	0.82	0.84	90
orchids	0.65	0.58	0.61	90
poppies	0.46	0.47	0.46	90
roses	0.70	0.66	0.68	90
sunflowers	0.62	0.62	0.62	90
tulips	0.71	0.66	0.68	90
bottles	0.59	0.57	0.58	90
bowls	0.75	0.83	0.79	90
cans	0.73	0.82	0.77	90
cups	0.61	0.57	0.59	90
plates	0.61	0.53	0.57	90
apples	0.80	0.84	0.82	90
mushrooms	0.70	0.82	0.76	90
oranges	0.78	0.80	0.79	90
pears	0.80	0.79	0.79	90
sweet peppers	0.77	0.81	0.79	90
clock	0.67	0.62	0.65	90
computer keyboard	0.51	0.51	0.51	90
lamp	0.45	0.54	0.49	90
telephone	0.82	0.78	0.80	90
television	0.81	0.62	0.70	90
bed	0.66	0.63	0.65	90
chair	0.73	0.68	0.71	90
couch	0.57	0.47	0.51	90
table	0.56	0.64	0.60	90
wardrobe	0.66	0.69	0.67	90
bee	0.37	0.32	0.34	90
beetle	0.68	0.68	0.68	90
butterfly	0.64	0.61	0.62	90
caterpillar	0.53	0.57	0.55	90
cockroach	0.93	0.86	0.89	90
bear	0.66	0.77	0.71	90
leopard	0.80	0.82	0.81	90
lion	0.61	0.67	0.64	90
tiger	0.75	0.69	0.72	90
wolf	0.43	0.48	0.45	90
bridge	0.48	0.48	0.48	90
castle	0.47	0.46	0.46	90
house	0.62	0.59	0.61	90
road	0.75	0.86	0.80	90
skyscraper	0.80	0.81	0.81	90
cloud	0.42	0.40	0.41	90
forest	0.75	0.73	0.74	90
mountain	0.59	0.68	0.63	90
plain	0.92	0.89	0.90	90
sea	0.64	0.78	0.70	90
camel	0.38	0.41	0.39	90
cattle	0.89	0.78	0.83	90
chimpanzee	0.75	0.77	0.76	90

elephant	0.87	0.73	0.80	90
kangaroo	0.62	0.53	0.57	90
fox	0.77	0.79	0.78	90
porcupine	0.73	0.72	0.73	90
possum	0.62	0.64	0.63	90
raccoon	0.73	0.63	0.68	90
skunk	0.49	0.41	0.45	90
crab	0.51	0.49	0.50	90
lobster	0.57	0.62	0.60	90
snail	0.68	0.62	0.65	90
spider	0.84	0.91	0.87	90
worm	0.80	0.91	0.85	90
baby	0.75	0.67	0.71	90
boy	0.74	0.78	0.76	90
girl	0.43	0.36	0.39	90
man	0.61	0.69	0.65	90
woman	0.37	0.37	0.37	90
crocodile	0.82	0.82	0.82	90
dinosaur	0.87	0.89	0.88	90
lizard	0.64	0.56	0.60	90
snake	0.67	0.67	0.67	90
turtle	0.67	0.69	0.68	90
hamster	0.39	0.38	0.38	90
mouse	0.57	0.62	0.60	90
rabbit	0.91	0.92	0.92	90
shrew	0.67	0.77	0.72	90
squirrel	0.66	0.53	0.59	90
maple	0.69	0.73	0.71	90
oak	0.82	0.81	0.82	90
palm	0.82	0.80	0.81	90
pine	0.59	0.63	0.61	90
willow	0.64	0.73	0.68	90
bicycle	0.60	0.60	0.60	90
bus	0.69	0.71	0.70	90
motorcycle	0.60	0.56	0.57	90
pickup truck	0.65	0.61	0.63	90
train	0.82	0.88	0.85	90
lawn-mower	0.71	0.73	0.72	90
rocket	0.51	0.56	0.53	90
streetcar	0.67	0.68	0.67	90
tank	0.41	0.38	0.39	90
tractor	0.81	0.79	0.80	90
accuracy			0.66	9000
macro avg	0.66	0.66	0.66	9000
weighted avg	0.66	0.66	0.66	9000

Dropout(0.6), L2Reg(3e-2), weight_decay(3e-2), train_augmentation

```
In [9]: for repeat_2_times in range(2):
        ##### <<<<<<<<<Load and process data>>>>>>>>>>>>
        # Load CIFAR-100 dataset
        (X_train, y_train), (X_test, y_test) = cifar100.load_data(label_mode='fine')

        # Split (8000) of training data into temporary set
```

```

X_temp, X_train, y_temp, y_train = train_test_split(X_train, y_train, test_s
print(f"X_temp.shape: {X_temp.shape}\n")

# Split temp data into equal validation (4000) and testing (4000) data
X_temp_val, X_temp_test, y_temp_val, y_temp_test = train_test_split(X_temp,
print(f"X_temp_val.shape: {X_temp_val.shape}")
print(f"y_temp_val.shape: {y_temp_val.shape}")
print(f"X_temp_test.shape: {X_temp_test.shape}")
print(f"y_temp_test.shape: {y_temp_test.shape}\n")

# Split test data into validation (5000) and testing (5000)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.

# Add temp_val to validation (9000) and temp_test to testing (9000) to get a
X_val = np.concatenate((X_val, X_temp_val), axis=0)
y_val = np.concatenate((y_val, y_temp_val), axis=0)
X_test = np.concatenate((X_test, X_temp_test), axis=0)
y_test = np.concatenate((y_test, y_temp_test), axis=0)

print(f"X_train.shape: {X_train.shape}")
print(f"y_train.shape: {y_train.shape}")
print(f"X_val.shape: {X_val.shape}")
print(f"y_val.shape: {y_val.shape}")
print(f"X_test.shape: {X_test.shape}")
print(f"y_test.shape: {y_test.shape}\n")

display_imgs(X_train, y_train)

# Normalise images (scale to range [0, 1]) - Improves convergence speed & ac
X_train, X_val, X_test = X_train / 255.0, X_val / 255.0, X_test / 255.0
display_imgs(X_train, y_train)

labels_names = ['beaver', 'dolphin', 'otter', 'seal', 'whale', 'aquarium fish', 'f
                'orchids', 'poppies', 'roses', 'sunflowers', 'tulips', 'bottles', '
                'apples', 'mushrooms', 'oranges', 'pears', 'sweet peppers', 'clock
                'telephone', 'television', 'bed', 'chair', 'couch', 'table', 'wardr
                'caterpillar', 'cockroach', 'bear', 'leopard', 'lion', 'tiger', 'wo
                'road', 'skyscraper', 'cloud', 'forest', 'mountain', 'plain', 'sea'
                'elephant', 'kangaroo', 'fox', 'porcupine', 'possum', 'raccoon', 's
                'spider', 'worm', 'baby', 'boy', 'girl', 'man', 'woman', 'crocodile'
                'turtle', 'hamster', 'mouse', 'rabbit', 'shrew', 'squirrel', 'maple
                'bicycle', 'bus', 'motorcycle', 'pickup truck', 'train', 'lawn-mow
                'tractor']

class_distrib(y_train, labels_names, "Training")
class_distrib(y_val, labels_names, "Validating")
class_distrib(y_test, labels_names, "Testing")

# Create TensorFlow datasets

batch_size = 64

train_dataset_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                  .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                     tf.squeeze(tf.one_hot(y, depth=100, dtype=
                  .batch(batch_size)
                  .prefetch(tf.data.experimental.AUTOTUNE)))

# DEBUG augment
train_dataset_aug = (tf.data.Dataset.from_tensor_slices((X_train, y_train))

```

```

        .map(augment_dataset, num_parallel_calls=tf.data.experimental
        .batch(batch_size)
        .prefetch(tf.data.experimental.AUTOTUNE))

# Combine the original dataset and the augmented dataset
train_dataset = train_dataset_.concatenate(train_dataset_aug)

val_dataset = (tf.data.Dataset.from_tensor_slices((X_val, y_val))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

test_dataset = (tf.data.Dataset.from_tensor_slices((X_test, y_test))
               .map(lambda x, y: (tf.image.resize(x, (224, 224)),
                                   tf.squeeze(tf.one_hot(y, depth=100, dtype=
               .batch(batch_size)
               .prefetch(tf.data.experimental.AUTOTUNE)))

print(f"Training dataset:\n {train_dataset}")
for img, lbl in train_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nValidation dataset:\n {val_dataset}")
for img, lbl in val_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl
print(f"\nTesting dataset:\n {test_dataset}")
for img, lbl in test_dataset.take(1):
    #if isinstance(batch, tuple) and len(batch) == 2:
    print(f"Image shape: {img.shape}") # Should be (batch_size, 224, 224, 3
    print(f"Label shape: {lbl.shape}") # Should be (batch_size, 10)
    del img, lbl

#### <<<<<<<<<Pre-trained model>>>>>>>>>
# Load ResNet50 pre-trained on ImageNet (w/out the top classification layer
resnet_50_base = tf.keras.applications.ResNet50V2(weights='imagenet', includ

# Freeze the Layers of VGG16 so they don't get updated during training - can
resnet_50_base.trainable = False

for layer in resnet_50_base.layers:
    if isinstance(layer, tf.keras.layers.Conv2D):
        layer.kernel_regularizer = tf.keras.regularizers.l2(3e-2) # DEBUG L2

# Add custom classification layers for CIFAR-100 (100 classes) - adapt model
model = models.Sequential([
    resnet_50_base,
    layers.GlobalAveragePooling2D(), # Better for ResNet than Flatten
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.6), # DEBUG dropout
    layers.Dense(100, activation='softmax') # CIFAR-100 has 100 classes
])

for sample in test_dataset.take(1):

```

```

print(type(sample)) # Should be <class 'tuple'>
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'tensor'
print(sample[0].shape) # Should be (batch_size, 224, 224, 3)
print(sample[1].shape) # Should be (batch_size, 100)
print(f"Model input shape: {model.input_shape}")
print(f"Model output shape: {model.output_shape}")
sample = next(iter(test_dataset.as_numpy_iterator()))
print(len(sample)) # Should be 2
print(type(sample[0]), type(sample[1])) # Both should be <class 'numpy.ndarray'
print(sample[0].shape, sample[1].shape) # Should match model input and output
print("\n")
# for x, y in test_dataset.take(1):
#     print(type(x), type(y)) # Both should be <class 'tensorflow.Tensor'>
# for x_batch, y_batch in test_dataset.take(1):
#     test_loss, test_acc = model.evaluate(x_batch, y_batch)
#     print(f"Test Loss: {test_loss}, Test Accuracy: {test_acc}")

# Compile the model
# tensorboard_callback = keras.callbacks.TensorBoard(log_dir="./Logs")
model.compile(optimizer=optimizers.Adam(learning_rate=1e-3, weight_decay=3e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

model.summary()

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>
early_stopping = EarlyStopping(monitor='val_loss', # or val_accuracy
                               patience=5, # Num. epochs with no improvement
                               restore_best_weights=True)

# reduce_lr = ReduceLROnPlateau(monitor='val_loss', # or val_accuracy
#                               factor=0.1, # Reduce lr by a factor
#                               patience=3, # Num epochs w/ no improvement
#                               min_lr=1e-6, # Min lr
#                               verbose=1)

# tensorboard = TensorBoard(log_dir='./logs', # Logs directory
#                             histogram_freq=1, # Logs histograms for weights/acc
#                             write_graph=True, # Logs graph of model
#                             write_images=True) # Log images like weight histograms
checkpoint = ModelCheckpoint('best_model_v3.keras', # Path where the best model is saved
                             monitor='val_loss', # What to monitor (validation loss)
                             save_best_only=True, # Only save model when val loss is best
                             mode='min', # Looking to minimize the validation loss
                             verbose=1) # Print out message when saving model
checkpoint_fine_tune = ModelCheckpoint('best_model_fine_tune_v3.keras', # Path where the best model is saved
                                       monitor='val_loss', # What to monitor (validation loss)
                                       save_best_only=True, # Only save model when val loss is best
                                       mode='min', # Looking to minimize the validation loss
                                       verbose=1) # Print out message when saving model
# csv_logger = CSVLogger('training_log.csv', separator=',', append=True) # Save training logs

# Train the model
history = model.fit(train_dataset, validation_data=val_dataset, epochs=25,
                   batch_size=batch_size, callbacks=[early_stopping, checkpoint])

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history.history['loss'])+1)
train_loss = history.history['loss']

```

```

val_loss = history.history['val_loss']
train_acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>

# Evaluate on test data
model = load_model('best_model_v3.keras')
results = model.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\n

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab
#tensorboard --logdir==path_to_your_logs

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history.history)
hist['epoch'] = history.epoch

# Finally, display the hist DataFrame.
hist

#### <<<<<<<<Fine-Tune>>>>>>>>
# Adapt Model
for layer in resnet_50_base.layers:
    layer.trainable = True # Allow layers to be updated

# Compile again w/ Lower Learning rate (prevents destroying Learned features
model.compile(optimizer=optimizers.Adam(learning_rate=1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', 'precision', 'f1_score'])

```

```

model.summary()

#### <<<<<<<<Modify Dataset>>>>>>>>>>>>

# DEBUG augment
train_dataset_aug_ = (tf.data.Dataset.from_tensor_slices((X_train, y_train))
                      .map(augment_dataset, num_parallel_calls=tf.data.experimental
                           .batch(batch_size)
                           .prefetch(tf.data.experimental.AUTOTUNE)))

# Combine the original dataset and the augmented dataset
train_dataset_aug = train_dataset.concatenate(train_dataset_aug_) # Not val

#### <<<<<<<<Train Model & Track Training/Validation Error>>>>>>>>>>>>

# Train the model
history_fine_tune = model.fit(train_dataset_aug, validation_data=val_dataset
                              batch_size=batch_size, callbacks=[early_stoppi

#### <<<<<<<<Plot Training & Validation Error>>>>>>>>>>>>

# Extract loss and accuracy
epochs = range(1, len(history_fine_tune.history['loss'])+1)
train_loss = history_fine_tune.history['loss']
val_loss = history_fine_tune.history['val_loss']
train_acc = history_fine_tune.history['accuracy']
val_acc = history_fine_tune.history['val_accuracy']

plot_evidence(epochs, train_loss, val_loss, train_acc, val_acc)

#### <<<<<<<<Evaluate Model on Test Data>>>>>>>>>>>>

# Evaluate on test data
model_fine_tune = load_model('best_model_fine_tune_v3.keras')
results = model_fine_tune.evaluate(test_dataset)
test_loss = results[0]
test_acc = results[1]
test_precision = results[2]
test_f1_scores = results[3]
print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Test Loss: {test_loss}")
print(f"Test Precision: {test_precision*100:.2f}%")
print(f"Test F1 Scores (Per Class): {test_f1_scores.numpy()*100}")
print(f"Average Test F1 Scores:{np.average(test_f1_scores.numpy()*100):.2f}\

#### <<<<<<<<Generate Confusion Matrix>>>>>>>>>>>>

# Get predictions
X_test_revised = tf.image.resize(X_test, (224, 224))
y_pred = model_fine_tune.predict(X_test_revised)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = y_test.flatten()

# Compute confusion matrix
conf_matrix = confusion_matrix(y_true_classes, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(20, 20))

```

```
sns.heatmap(conf_matrix) #cmap='Blues', fmt='d'
plt.xlabel('Predicted Labels')
plt.ylabel('True Labels')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.show()

print(classification_report(y_true_classes, y_pred_classes, target_names=lab

# Create a DataFrame from the history of the training and store the epoch va
hist = pd.DataFrame(history_fine_tune.history)
hist['epoch'] = history_fine_tune.epoch

# Finally, display the hist DataFrame.
hist
```

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

X_val.shape: (9000, 32, 32, 3)

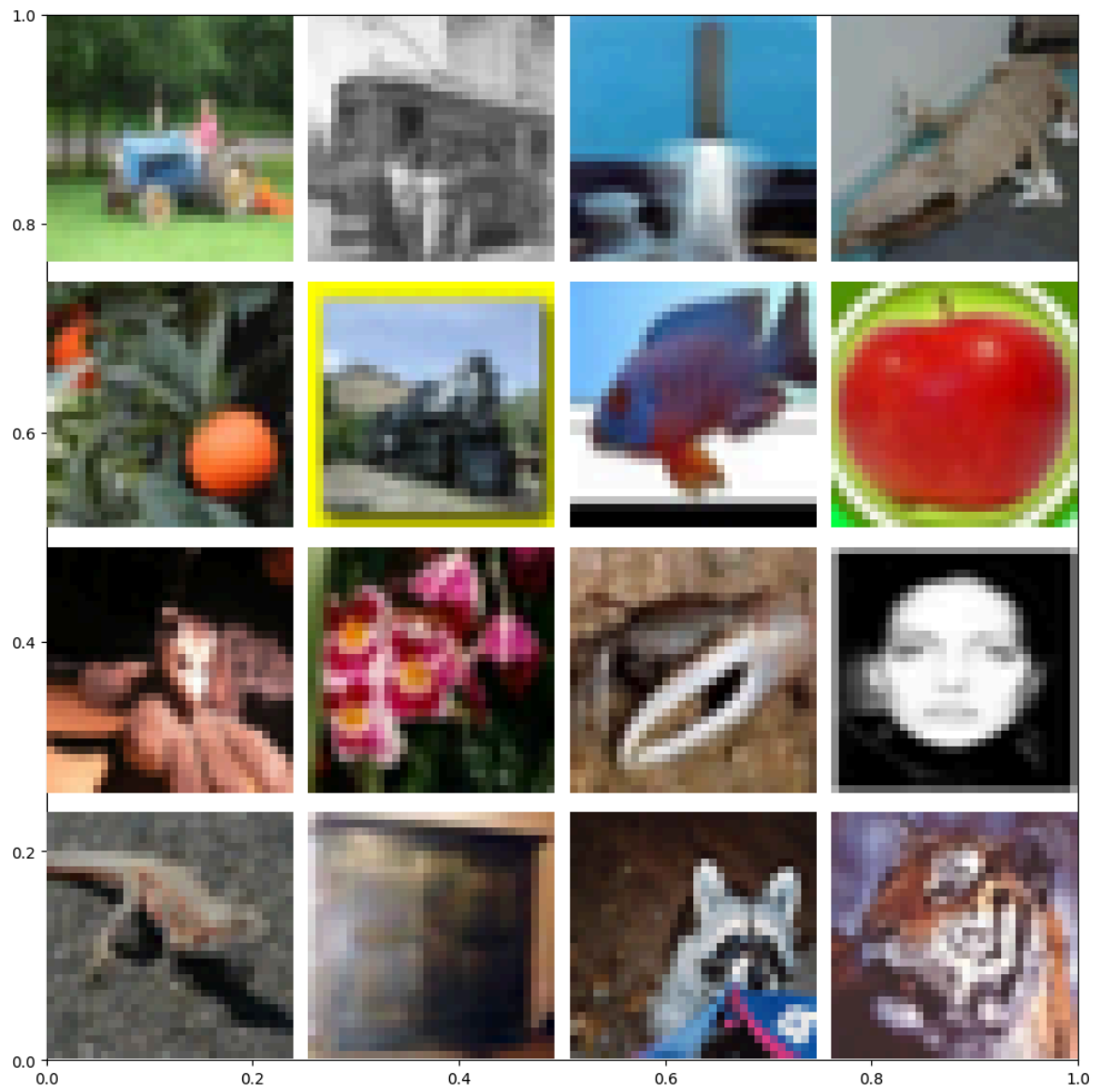
y_val.shape: (9000, 1)

X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)

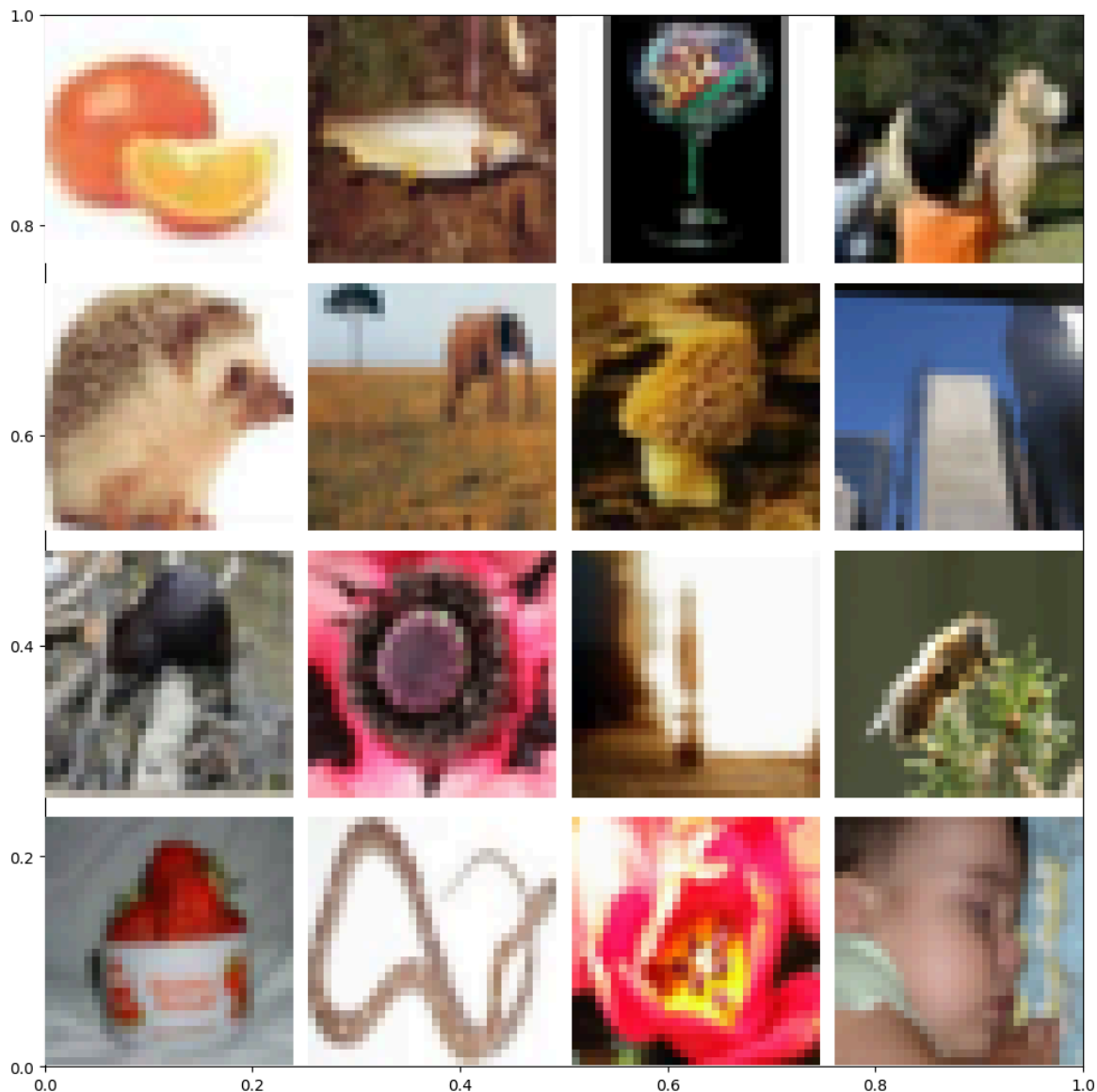
labels[12181].shape: (1,)

imgs[12181].shape: (32, 32, 3)



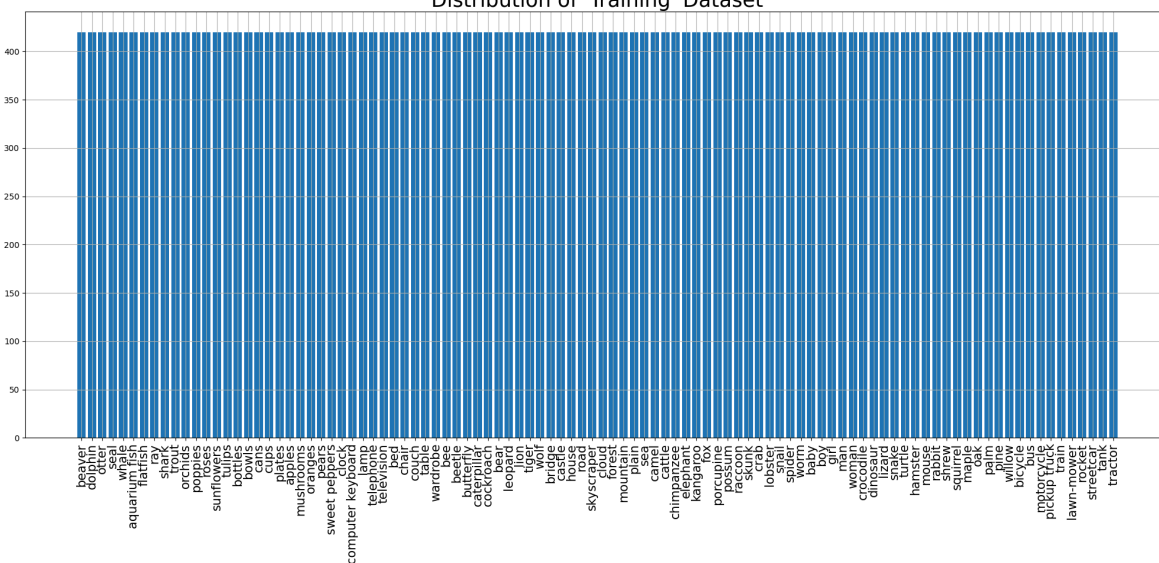
labels[14121].shape: (1,)

imgs[14121].shape: (32, 32, 3)

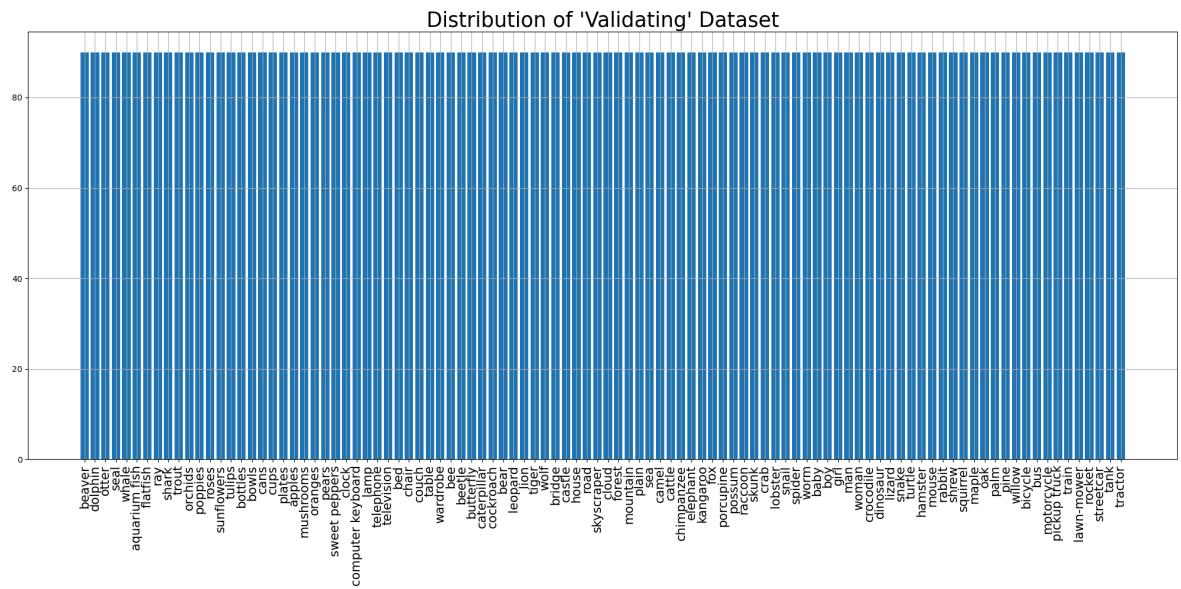


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

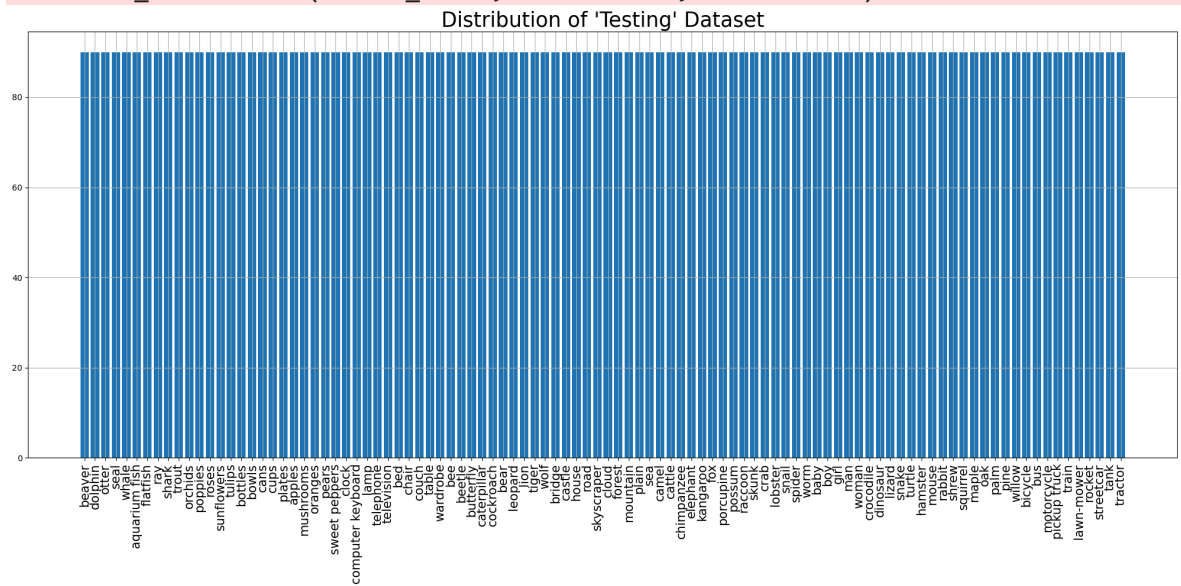
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

```
<class 'tuple'>
```

2

```
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
```

(64, 224, 224, 3)

(64, 100)

Model input shape: (None, 224, 224, 3)

Model output shape: (None, 100)

2

```
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
```

(64, 224, 224, 3) (64, 100)

Model: "sequential_6"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_6 (GlobalAveragePooling2D)	(None , 2048)
dense_12 (Dense)	(None , 512)
dropout_6 (Dropout)	(None , 512)
dense_13 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
1314/1314 ————— 0s 27ms/step - accuracy: 0.3805 - f1_score: 0.3768
- loss: 2.5198 - precision: 0.6877
Epoch 1: val_loss improved from inf to 1.39783, saving model to best_model_v3.keras
1314/1314 ————— 57s 35ms/step - accuracy: 0.3806 - f1_score: 0.3769
9 - loss: 2.5194 - precision: 0.6878 - val_accuracy: 0.6050 - val_f1_score: 0.6005
5 - val_loss: 1.3978 - val_precision: 0.8141
Epoch 2/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.5653 - f1_score: 0.5622
- loss: 1.5643 - precision: 0.7582
Epoch 2: val_loss improved from 1.39783 to 1.31449, saving model to best_model_v3.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.5653 - f1_score: 0.5622
2 - loss: 1.5643 - precision: 0.7582 - val_accuracy: 0.6290 - val_f1_score: 0.6256
6 - val_loss: 1.3145 - val_precision: 0.8129
Epoch 3/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.6051 - f1_score: 0.6030
- loss: 1.3909 - precision: 0.7773
Epoch 3: val_loss improved from 1.31449 to 1.30277, saving model to best_model_v3.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.6051 - f1_score: 0.6030
0 - loss: 1.3909 - precision: 0.7773 - val_accuracy: 0.6337 - val_f1_score: 0.6314
4 - val_loss: 1.3028 - val_precision: 0.8022
Epoch 4/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.6269 - f1_score: 0.6246
- loss: 1.2810 - precision: 0.7859
Epoch 4: val_loss improved from 1.30277 to 1.29109, saving model to best_model_v3.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.6269 - f1_score: 0.6246
6 - loss: 1.2810 - precision: 0.7859 - val_accuracy: 0.6387 - val_f1_score: 0.6369
9 - val_loss: 1.2911 - val_precision: 0.7950
Epoch 5/25
1313/1314 ————— 0s 24ms/step - accuracy: 0.6476 - f1_score: 0.6454
- loss: 1.2017 - precision: 0.7978
Epoch 5: val_loss improved from 1.29109 to 1.28845, saving model to best_model_v3.keras
1314/1314 ————— 36s 27ms/step - accuracy: 0.6476 - f1_score: 0.6454
4 - loss: 1.2017 - precision: 0.7978 - val_accuracy: 0.6381 - val_f1_score: 0.6365
5 - val_loss: 1.2884 - val_precision: 0.7902
Epoch 6/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.6650 - f1_score: 0.6632
- loss: 1.1314 - precision: 0.8062
Epoch 6: val_loss did not improve from 1.28845
1314/1314 ————— 35s 26ms/step - accuracy: 0.6650 - f1_score: 0.6632
2 - loss: 1.1314 - precision: 0.8062 - val_accuracy: 0.6424 - val_f1_score: 0.6408
8 - val_loss: 1.2899 - val_precision: 0.7889
Epoch 7/25
1314/1314 ————— 0s 24ms/step - accuracy: 0.6803 - f1_score: 0.6781
- loss: 1.0728 - precision: 0.8092
Epoch 7: val_loss did not improve from 1.28845
1314/1314 ————— 35s 26ms/step - accuracy: 0.6803 - f1_score: 0.6781
1 - loss: 1.0728 - precision: 0.8092 - val_accuracy: 0.6450 - val_f1_score: 0.6447
7 - val_loss: 1.2947 - val_precision: 0.7867
Epoch 8/25
1312/1314 ————— 0s 24ms/step - accuracy: 0.6908 - f1_score: 0.6892
- loss: 1.0289 - precision: 0.8147
Epoch 8: val_loss did not improve from 1.28845
1314/1314 ————— 35s 26ms/step - accuracy: 0.6908 - f1_score: 0.6892
2 - loss: 1.0290 - precision: 0.8147 - val_accuracy: 0.6491 - val_f1_score: 0.648

5 - val_loss: 1.3025 - val_precision: 0.7856

Epoch 9/25

1312/1314 ————— 0s 24ms/step - accuracy: 0.7030 - f1_score: 0.7013
- loss: 0.9845 - precision: 0.8230

Epoch 9: val_loss did not improve from 1.28845

1314/1314 ————— 35s 26ms/step - accuracy: 0.7030 - f1_score: 0.7013
3 - loss: 0.9846 - precision: 0.8229 - val_accuracy: 0.6503 - val_f1_score: 0.6492
2 - val_loss: 1.3201 - val_precision: 0.7843

Epoch 10/25

1312/1314 ————— 0s 24ms/step - accuracy: 0.7116 - f1_score: 0.7101
- loss: 0.9450 - precision: 0.8243

Epoch 10: val_loss did not improve from 1.28845

1314/1314 ————— 34s 26ms/step - accuracy: 0.7116 - f1_score: 0.7101
1 - loss: 0.9451 - precision: 0.8243 - val_accuracy: 0.6498 - val_f1_score: 0.6496
6 - val_loss: 1.3075 - val_precision: 0.7844



141/141 ————— 11s 38ms/step - accuracy: 0.6348 - f1_score: 0.6276
- loss: 1.3111 - precision: 0.7859

Test Accuracy: 64.30%

Test Loss: 1.2860703468322754

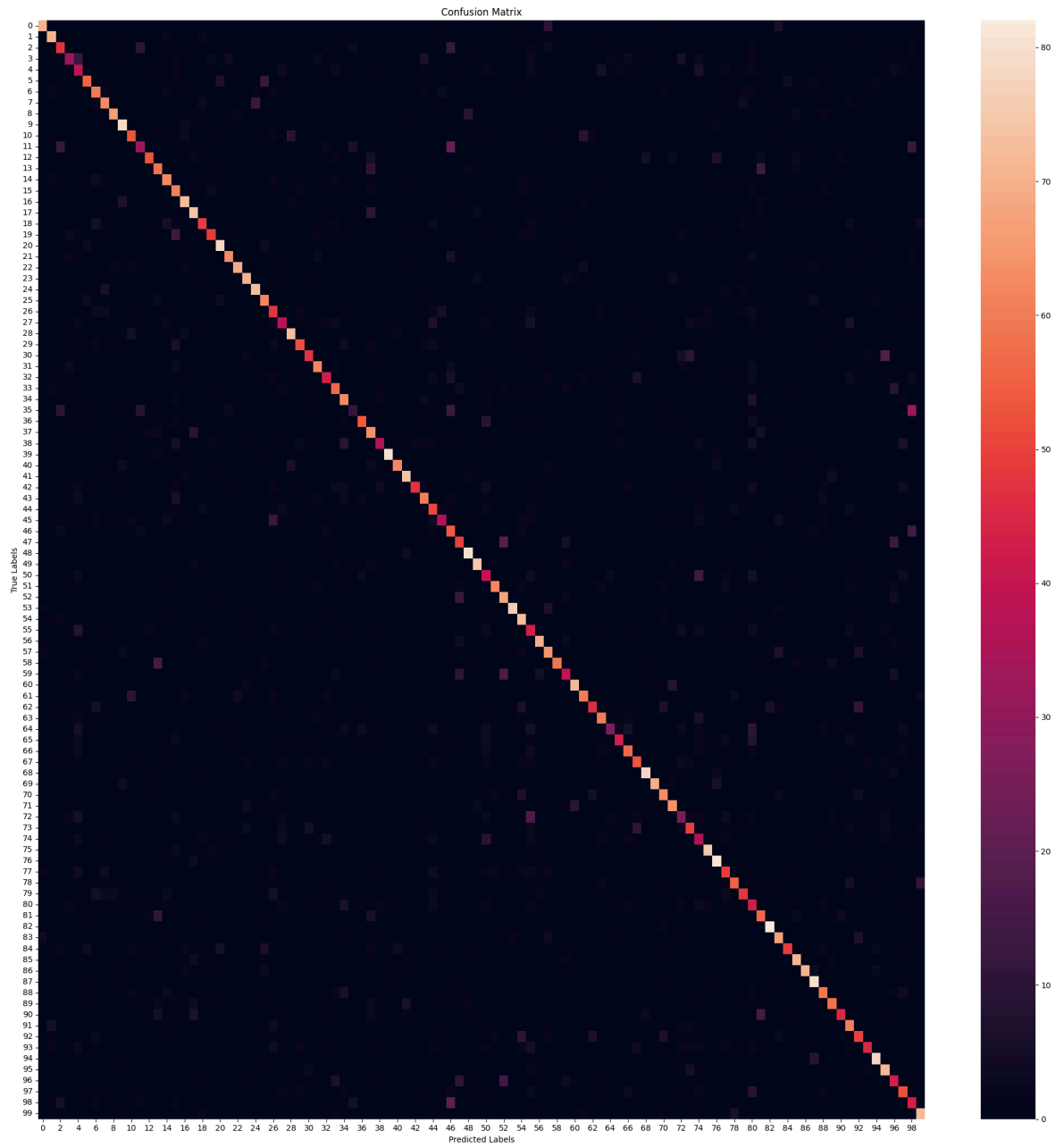
Test Precision: 79.51%

Test F1 Scores (Per Class): [82.558136 76.34408 53.40909 41.610733 37.499992 67.878784 62.176155

69.99999 76.300575 78.60696 59.886997 42.3841 67.08861 58.883244
65.957436 54.222214 77.24867 71.497574 59.756096 59.756096 81.24999
72.94117 80.701744 78.88888 79.347824 63.917522 47.76119 45.23809
78.49462 64.59627 61.03895 69.27374 52.760727 56.999992 62.999992
18.965513 68.78981 57.3991 50.68492 91.428566 72.94117 80.43478
64.38356 67.77777 50.74626 47.999996 43.724693 52.910053 82.90154
82.872925 40.60913 72.51461 64.15093 87.86127 67.2811 38.05309
77.77777 65.65657 78.14569 50.31446 80. 70.58823 59.740257
67.796616 38.80597 56.953632 65.89595 57.297295 85.71427 82.352936
67.02127 73.14285 30.4878 60.24096 36.923073 81.52173 79.80295
55.367226 68.71165 60.75949 35.146435 58.461536 91.11111 69.791664
62.820507 77.173904 84.02366 78.048775 65.92178 67.83625 60.810806
62.5641 54.34782 59.999996 85.71427 71.641785 46.99453 58.426964
42.51207 77.83783]

Average Test F1 Scores:64.26

282/282 ————— 12s 28ms/step



	precision	recall	f1-score	support
beaver	0.87	0.79	0.83	90
dolphin	0.74	0.79	0.76	90
otter	0.55	0.52	0.53	90
seal	0.53	0.34	0.42	90
whale	0.33	0.43	0.38	90
aquarium fish	0.75	0.62	0.68	90
flatfish	0.58	0.67	0.62	90
ray	0.70	0.70	0.70	90
shark	0.80	0.73	0.76	90
trout	0.71	0.88	0.79	90
orchids	0.61	0.59	0.60	90
poppies	0.52	0.36	0.42	90
roses	0.78	0.59	0.67	90
sunflowers	0.54	0.64	0.59	90
tulips	0.63	0.69	0.66	90
bottles	0.45	0.68	0.54	90
bowls	0.74	0.81	0.77	90
cans	0.63	0.82	0.71	90
cups	0.66	0.54	0.60	90
plates	0.66	0.54	0.60	90
apples	0.76	0.87	0.81	90
mushrooms	0.78	0.70	0.74	90
oranges	0.85	0.77	0.81	90
pears	0.79	0.79	0.79	90
sweet peppers	0.78	0.81	0.79	90
clock	0.60	0.69	0.64	90
computer keyboard	0.43	0.53	0.48	90
lamp	0.49	0.42	0.45	90
telephone	0.76	0.81	0.78	90
television	0.73	0.58	0.65	90
bed	0.73	0.52	0.61	90
chair	0.70	0.69	0.69	90
couch	0.59	0.48	0.53	90
table	0.52	0.63	0.57	90
wardrobe	0.57	0.70	0.63	90
bee	0.42	0.12	0.19	90
beetle	0.81	0.60	0.69	90
butterfly	0.48	0.71	0.57	90
caterpillar	0.66	0.41	0.51	90
cockroach	0.94	0.89	0.91	90
bear	0.78	0.69	0.73	90
leopard	0.79	0.82	0.80	90
lion	0.84	0.52	0.64	90
tiger	0.68	0.68	0.68	90
wolf	0.46	0.57	0.51	90
bridge	0.60	0.40	0.48	90
castle	0.34	0.60	0.44	90
house	0.51	0.56	0.53	90
road	0.78	0.89	0.83	90
skyscraper	0.82	0.83	0.83	90
cloud	0.37	0.44	0.41	90
forest	0.77	0.69	0.73	90
mountain	0.56	0.76	0.64	90
plain	0.92	0.84	0.88	90
sea	0.57	0.81	0.67	90
camel	0.32	0.48	0.38	90
cattle	0.78	0.78	0.78	90
chimpanzee	0.60	0.72	0.66	90

elephant	0.97	0.66	0.78	90
kangaroo	0.58	0.44	0.50	90
fox	0.80	0.80	0.80	90
porcupine	0.75	0.67	0.71	90
possum	0.72	0.51	0.60	90
raccoon	0.69	0.67	0.68	90
skunk	0.59	0.29	0.39	90
crab	0.70	0.48	0.57	90
lobster	0.69	0.63	0.66	90
snail	0.56	0.59	0.57	90
spider	0.85	0.87	0.86	90
worm	0.88	0.78	0.82	90
baby	0.64	0.70	0.67	90
boy	0.75	0.71	0.73	90
girl	0.34	0.28	0.31	90
man	0.66	0.56	0.60	90
woman	0.34	0.40	0.37	90
crocodile	0.80	0.83	0.82	90
dinosaur	0.72	0.90	0.80	90
lizard	0.56	0.54	0.55	90
snake	0.77	0.62	0.69	90
turtle	0.71	0.53	0.61	90
hamster	0.28	0.47	0.35	90
mouse	0.54	0.63	0.58	90
rabbit	0.91	0.91	0.91	90
shrew	0.66	0.74	0.70	90
squirrel	0.73	0.54	0.62	90
maple	0.76	0.79	0.77	90
oak	0.90	0.79	0.84	90
palm	0.70	0.89	0.78	90
pine	0.66	0.66	0.66	90
willow	0.72	0.64	0.68	90
bicycle	0.78	0.50	0.61	90
bus	0.58	0.68	0.62	90
motorcycle	0.53	0.56	0.54	90
pickup truck	0.75	0.50	0.60	90
train	0.85	0.87	0.86	90
lawn-mower	0.65	0.80	0.72	90
rocket	0.46	0.48	0.47	90
streetcar	0.59	0.58	0.58	90
tank	0.37	0.48	0.42	90
tractor	0.76	0.80	0.78	90
accuracy			0.64	9000
macro avg	0.66	0.64	0.64	9000
weighted avg	0.66	0.64	0.64	9000

Model: "sequential_6"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_6 (GlobalAveragePooling2D)	(None , 2048)
dense_12 (Dense)	(None , 512)
dropout_6 (Dropout)	(None , 512)
dense_13 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15

1970/1971 ————— **0s** 26ms/step - accuracy: 0.6739 - f1_score: 0.6720
- loss: 1.0964 - precision: 0.8154

Epoch 1: val_loss improved from inf to 1.23921, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **74s** 31ms/step - accuracy: 0.6739 - f1_score: 0.6720 - loss: 1.0964 - precision: 0.8154 - val_accuracy: 0.6564 - val_f1_score: 0.6555 - val_loss: 1.2392 - val_precision: 0.7977

Epoch 2/15

1970/1971 ————— **0s** 24ms/step - accuracy: 0.7031 - f1_score: 0.7011
- loss: 0.9966 - precision: 0.8331

Epoch 2: val_loss improved from 1.23921 to 1.23250, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **52s** 27ms/step - accuracy: 0.7031 - f1_score: 0.7011 - loss: 0.9966 - precision: 0.8331 - val_accuracy: 0.6601 - val_f1_score: 0.6593 - val_loss: 1.2325 - val_precision: 0.7946

Epoch 3/15

1969/1971 ————— **0s** 24ms/step - accuracy: 0.7097 - f1_score: 0.7076
- loss: 0.9694 - precision: 0.8362

Epoch 3: val_loss improved from 1.23250 to 1.22821, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **52s** 27ms/step - accuracy: 0.7097 - f1_score: 0.7076 - loss: 0.9694 - precision: 0.8362 - val_accuracy: 0.6611 - val_f1_score: 0.6603 - val_loss: 1.2282 - val_precision: 0.7965

Epoch 4/15

1969/1971 ————— **0s** 24ms/step - accuracy: 0.7155 - f1_score: 0.7140
- loss: 0.9435 - precision: 0.8423

Epoch 4: val_loss improved from 1.22821 to 1.22533, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **52s** 26ms/step - accuracy: 0.7155 - f1_score: 0.7140 - loss: 0.9436 - precision: 0.8423 - val_accuracy: 0.6634 - val_f1_score: 0.6628 - val_loss: 1.2253 - val_precision: 0.7958

Epoch 5/15

1970/1971 ————— **0s** 24ms/step - accuracy: 0.7183 - f1_score: 0.7168
- loss: 0.9320 - precision: 0.8452

Epoch 5: val_loss improved from 1.22533 to 1.22237, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **52s** 27ms/step - accuracy: 0.7183 - f1_score: 0.7168 - loss: 0.9321 - precision: 0.8452 - val_accuracy: 0.6639 - val_f1_score: 0.6633 - val_loss: 1.2224 - val_precision: 0.7958

Epoch 6/15

1969/1971 ————— **0s** 24ms/step - accuracy: 0.7215 - f1_score: 0.7197
- loss: 0.9226 - precision: 0.8468

Epoch 6: val_loss improved from 1.22237 to 1.21909, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **52s** 27ms/step - accuracy: 0.7215 - f1_score: 0.7197 - loss: 0.9226 - precision: 0.8468 - val_accuracy: 0.6662 - val_f1_score: 0.6655 - val_loss: 1.2191 - val_precision: 0.7969

Epoch 7/15

1970/1971 ————— **0s** 24ms/step - accuracy: 0.7243 - f1_score: 0.7224
- loss: 0.9097 - precision: 0.8489

Epoch 7: val_loss improved from 1.21909 to 1.21715, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— **52s** 27ms/step - accuracy: 0.7243 - f1_score: 0.7224 - loss: 0.9097 - precision: 0.8488 - val_accuracy: 0.6664 - val_f1_score: 0.6658 - val_loss: 1.2171 - val_precision: 0.7970

Epoch 8/15

1969/1971 ————— **0s** 25ms/step - accuracy: 0.7301 - f1_score: 0.7286
- loss: 0.8936 - precision: 0.8531

Epoch 8: val_loss improved from 1.21715 to 1.21569, saving model to best_model_fi

ne_tune_v3.keras

1971/1971 ————— 52s 27ms/step - accuracy: 0.7300 - f1_score: 0.7285 - loss: 0.8936 - precision: 0.8531 - val_accuracy: 0.6664 - val_f1_score: 0.6659 - val_loss: 1.2157 - val_precision: 0.7970

Epoch 9/15

1971/1971 ————— 0s 24ms/step - accuracy: 0.7310 - f1_score: 0.7293 - loss: 0.8864 - precision: 0.8537

Epoch 9: val_loss improved from 1.21569 to 1.21425, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 52s 27ms/step - accuracy: 0.7310 - f1_score: 0.7293 - loss: 0.8864 - precision: 0.8537 - val_accuracy: 0.6668 - val_f1_score: 0.6664 - val_loss: 1.2142 - val_precision: 0.7946

Epoch 10/15

1970/1971 ————— 0s 25ms/step - accuracy: 0.7346 - f1_score: 0.7330 - loss: 0.8817 - precision: 0.8573

Epoch 10: val_loss improved from 1.21425 to 1.21206, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 53s 27ms/step - accuracy: 0.7346 - f1_score: 0.7330 - loss: 0.8817 - precision: 0.8573 - val_accuracy: 0.6668 - val_f1_score: 0.6663 - val_loss: 1.2121 - val_precision: 0.7957

Epoch 11/15

1970/1971 ————— 0s 25ms/step - accuracy: 0.7397 - f1_score: 0.7380 - loss: 0.8597 - precision: 0.8590

Epoch 11: val_loss improved from 1.21206 to 1.21046, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 53s 27ms/step - accuracy: 0.7397 - f1_score: 0.7380 - loss: 0.8597 - precision: 0.8590 - val_accuracy: 0.6687 - val_f1_score: 0.6681 - val_loss: 1.2105 - val_precision: 0.7963

Epoch 12/15

1969/1971 ————— 0s 25ms/step - accuracy: 0.7400 - f1_score: 0.7386 - loss: 0.8574 - precision: 0.8587

Epoch 12: val_loss improved from 1.21046 to 1.20947, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 53s 27ms/step - accuracy: 0.7399 - f1_score: 0.7386 - loss: 0.8574 - precision: 0.8587 - val_accuracy: 0.6689 - val_f1_score: 0.6685 - val_loss: 1.2095 - val_precision: 0.7951

Epoch 13/15

1970/1971 ————— 0s 25ms/step - accuracy: 0.7427 - f1_score: 0.7411 - loss: 0.8519 - precision: 0.8616

Epoch 13: val_loss improved from 1.20947 to 1.20788, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 53s 27ms/step - accuracy: 0.7427 - f1_score: 0.7411 - loss: 0.8519 - precision: 0.8616 - val_accuracy: 0.6686 - val_f1_score: 0.6679 - val_loss: 1.2079 - val_precision: 0.7972

Epoch 14/15

1970/1971 ————— 0s 25ms/step - accuracy: 0.7444 - f1_score: 0.7427 - loss: 0.8427 - precision: 0.8640

Epoch 14: val_loss improved from 1.20788 to 1.20748, saving model to best_model_fine_tune_v3.keras

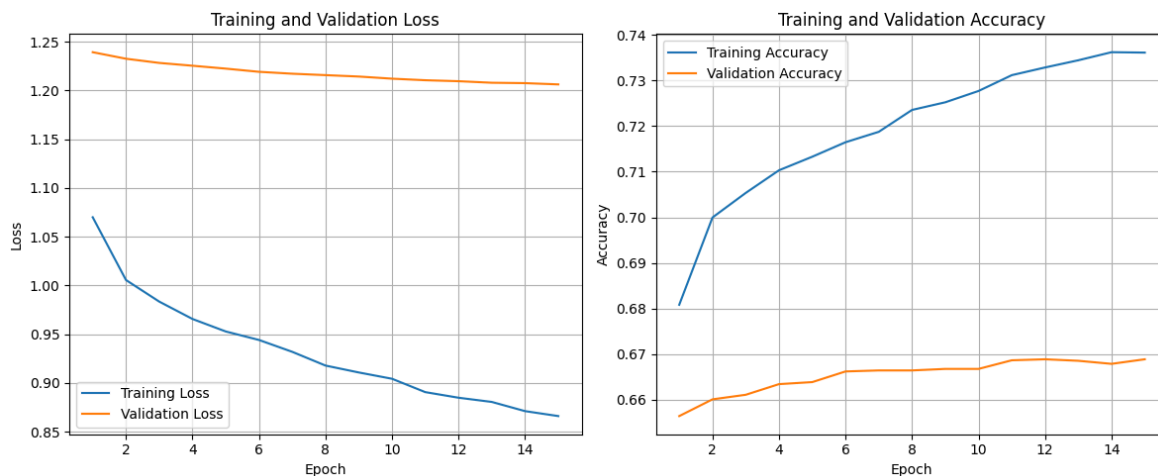
1971/1971 ————— 53s 27ms/step - accuracy: 0.7444 - f1_score: 0.7427 - loss: 0.8427 - precision: 0.8640 - val_accuracy: 0.6679 - val_f1_score: 0.6675 - val_loss: 1.2075 - val_precision: 0.7973

Epoch 15/15

1969/1971 ————— 0s 24ms/step - accuracy: 0.7433 - f1_score: 0.7419 - loss: 0.8376 - precision: 0.8607

Epoch 15: val_loss improved from 1.20748 to 1.20619, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 52s 27ms/step - accuracy: 0.7433 - f1_score: 0.7418 - loss: 0.8377 - precision: 0.8607 - val_accuracy: 0.6689 - val_f1_score: 0.6684 - val_loss: 1.2062 - val_precision: 0.7978



141/141 ————— **11s** 38ms/step - accuracy: 0.6626 - f1_score: 0.6547

- loss: 1.2223 - precision: 0.7941

Test Accuracy: 67.26%

Test Loss: 1.202204942703247

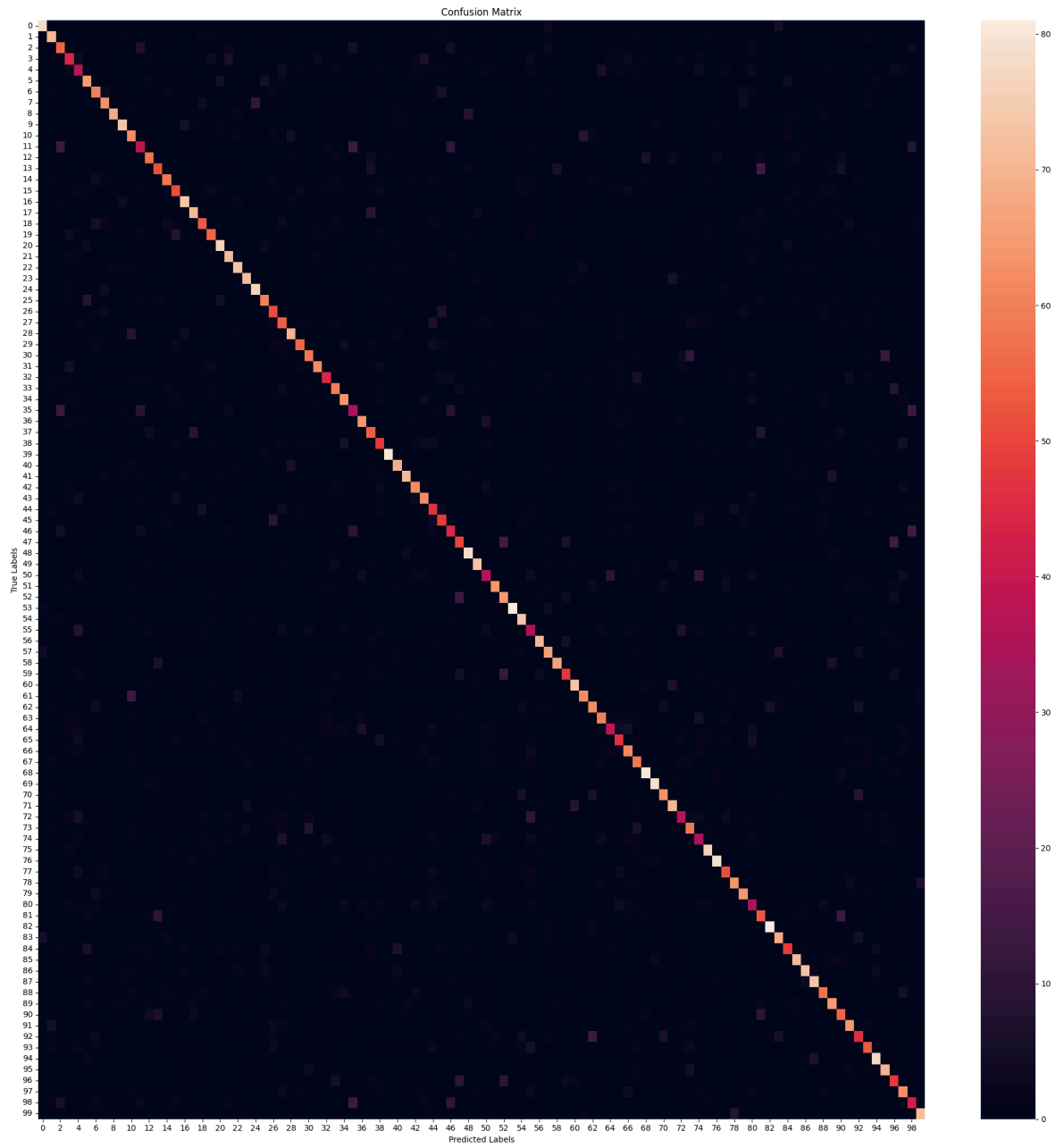
Test Precision: 80.19%

Test F1 Scores (Per Class): [85.71427 76.34408 57.894737 50.28571 39.784943 69.89246 62.827217

70.786514 79.99999 82.02247 63.589745 47.204964 65.14285 60.1156
68.63905 56.98323 78.30688 75.13227 58.88888 63.636356 83.060104
75.93583 80.21977 80.446915 83.060104 68.57143 54.838707 54.54545
78.65168 67.90123 66.666664 72.94117 55.345905 61.13989 69.565216
39.999996 72.31638 58.064507 54.857136 92.48554 73.79678 81.60919
67.02702 69.66292 48.958324 53.84615 48.087425 57.471264 82.10526
82.48587 41.379307 73.56321 67.357506 89.99999 74.111664 38.88888
81.39534 71.35134 79.761894 58.18181 82.02247 73.809525 66.31015
68.181816 48.447197 54.02298 63.541664 62.032085 85.56149 81.44328
71.590904 75.675674 44.047615 63.387978 37.77777 82.608696 88.26815
61.176468 73.56321 71.91011 37.988823 55.208332 89.99999 73.11827
59.036137 78.02197 80.66298 80.66298 63.33333 68.06282 59.139782
69.18918 54.43787 61.6279 84.61538 75.675674 52.459015 66.66666
47.252743 80.446915]

Average Test F1 Scores:67.18

282/282 ————— **12s** 29ms/step



	precision	recall	f1-score	support
beaver	0.85	0.87	0.86	90
dolphin	0.74	0.79	0.76	90
otter	0.55	0.61	0.58	90
seal	0.52	0.49	0.50	90
whale	0.39	0.41	0.40	90
aquarium fish	0.68	0.72	0.70	90
flatfish	0.59	0.67	0.63	90
ray	0.72	0.70	0.71	90
shark	0.85	0.76	0.80	90
trout	0.83	0.81	0.82	90
orchids	0.59	0.69	0.64	90
poppies	0.53	0.42	0.47	90
roses	0.67	0.63	0.65	90
sunflowers	0.63	0.58	0.60	90
tulips	0.73	0.64	0.69	90
bottles	0.57	0.57	0.57	90
bowls	0.75	0.82	0.78	90
cans	0.72	0.79	0.75	90
cups	0.59	0.59	0.59	90
plates	0.65	0.61	0.63	90
apples	0.82	0.84	0.83	90
mushrooms	0.73	0.79	0.76	90
oranges	0.79	0.81	0.80	90
pears	0.81	0.80	0.80	90
sweet peppers	0.82	0.84	0.83	90
clock	0.71	0.67	0.69	90
computer keyboard	0.53	0.57	0.55	90
lamp	0.50	0.60	0.55	90
telephone	0.80	0.78	0.79	90
television	0.76	0.61	0.68	90
bed	0.69	0.64	0.67	90
chair	0.78	0.69	0.73	90
couch	0.64	0.49	0.55	90
table	0.57	0.66	0.61	90
wardrobe	0.68	0.71	0.70	90
bee	0.42	0.38	0.40	90
beetle	0.74	0.71	0.72	90
butterfly	0.56	0.60	0.58	90
caterpillar	0.56	0.53	0.55	90
cockroach	0.96	0.89	0.92	90
bear	0.71	0.77	0.74	90
leopard	0.85	0.79	0.82	90
lion	0.65	0.69	0.67	90
tiger	0.71	0.69	0.70	90
wolf	0.46	0.52	0.49	90
bridge	0.53	0.54	0.54	90
castle	0.48	0.49	0.48	90
house	0.60	0.56	0.57	90
road	0.78	0.87	0.82	90
skyscraper	0.84	0.81	0.82	90
cloud	0.43	0.40	0.41	90
forest	0.76	0.71	0.74	90
mountain	0.63	0.72	0.67	90
plain	0.90	0.90	0.90	90
sea	0.68	0.81	0.74	90
camel	0.39	0.39	0.39	90
cattle	0.86	0.79	0.82	90
chimpanzee	0.69	0.73	0.71	90

elephant	0.86	0.74	0.80	90
kangaroo	0.64	0.53	0.58	90
fox	0.83	0.81	0.82	90
porcupine	0.79	0.69	0.74	90
possum	0.64	0.69	0.66	90
raccoon	0.70	0.67	0.68	90
skunk	0.55	0.43	0.48	90
crab	0.56	0.52	0.54	90
lobster	0.59	0.68	0.63	90
snail	0.60	0.64	0.62	90
spider	0.82	0.89	0.86	90
worm	0.76	0.88	0.81	90
baby	0.73	0.70	0.72	90
boy	0.74	0.78	0.76	90
girl	0.47	0.41	0.44	90
man	0.62	0.64	0.63	90
woman	0.38	0.38	0.38	90
crocodile	0.81	0.84	0.83	90
dinosaur	0.89	0.88	0.88	90
lizard	0.65	0.58	0.61	90
snake	0.76	0.71	0.74	90
turtle	0.73	0.71	0.72	90
hamster	0.38	0.38	0.38	90
mouse	0.52	0.59	0.55	90
rabbit	0.90	0.90	0.90	90
shrew	0.71	0.76	0.73	90
squirrel	0.64	0.54	0.59	90
maple	0.77	0.79	0.78	90
oak	0.80	0.81	0.81	90
palm	0.80	0.81	0.81	90
pine	0.63	0.63	0.63	90
willow	0.64	0.72	0.68	90
bicycle	0.57	0.61	0.59	90
bus	0.67	0.71	0.69	90
motorcycle	0.58	0.51	0.54	90
pickup truck	0.65	0.59	0.62	90
train	0.84	0.86	0.85	90
lawn-mower	0.74	0.78	0.76	90
rocket	0.52	0.53	0.52	90
streetcar	0.65	0.69	0.67	90
tank	0.47	0.48	0.47	90
tractor	0.81	0.80	0.80	90
accuracy			0.67	9000
macro avg	0.67	0.67	0.67	9000
weighted avg	0.67	0.67	0.67	9000

X_temp.shape: (8000, 32, 32, 3)

X_temp_val.shape: (4000, 32, 32, 3)

y_temp_val.shape: (4000, 1)

X_temp_test.shape: (4000, 32, 32, 3)

y_temp_test.shape: (4000, 1)

X_train.shape: (42000, 32, 32, 3)

y_train.shape: (42000, 1)

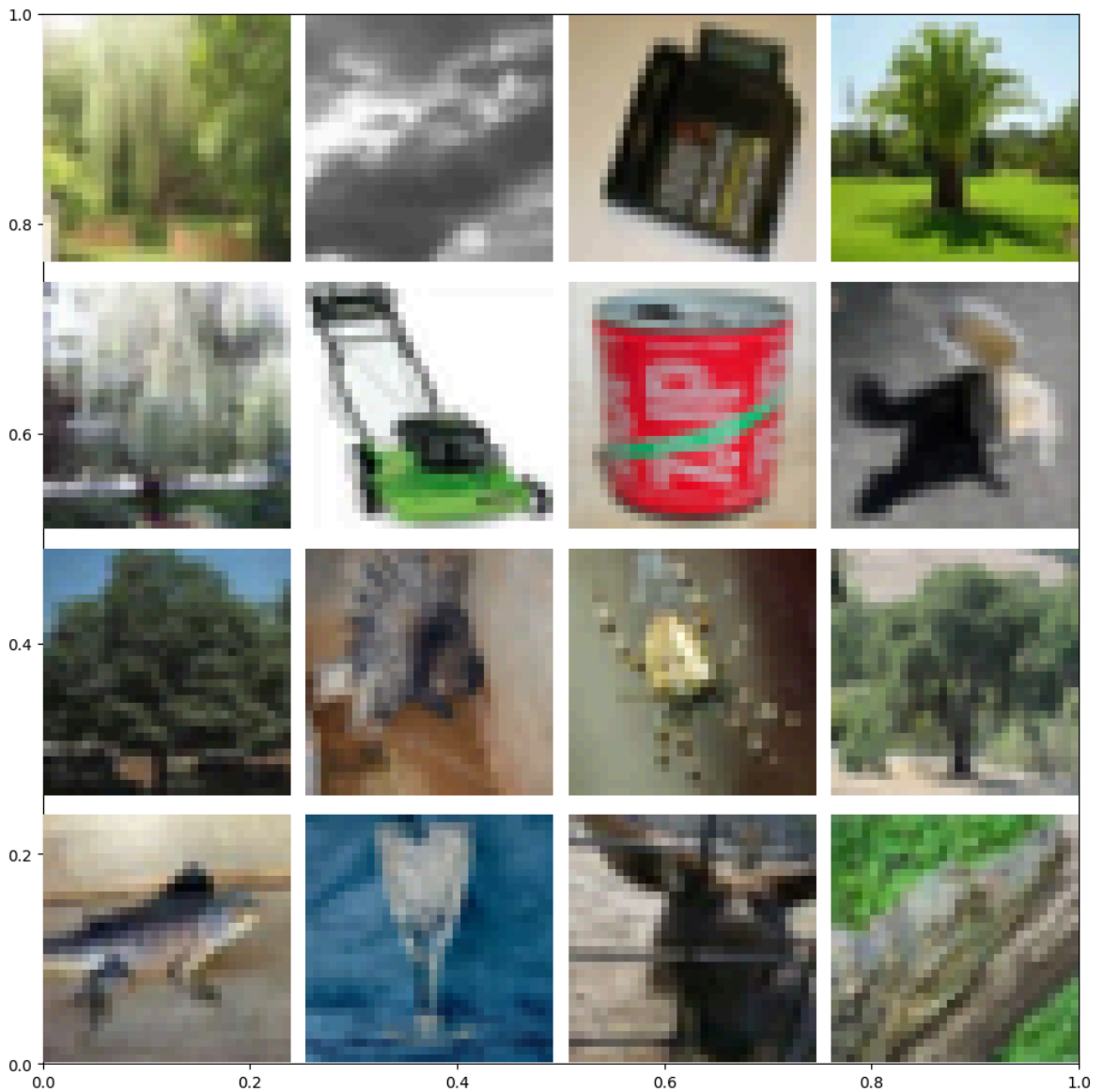
X_val.shape: (9000, 32, 32, 3)

y_val.shape: (9000, 1)

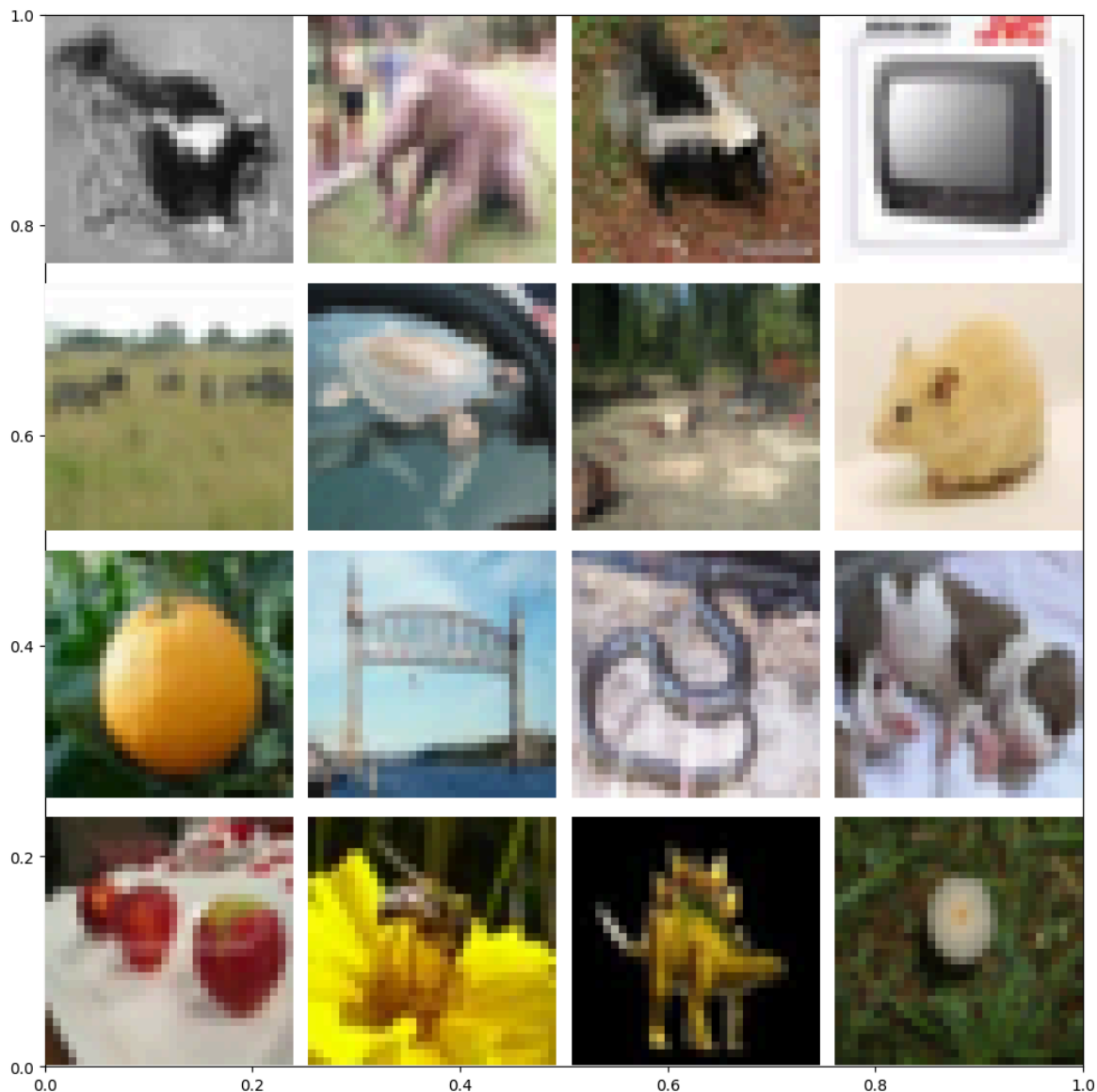
X_test.shape: (9000, 32, 32, 3)

y_test.shape: (9000, 1)


```
labels[3142].shape: (1,)  
imgs[3142].shape: (32, 32, 3)
```

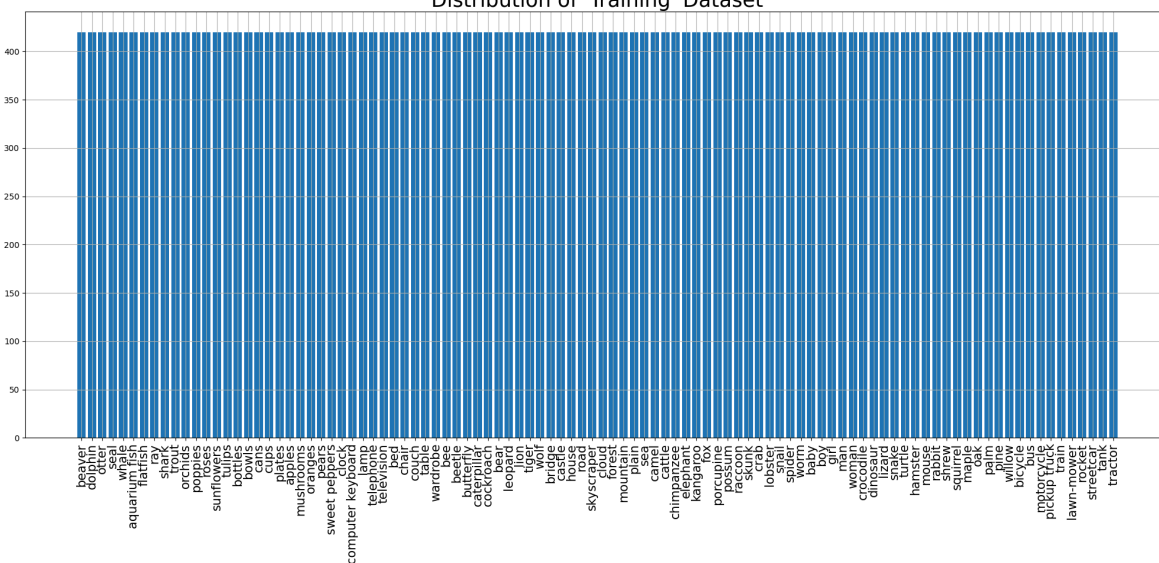


```
labels[3318].shape: (1,)  
imgs[3318].shape: (32, 32, 3)
```

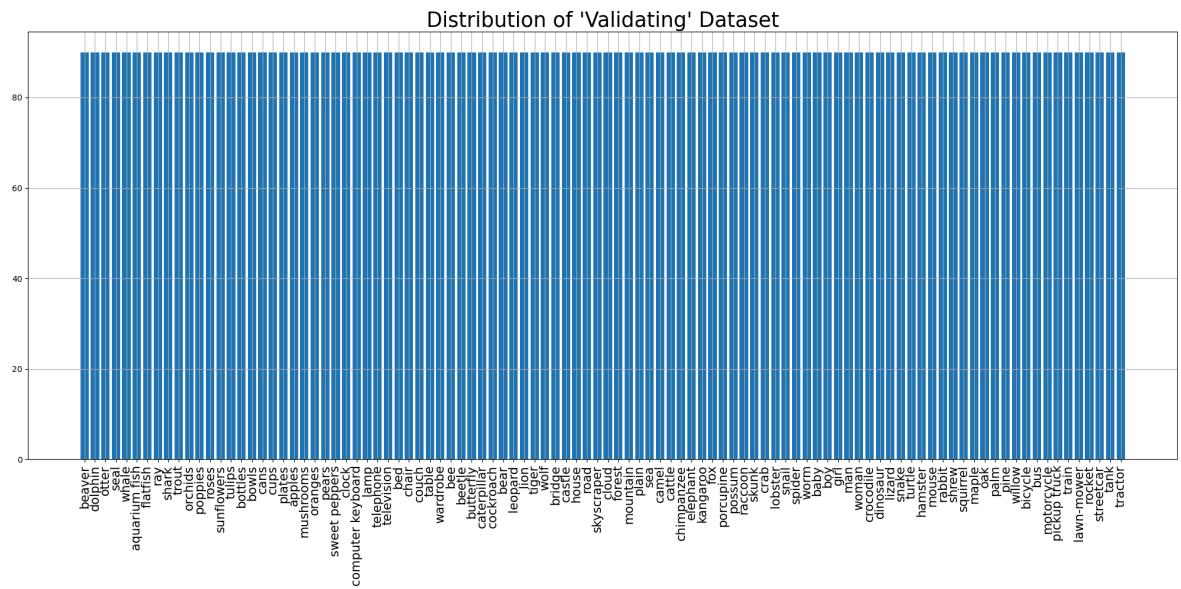


```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```

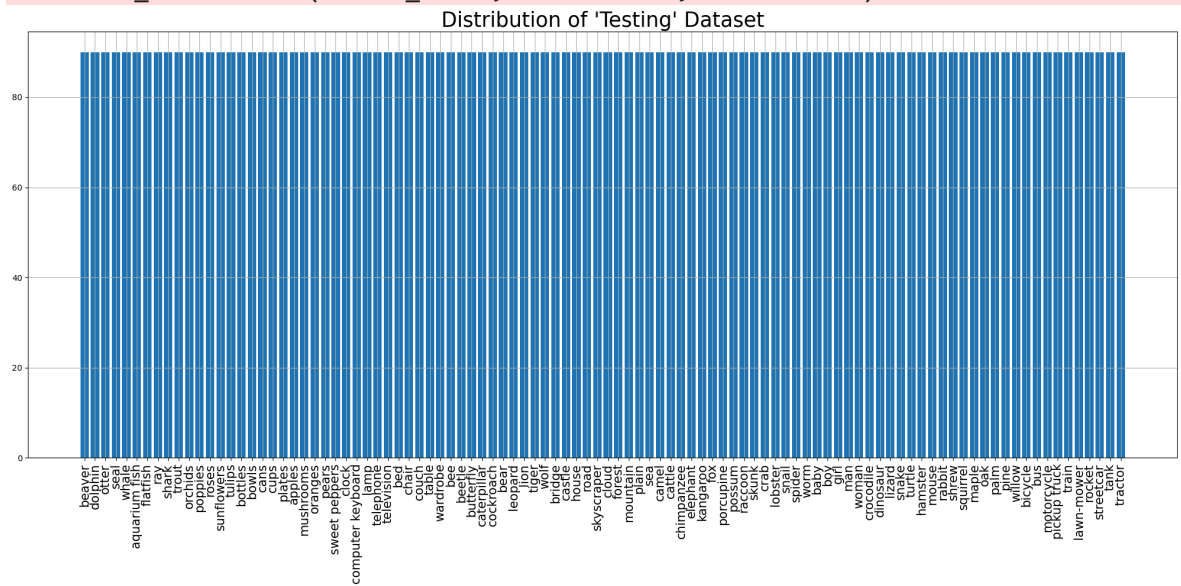
Distribution of 'Training' Dataset



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



```
<ipython-input-4-78561581887d>:21: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.  
ax.set_xticklabels(labels_names, rotation=90, fontsize=15)
```



Training dataset:

```
<_ConcatenateDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Validation dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

Testing dataset:

```
<_PrefetchDataset element_spec=(TensorSpec(shape=(None, 224, 224, 3), dtype=tf.float32, name=None), TensorSpec(shape=(None, 100), dtype=tf.float32, name=None))>
```

Image shape: (64, 224, 224, 3)

Label shape: (64, 100)

```
<class 'tuple'>
```

2

```
<class 'tensorflow.python.framework.ops.EagerTensor'> <class 'tensorflow.python.framework.ops.EagerTensor'>
```

(64, 224, 224, 3)

(64, 100)

Model input shape: (None, 224, 224, 3)

Model output shape: (None, 100)

2

```
<class 'numpy.ndarray'> <class 'numpy.ndarray'>
```

(64, 224, 224, 3) (64, 100)

Model: "sequential_7"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_7 (GlobalAveragePooling2D)	(None , 2048)
dense_14 (Dense)	(None , 512)
dropout_7 (Dropout)	(None , 512)
dense_15 (Dense)	(None , 100)

Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/25
1312/1314 ————— **0s** 27ms/step - accuracy: 0.3853 - f1_score: 0.3806
- loss: 2.5243 - precision: 0.6891
Epoch 1: val_loss improved from inf to 1.39970, saving model to best_model_v3.keras
1314/1314 ————— **56s** 35ms/step - accuracy: 0.3855 - f1_score: 0.3808
- loss: 2.5232 - precision: 0.6892 - val_accuracy: 0.6044 - val_f1_score: 0.5999
- val_loss: 1.3997 - val_precision: 0.8074
Epoch 2/25
1314/1314 ————— **0s** 24ms/step - accuracy: 0.5628 - f1_score: 0.5594
- loss: 1.5709 - precision: 0.7598
Epoch 2: val_loss improved from 1.39970 to 1.33181, saving model to best_model_v3.keras
1314/1314 ————— **36s** 27ms/step - accuracy: 0.5628 - f1_score: 0.5594
- loss: 1.5709 - precision: 0.7598 - val_accuracy: 0.6269 - val_f1_score: 0.6269
- val_loss: 1.3318 - val_precision: 0.8146
Epoch 3/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.6056 - f1_score: 0.6029
- loss: 1.3928 - precision: 0.7782
Epoch 3: val_loss improved from 1.33181 to 1.31399, saving model to best_model_v3.keras
1314/1314 ————— **36s** 27ms/step - accuracy: 0.6056 - f1_score: 0.6029
- loss: 1.3928 - precision: 0.7782 - val_accuracy: 0.6302 - val_f1_score: 0.6293
- val_loss: 1.3140 - val_precision: 0.7983
Epoch 4/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.6298 - f1_score: 0.6273
- loss: 1.2841 - precision: 0.7893
Epoch 4: val_loss improved from 1.31399 to 1.30608, saving model to best_model_v3.keras
1314/1314 ————— **36s** 28ms/step - accuracy: 0.6298 - f1_score: 0.6273
- loss: 1.2841 - precision: 0.7893 - val_accuracy: 0.6351 - val_f1_score: 0.6336
- val_loss: 1.3061 - val_precision: 0.7951
Epoch 5/25
1314/1314 ————— **0s** 24ms/step - accuracy: 0.6486 - f1_score: 0.6464
- loss: 1.2032 - precision: 0.7984
Epoch 5: val_loss improved from 1.30608 to 1.29241, saving model to best_model_v3.keras
1314/1314 ————— **36s** 27ms/step - accuracy: 0.6486 - f1_score: 0.6464
- loss: 1.2032 - precision: 0.7984 - val_accuracy: 0.6383 - val_f1_score: 0.6370
- val_loss: 1.2924 - val_precision: 0.7867
Epoch 6/25
1314/1314 ————— **0s** 24ms/step - accuracy: 0.6644 - f1_score: 0.6624
- loss: 1.1272 - precision: 0.8031
Epoch 6: val_loss did not improve from 1.29241
1314/1314 ————— **35s** 27ms/step - accuracy: 0.6644 - f1_score: 0.6623
- loss: 1.1273 - precision: 0.8031 - val_accuracy: 0.6432 - val_f1_score: 0.6417
- val_loss: 1.3053 - val_precision: 0.7851
Epoch 7/25
1313/1314 ————— **0s** 24ms/step - accuracy: 0.6809 - f1_score: 0.6790
- loss: 1.0660 - precision: 0.8116
Epoch 7: val_loss did not improve from 1.29241
1314/1314 ————— **35s** 27ms/step - accuracy: 0.6809 - f1_score: 0.6790
- loss: 1.0660 - precision: 0.8116 - val_accuracy: 0.6448 - val_f1_score: 0.6430
- val_loss: 1.3016 - val_precision: 0.7889
Epoch 8/25
1314/1314 ————— **0s** 24ms/step - accuracy: 0.6919 - f1_score: 0.6901
- loss: 1.0191 - precision: 0.8167
Epoch 8: val_loss did not improve from 1.29241
1314/1314 ————— **35s** 27ms/step - accuracy: 0.6919 - f1_score: 0.6901
- loss: 1.0191 - precision: 0.8167 - val_accuracy: 0.6472 - val_f1_score: 0.6471

3 - val_loss: 1.3049 - val_precision: 0.7836

Epoch 9/25

1314/1314 ————— 0s 24ms/step - accuracy: 0.7080 - f1_score: 0.7063
- loss: 0.9669 - precision: 0.8234

Epoch 9: val_loss did not improve from 1.29241

1314/1314 ————— 35s 27ms/step - accuracy: 0.7080 - f1_score: 0.706

3 - loss: 0.9670 - precision: 0.8234 - val_accuracy: 0.6436 - val_f1_score: 0.642

5 - val_loss: 1.3195 - val_precision: 0.7767

Epoch 10/25

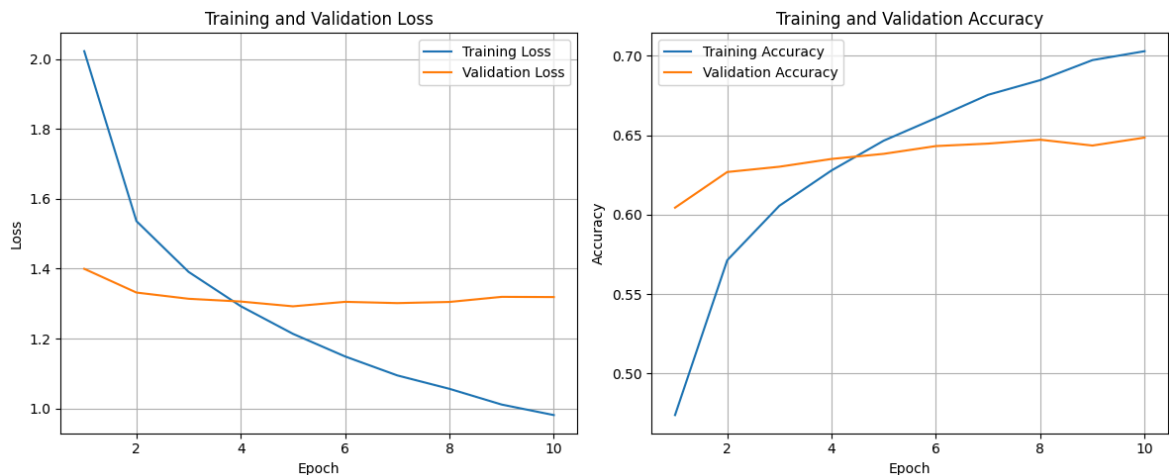
1313/1314 ————— 0s 24ms/step - accuracy: 0.7142 - f1_score: 0.7126
- loss: 0.9356 - precision: 0.8250

Epoch 10: val_loss did not improve from 1.29241

1314/1314 ————— 35s 26ms/step - accuracy: 0.7142 - f1_score: 0.712

5 - loss: 0.9357 - precision: 0.8249 - val_accuracy: 0.6486 - val_f1_score: 0.647

3 - val_loss: 1.3189 - val_precision: 0.7719



141/141 ————— 11s 38ms/step - accuracy: 0.6366 - f1_score: 0.6285
- loss: 1.3105 - precision: 0.7859

Test Accuracy: 64.27%

Test Loss: 1.2877730131149292

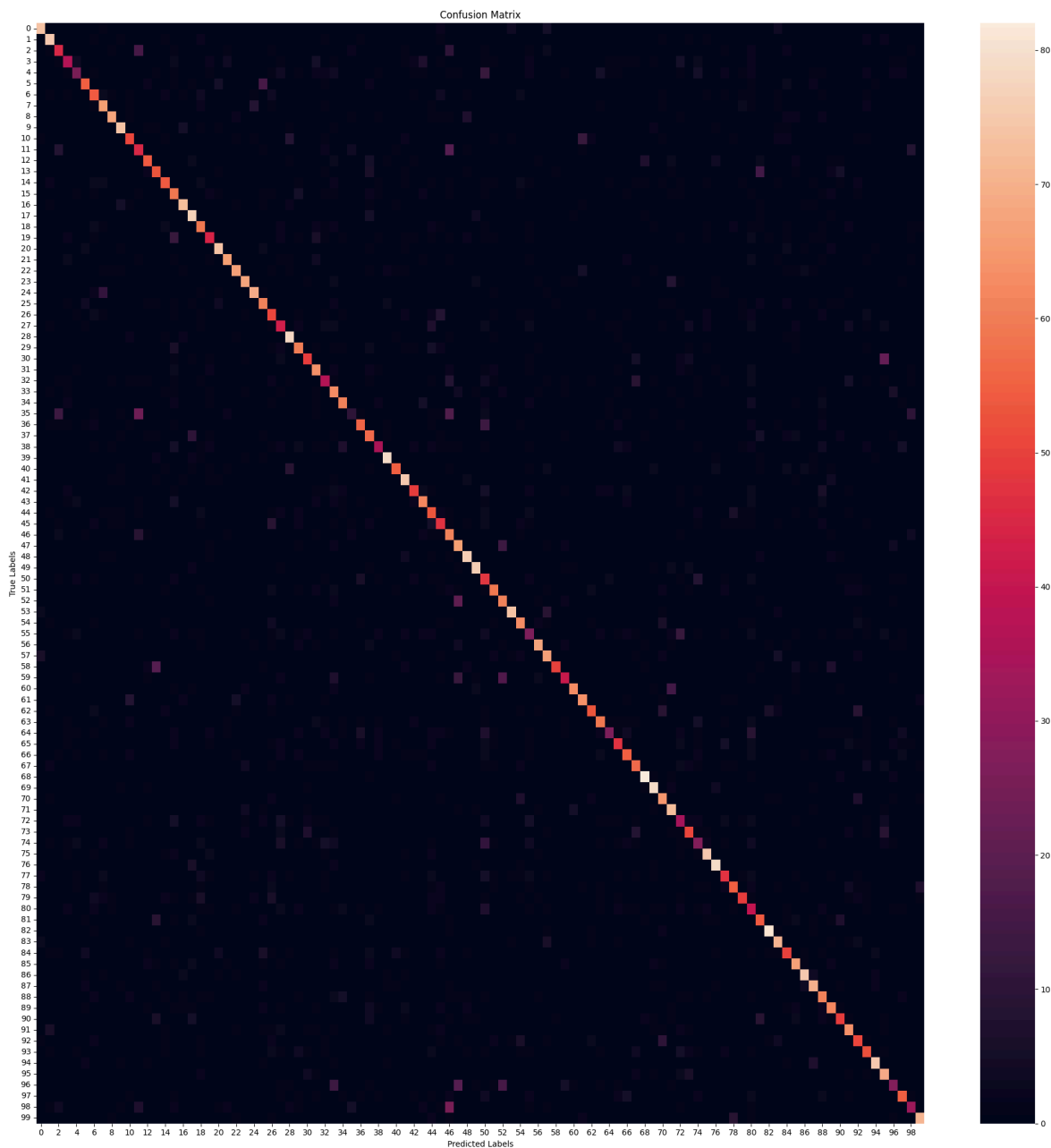
Test Precision: 79.15%

Test F1 Scores (Per Class): [82.68156 79.365074 52.941174 48.36601 35.460987 6
6.26506 58.69565

69.47368	79.99999	81.31868	61.445778	45.45454	66.25767	56.989246
65.060234	53.88127	81.11111	73.684204	59.89847	53.987724	80.43478
77.64706	77.456635	77.27273	79.768776	62.5641	50.495045	45.74468
78.39195	64.17112	61.25	65.64102	45.67901	53.164555	69.31818
17.3913	67.46988	56.15763	47.61904	87.29281	69.62025	77.31957
62.820507	69.66292	52.216743	50.810806	47.47081	59.192818	79.99999
83.51647	38.247005	69.005844	60.999996	85.549126	67.37968	31.70731
78.61271	69.74358	69.93006	53.59477	77.77777	73.44632	67.08861
68.604645	40.310078	58.024685	62.499992	60.869556	87.23404	82.291664
65.65657	70.87379	36.756752	59.30232	33.749996	81.96721	84.04254
57.317066	65.868256	64.052284	35.87443	57.754005	89.01098	73.57513
59.171593	70.526306	82.872925	74.8663	61.928932	67.02702	59.39393
68.449196	57.303364	58.426964	85.71427	65.09434	37.499992	65.088745
42.10526	81.60919					

Average Test F1 Scores:64.06

282/282 ————— 12s 28ms/step



	precision	recall	f1-score	support
beaver	0.82	0.82	0.82	90
dolphin	0.76	0.83	0.79	90
otter	0.56	0.50	0.53	90
seal	0.59	0.41	0.48	90
whale	0.49	0.28	0.35	90
aquarium fish	0.72	0.61	0.66	90
flatfish	0.57	0.60	0.59	90
ray	0.66	0.73	0.69	90
shark	0.85	0.76	0.80	90
trout	0.80	0.82	0.81	90
orchids	0.66	0.57	0.61	90
poppies	0.42	0.50	0.45	90
roses	0.74	0.60	0.66	90
sunflowers	0.55	0.59	0.57	90
tulips	0.71	0.60	0.65	90
bottles	0.46	0.66	0.54	90
bowls	0.81	0.81	0.81	90
cans	0.65	0.86	0.74	90
cups	0.55	0.66	0.60	90
plates	0.60	0.49	0.54	90
apples	0.79	0.82	0.80	90
mushrooms	0.82	0.73	0.78	90
oranges	0.81	0.74	0.77	90
pears	0.79	0.76	0.77	90
sweet peppers	0.83	0.77	0.80	90
clock	0.58	0.68	0.63	90
computer keyboard	0.46	0.57	0.51	90
lamp	0.44	0.48	0.46	90
telephone	0.72	0.87	0.78	90
television	0.62	0.67	0.64	90
bed	0.70	0.54	0.61	90
chair	0.61	0.71	0.66	90
couch	0.51	0.41	0.46	90
table	0.43	0.70	0.53	90
wardrobe	0.71	0.68	0.69	90
bee	0.40	0.11	0.17	90
beetle	0.73	0.62	0.67	90
butterfly	0.50	0.63	0.56	90
caterpillar	0.61	0.39	0.48	90
cockroach	0.87	0.88	0.87	90
bear	0.81	0.61	0.70	90
leopard	0.72	0.83	0.77	90
lion	0.74	0.54	0.63	90
tiger	0.70	0.69	0.70	90
wolf	0.47	0.59	0.52	90
bridge	0.49	0.52	0.51	90
castle	0.37	0.68	0.47	90
house	0.50	0.73	0.59	90
road	0.76	0.84	0.80	90
skyscraper	0.83	0.84	0.84	90
cloud	0.30	0.53	0.38	90
forest	0.73	0.66	0.69	90
mountain	0.55	0.68	0.61	90
plain	0.89	0.82	0.86	90
sea	0.65	0.70	0.67	90
camel	0.35	0.29	0.32	90
cattle	0.82	0.76	0.79	90
chimpanzee	0.64	0.74	0.69	90

elephant	0.94	0.56	0.70	90
kangaroo	0.65	0.46	0.54	90
fox	0.88	0.70	0.78	90
porcupine	0.74	0.71	0.73	90
possum	0.78	0.59	0.67	90
raccoon	0.72	0.66	0.69	90
skunk	0.67	0.29	0.40	90
crab	0.65	0.52	0.58	90
lobster	0.64	0.61	0.62	90
snail	0.60	0.63	0.62	90
spider	0.84	0.91	0.87	90
worm	0.77	0.88	0.82	90
baby	0.60	0.72	0.65	90
boy	0.63	0.81	0.71	90
girl	0.36	0.38	0.37	90
man	0.63	0.57	0.60	90
woman	0.39	0.30	0.34	90
crocodile	0.81	0.83	0.82	90
dinosaur	0.81	0.88	0.84	90
lizard	0.64	0.52	0.57	90
snake	0.71	0.61	0.66	90
turtle	0.78	0.54	0.64	90
hamster	0.30	0.44	0.36	90
mouse	0.56	0.60	0.58	90
rabbit	0.88	0.90	0.89	90
shrew	0.69	0.79	0.74	90
squirrel	0.63	0.56	0.59	90
maple	0.67	0.74	0.71	90
oak	0.82	0.83	0.83	90
palm	0.72	0.78	0.75	90
pine	0.57	0.68	0.62	90
willow	0.65	0.69	0.67	90
bicycle	0.65	0.54	0.59	90
bus	0.66	0.71	0.68	90
motorcycle	0.58	0.57	0.57	90
pickup truck	0.59	0.58	0.58	90
train	0.88	0.83	0.86	90
lawn-mower	0.57	0.77	0.65	90
rocket	0.50	0.30	0.38	90
streetcar	0.70	0.61	0.65	90
tank	0.52	0.36	0.42	90
tractor	0.85	0.79	0.82	90
accuracy			0.64	9000
macro avg	0.65	0.64	0.64	9000
weighted avg	0.65	0.64	0.64	9000

Model: "sequential_7"

Layer (type)	Output Shape
resnet50v2 (Functional)	(None , 7 , 7 , 2048)
global_average_pooling2d_7 (GlobalAveragePooling2D)	(None , 2048)
dense_14 (Dense)	(None , 512)
dropout_7 (Dropout)	(None , 512)
dense_15 (Dense)	(None , 100)



Total params: [24,665,188](#) (94.09 MB)

Trainable params: [1,100,388](#) (4.20 MB)

Non-trainable params: [23,564,800](#) (89.89 MB)

Epoch 1/15
1971/1971 ————— **0s** 26ms/step - accuracy: 0.6780 - f1_score: 0.6761
- loss: 1.0871 - precision: 0.8185
Epoch 1: val_loss improved from inf to 1.24744, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **73s** 31ms/step - accuracy: 0.6780 - f1_score: 0.6761 - loss: 1.0871 - precision: 0.8185 - val_accuracy: 0.6522 - val_f1_score: 0.6510 - val_loss: 1.2474 - val_precision: 0.7970
Epoch 2/15
1970/1971 ————— **0s** 25ms/step - accuracy: 0.7049 - f1_score: 0.7028 - loss: 0.9853 - precision: 0.8332
Epoch 2: val_loss improved from 1.24744 to 1.24231, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **53s** 27ms/step - accuracy: 0.7049 - f1_score: 0.7028 - loss: 0.9853 - precision: 0.8332 - val_accuracy: 0.6566 - val_f1_score: 0.6557 - val_loss: 1.2423 - val_precision: 0.7936
Epoch 3/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.7120 - f1_score: 0.7102 - loss: 0.9631 - precision: 0.8397
Epoch 3: val_loss improved from 1.24231 to 1.23910, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **52s** 27ms/step - accuracy: 0.7120 - f1_score: 0.7102 - loss: 0.9631 - precision: 0.8397 - val_accuracy: 0.6584 - val_f1_score: 0.6576 - val_loss: 1.2391 - val_precision: 0.7933
Epoch 4/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.7178 - f1_score: 0.7158 - loss: 0.9401 - precision: 0.8404
Epoch 4: val_loss improved from 1.23910 to 1.23705, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **52s** 27ms/step - accuracy: 0.7178 - f1_score: 0.7158 - loss: 0.9401 - precision: 0.8404 - val_accuracy: 0.6602 - val_f1_score: 0.6593 - val_loss: 1.2371 - val_precision: 0.7938
Epoch 5/15
1971/1971 ————— **0s** 25ms/step - accuracy: 0.7203 - f1_score: 0.7185 - loss: 0.9270 - precision: 0.8429
Epoch 5: val_loss improved from 1.23705 to 1.23483, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **53s** 27ms/step - accuracy: 0.7203 - f1_score: 0.7185 - loss: 0.9270 - precision: 0.8429 - val_accuracy: 0.6591 - val_f1_score: 0.6584 - val_loss: 1.2348 - val_precision: 0.7953
Epoch 6/15
1969/1971 ————— **0s** 24ms/step - accuracy: 0.7222 - f1_score: 0.7204 - loss: 0.9198 - precision: 0.8451
Epoch 6: val_loss improved from 1.23483 to 1.23281, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **52s** 27ms/step - accuracy: 0.7222 - f1_score: 0.7204 - loss: 0.9198 - precision: 0.8451 - val_accuracy: 0.6603 - val_f1_score: 0.6595 - val_loss: 1.2328 - val_precision: 0.7956
Epoch 7/15
1970/1971 ————— **0s** 25ms/step - accuracy: 0.7266 - f1_score: 0.7251 - loss: 0.9008 - precision: 0.8492
Epoch 7: val_loss improved from 1.23281 to 1.23056, saving model to best_model_fine_tune_v3.keras
1971/1971 ————— **53s** 27ms/step - accuracy: 0.7266 - f1_score: 0.7251 - loss: 0.9008 - precision: 0.8492 - val_accuracy: 0.6609 - val_f1_score: 0.6602 - val_loss: 1.2306 - val_precision: 0.7965
Epoch 8/15
1970/1971 ————— **0s** 24ms/step - accuracy: 0.7296 - f1_score: 0.7281 - loss: 0.8923 - precision: 0.8496
Epoch 8: val_loss improved from 1.23056 to 1.22925, saving model to best_model_fine_tune_v3.keras

ne_tune_v3.keras

1971/1971 ————— 52s 27ms/step - accuracy: 0.7296 - f1_score: 0.7281 - loss: 0.8923 - precision: 0.8496 - val_accuracy: 0.6614 - val_f1_score: 0.6609 - val_loss: 1.2293 - val_precision: 0.7973

Epoch 9/15

1970/1971 ————— 0s 24ms/step - accuracy: 0.7323 - f1_score: 0.7311 - loss: 0.8774 - precision: 0.8529

Epoch 9: val_loss improved from 1.22925 to 1.22797, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 52s 27ms/step - accuracy: 0.7323 - f1_score: 0.7311 - loss: 0.8774 - precision: 0.8529 - val_accuracy: 0.6610 - val_f1_score: 0.6605 - val_loss: 1.2280 - val_precision: 0.7975

Epoch 10/15

1969/1971 ————— 0s 24ms/step - accuracy: 0.7344 - f1_score: 0.7329 - loss: 0.8713 - precision: 0.8543

Epoch 10: val_loss improved from 1.22797 to 1.22713, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 52s 26ms/step - accuracy: 0.7344 - f1_score: 0.7329 - loss: 0.8714 - precision: 0.8543 - val_accuracy: 0.6629 - val_f1_score: 0.6624 - val_loss: 1.2271 - val_precision: 0.7968

Epoch 11/15

1969/1971 ————— 0s 24ms/step - accuracy: 0.7385 - f1_score: 0.7370 - loss: 0.8614 - precision: 0.8568

Epoch 11: val_loss improved from 1.22713 to 1.22501, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 52s 26ms/step - accuracy: 0.7385 - f1_score: 0.7370 - loss: 0.8615 - precision: 0.8568 - val_accuracy: 0.6614 - val_f1_score: 0.6610 - val_loss: 1.2250 - val_precision: 0.7967

Epoch 12/15

1969/1971 ————— 0s 24ms/step - accuracy: 0.7386 - f1_score: 0.7371 - loss: 0.8596 - precision: 0.8580

Epoch 12: val_loss improved from 1.22501 to 1.22251, saving model to best_model_fine_tune_v3.keras

1971/1971 ————— 52s 26ms/step - accuracy: 0.7386 - f1_score: 0.7371 - loss: 0.8597 - precision: 0.8579 - val_accuracy: 0.6622 - val_f1_score: 0.6616 - val_loss: 1.2225 - val_precision: 0.7986

Epoch 13/15

1971/1971 ————— 0s 24ms/step - accuracy: 0.7436 - f1_score: 0.7419 - loss: 0.8443 - precision: 0.8595

Epoch 13: val_loss did not improve from 1.22251

1971/1971 ————— 51s 26ms/step - accuracy: 0.7436 - f1_score: 0.7419 - loss: 0.8443 - precision: 0.8595 - val_accuracy: 0.6641 - val_f1_score: 0.6637 - val_loss: 1.2227 - val_precision: 0.7985

Epoch 14/15

1969/1971 ————— 0s 24ms/step - accuracy: 0.7442 - f1_score: 0.7426 - loss: 0.8453 - precision: 0.8618

Epoch 14: val_loss improved from 1.22251 to 1.22206, saving model to best_model_fine_tune_v3.keras

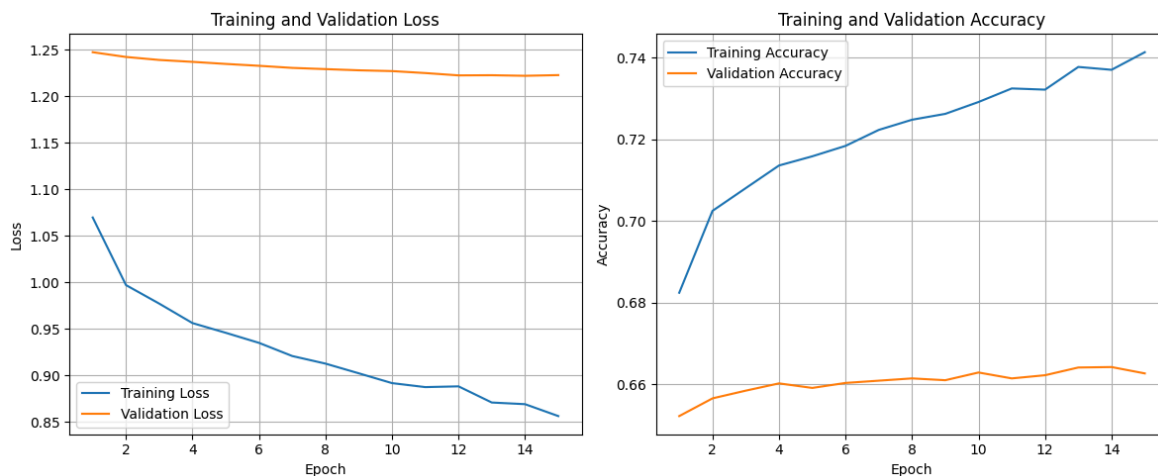
1971/1971 ————— 52s 26ms/step - accuracy: 0.7442 - f1_score: 0.7426 - loss: 0.8453 - precision: 0.8618 - val_accuracy: 0.6642 - val_f1_score: 0.6635 - val_loss: 1.2221 - val_precision: 0.7983

Epoch 15/15

1969/1971 ————— 0s 24ms/step - accuracy: 0.7481 - f1_score: 0.7467 - loss: 0.8332 - precision: 0.8642

Epoch 15: val_loss did not improve from 1.22206

1971/1971 ————— 51s 26ms/step - accuracy: 0.7481 - f1_score: 0.7467 - loss: 0.8333 - precision: 0.8642 - val_accuracy: 0.6627 - val_f1_score: 0.6620 - val_loss: 1.2228 - val_precision: 0.7975



141/141 ————— **11s** 38ms/step - accuracy: 0.6574 - f1_score: 0.6494

- loss: 1.2309 - precision: 0.7864

Test Accuracy: 66.71%

Test Loss: 1.2028621435165405

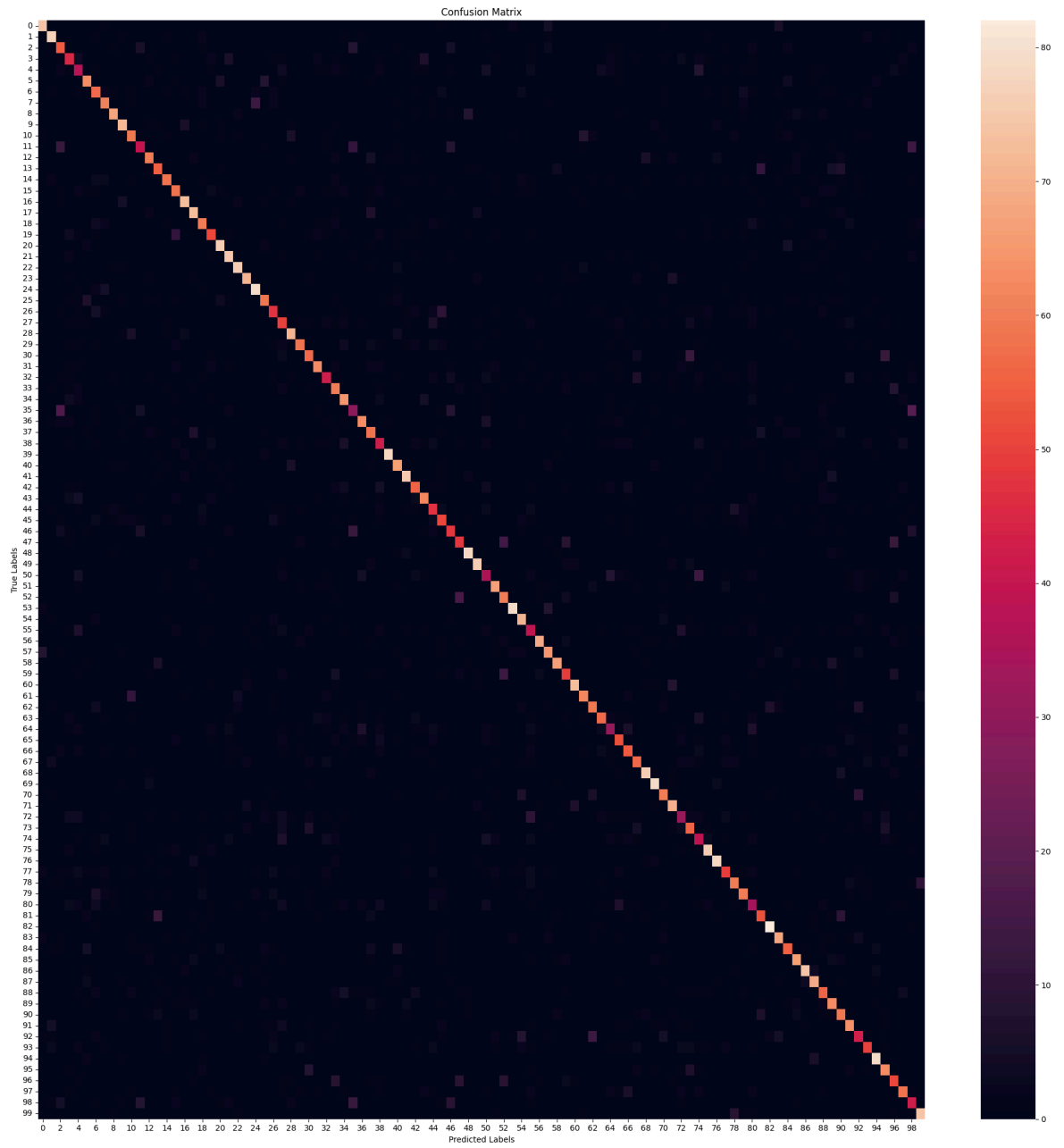
Test Precision: 79.37%

Test F1 Scores (Per Class): [81.767944 79.79274 54.27136 50.56179 40.86021 69.23077 57.868015

68.53932	79.99999	78.68852	64.83515	50.31446	68.57143	63.218384
69.047615	62.702698	80.21977	76.8421	65.217384	59.30232	81.72042
77.31957	82.16216	82.08092	82.7225	67.04545	55.294113	49.751236
78.02197	70.731705	64.04494	74.556206	51.533733	60.301506	67.357506
34.939754	71.99999	60.416664	47.999996	90.69767	69.47368	81.52173
62.92134	71.67629	50.810806	55.43478	53.631275	56.470577	84.32432
83.51647	41.176464	74.03315	63.157898	88.76403	71.71716	42.857136
80.701744	69.89246	79.28994	56.97674	83.42856	73.37277	66.29213
67.46988	39.240498	58.426964	59.016384	64.04494	83.51647	84.491974
67.415726	75.268814	39.49044	59.574467	41.711224	82.608696	86.18784
58.33332	68.181816	68.926544	38.20224	59.428566	88.64864	74.31693
64.32748	72.52747	82.22221	74.999985	62.569828	65.284966	60.51281
69.18918	50.887566	59.171593	86.33879	67.02127	54.545444	65.16853
44.680847	80.87432					

Average Test F1 Scores:66.59

282/282 ————— **13s** 34ms/step



	precision	recall	f1-score	support
beaver	0.82	0.82	0.82	90
dolphin	0.75	0.86	0.80	90
otter	0.50	0.60	0.54	90
seal	0.51	0.50	0.51	90
whale	0.40	0.42	0.41	90
aquarium fish	0.68	0.70	0.69	90
flatfish	0.53	0.63	0.58	90
ray	0.69	0.68	0.69	90
shark	0.85	0.76	0.80	90
trout	0.77	0.80	0.79	90
orchids	0.64	0.66	0.65	90
poppies	0.58	0.44	0.50	90
roses	0.71	0.67	0.69	90
sunflowers	0.65	0.61	0.63	90
tulips	0.73	0.64	0.69	90
bottles	0.61	0.64	0.63	90
bowls	0.78	0.81	0.80	90
cans	0.73	0.81	0.77	90
cups	0.64	0.67	0.65	90
plates	0.62	0.57	0.59	90
apples	0.79	0.84	0.82	90
mushrooms	0.72	0.83	0.77	90
oranges	0.80	0.84	0.82	90
pears	0.86	0.79	0.82	90
sweet peppers	0.78	0.88	0.83	90
clock	0.69	0.66	0.67	90
computer keyboard	0.59	0.52	0.55	90
lamp	0.45	0.56	0.50	90
telephone	0.77	0.79	0.78	90
television	0.78	0.64	0.71	90
bed	0.66	0.63	0.64	90
chair	0.80	0.70	0.75	90
couch	0.58	0.47	0.52	90
table	0.55	0.67	0.60	90
wardrobe	0.63	0.72	0.67	90
bee	0.38	0.32	0.35	90
beetle	0.74	0.70	0.72	90
butterfly	0.57	0.64	0.60	90
caterpillar	0.49	0.47	0.48	90
cockroach	0.95	0.87	0.91	90
bear	0.66	0.73	0.69	90
leopard	0.80	0.83	0.82	90
lion	0.64	0.62	0.63	90
tiger	0.75	0.69	0.72	90
wolf	0.49	0.52	0.51	90
bridge	0.54	0.57	0.55	90
castle	0.54	0.53	0.54	90
house	0.60	0.53	0.56	90
road	0.82	0.87	0.84	90
skyscraper	0.83	0.84	0.84	90
cloud	0.44	0.39	0.41	90
forest	0.74	0.74	0.74	90
mountain	0.60	0.67	0.63	90
plain	0.90	0.88	0.89	90
sea	0.66	0.79	0.72	90
camel	0.42	0.43	0.43	90
cattle	0.85	0.77	0.81	90
chimpanzee	0.68	0.72	0.70	90

elephant	0.85	0.74	0.79	90
kangaroo	0.60	0.54	0.57	90
fox	0.86	0.81	0.83	90
porcupine	0.78	0.69	0.73	90
possum	0.67	0.66	0.66	90
raccoon	0.74	0.63	0.68	90
skunk	0.46	0.34	0.39	90
crab	0.59	0.58	0.58	90
lobster	0.58	0.60	0.59	90
snail	0.65	0.63	0.64	90
spider	0.83	0.84	0.84	90
worm	0.81	0.88	0.84	90
baby	0.68	0.67	0.67	90
boy	0.73	0.78	0.75	90
girl	0.46	0.34	0.39	90
man	0.57	0.62	0.60	90
woman	0.40	0.43	0.42	90
crocodile	0.81	0.84	0.83	90
dinosaur	0.86	0.87	0.86	90
lizard	0.63	0.54	0.58	90
snake	0.70	0.67	0.68	90
turtle	0.70	0.67	0.68	90
hamster	0.39	0.38	0.38	90
mouse	0.61	0.58	0.59	90
rabbit	0.86	0.91	0.89	90
shrew	0.73	0.76	0.74	90
squirrel	0.68	0.61	0.64	90
maple	0.72	0.73	0.73	90
oak	0.82	0.82	0.82	90
palm	0.73	0.77	0.75	90
pine	0.63	0.62	0.63	90
willow	0.61	0.70	0.65	90
bicycle	0.56	0.66	0.61	90
bus	0.67	0.71	0.69	90
motorcycle	0.54	0.48	0.51	90
pickup truck	0.63	0.56	0.59	90
train	0.85	0.88	0.86	90
lawn-mower	0.64	0.70	0.67	90
rocket	0.53	0.57	0.55	90
streetcar	0.66	0.64	0.65	90
tank	0.43	0.47	0.45	90
tractor	0.80	0.82	0.81	90
accuracy			0.67	9000
macro avg	0.67	0.67	0.67	9000
weighted avg	0.67	0.67	0.67	9000

In []: